

Orale Algoritmi e strutture dati

Cornelio

Data : 26/01/2026

Programmazione (Cabodi) :

- Fare una funzione che riconosce se un grafo è un DAG

1. Funzione per riconoscere se un grafo è un DAG

- **Concetto:** Un DAG è un Grafo Diretto Aciclico. Si usa una visita DFS con l'array dei 3 colori: 0=Bianco (non visitato), 1=Grigio (in visita), 2=Nero (completato). L'uso dei 3 colori è fondamentale: se usassimo solo "visitato/non visitato", non sapremmo distinguere un *back-edge* (che torna a un nodo nel percorso corrente, creando un ciclo) da un innocuo *cross-edge* (che punta a un ramo già esplorato in un'altra iterazione).
- **Condizione di ciclo:** Se durante la DFS trovo un nodo Grigio, ho la certezza matematica di un ciclo, quindi NON è un DAG.
- **Complessità:** Tempo $O(V+E)$ perché esploriamo ogni vertice e arco al massimo una volta. Spazio $O(V)$ per l'array dei colori e lo stack della ricorsione.

```
#define WHITE 0
#define GRAY 1
#define BLACK 2

// Assumo struttura Graph con adjLists (liste di adiacenza) e numVertices

bool hasCycle(Graph* g, int v, int* color) {
    color[v] = GRAY; // Nodo in elaborazione (sullo stack)

    for (Node* temp = g->adjLists[v]; temp != NULL; temp = temp->next) {
        int adj = temp->vertex;
        if (color[adj] == GRAY) return true; // Ciclo trovato!
        if (color[adj] == WHITE && hasCycle(g, adj, color)) return true;
    }

    color[v] = BLACK; // Elaborazione finita
    return false;
}

bool isDAG(Graph* g) {
    int* color = calloc(g->numVertices, sizeof(int)); // Inizializza a WHITE
```

```

for (int i = 0; i < g->numVertices; i++) {
    // Avvia DFS per ogni componente connessa
    if (color[i] == WHITE && hasCycle(g, i, color)) {
        free(color);
        return false; // Trovato ciclo, non è DAG
    }
}
free(color);
return true; // Nessun ciclo, è DAG
}

```

Teoria (Camurati) :

● Algoritmo di Karatsuba e differenze con algoritmo standard, anche efficienze ecc...

1. Algoritmo di Karatsuba e differenze con standard

- **Algoritmo Standard:** La moltiplicazione classica in colonna è $O(n^2)$. Un Divide et Impera banale divide i numeri a metà facendo 4 sottomoltiplicazioni, con equazione $T(n) = 4T(n/2) + O(n)$, che per il Master Theorem rimane $O(n^2)$.
- **Il trucco di Karatsuba:** Usa un'astuzia algebrica per calcolare i prodotti incrociati usando una sola moltiplicazione delle somme divide i due numeri grandi a metà, riducendo la grande moltiplicazione in moltiplicazioni più piccole., scambiando di fatto una costosa chiamata ricorsiva con delle semplici addizioni $O(n)$. Le moltiplicazioni ricorsive scendono così da 4 a **3**.
- **Efficienza:** L'equazione diventa $T(n) = 3T(n/2) + O(n)$. Per il Master Theorem la complessità asintotica scende a $O(n^{\log_2(3)})$, circa **$O(n^{1.58})$** .

● Si può applicare un similare di Karatsuba con le matrici e in che modo, spiegare differenze di complessità con moltiplicazione di matrici $n \times n$ normale che ha complessità $O(n^3)$

2. Karatsuba per Matrici (Algoritmo di Strassen)

- **Standard vs Strassen:** La moltiplicazione riga per colonna è $O(n^3)$. Il Divide et Impera divide le matrici in blocchi e fa 8 moltiplicazioni: $T(n) = 8T(n/2) + O(n^2)$, restando a $O(n^3)$. Strassen applica la logica di Karatsuba alle matrici: usa un complesso set di somme e sottrazioni tra blocchi per scendere da 8 a **7 moltiplicazioni**.
- **Differenza di complessità:** L'equazione diventa $T(n) = 7T(n/2) + O(n^2)$, che per il Master Theorem porta a $O(n^{\log_2(7)})$, circa **$O(n^{2.81})$** , rendendolo asintoticamente più veloce di $O(n^3)$.

- **Nota pratica per l'orale:** Avendo molte addizioni extra, Strassen ha una costante di tempo molto alta, quindi conviene rispetto al metodo standard solo per matrici di dimensioni enormi.

● Funzioni tail recursive

3. Funzioni tail recursive

- **Cosa sono:** Funzioni in cui la chiamata ricorsiva è l'esatta ultima operazione eseguita. Ad esempio, il calcolo del fattoriale passando l'accumulatore come parametro è tail recursive, mentre "return n * fattoriale(n-1)" non lo è, perché la moltiplicazione deve essere calcolata dopo il ritorno della ricorsione.
 - **Efficienza e Tail Call Optimization (TCO):** La TCO è un'ottimizzazione fatta dal compilatore. Quando il compilatore si accorge che la funzione chiamante non deve fare più nessuna operazione dopo la chiamata ricorsiva (Tail Recursion), capisce che non serve salvare lo stato attuale in memoria. Invece di creare un nuovo "Stack Frame" per la nuova chiamata, sovrascrive semplicemente i parametri nel frame corrente.
 - **Vantaggio:** A livello macchina, il compilatore trasforma la ricorsione in un banale ciclo while. La complessità spaziale (memoria usata) crolla da $O(n)$ a **$O(1)$** , azzerando il rischio di errore per Stack Overflow.
-

Data : 27 -> 30L (+1.75 lab) (26/01/2026)

Programmazione (Cabodi) :

- Dato un grafo orientato e un insieme (vettore) dei suoi vertici calcolare se è una componente fortemente connessa, ha detto che non serve trovare tutte le componenti connesse con kosaraju ma solo verificare se è una componente fortemente connessa attraverso un algoritmo semplice

1. Verificare se un vettore di vertici è una Componente Fortemente Connessa (SCC)

- **Concetto:** In un grafo orientato, un sottoinsieme di vertici è fortemente connesso se ogni nodo del sottoinsieme può raggiungere tutti gli altri nodi del sottoinsieme, usando solo percorsi interni ad esso.
- **Logica dell'algoritmo "semplice":** Invece di usare Kosaraju (che richiede di calcolare il grafo trasposto), iteriamo su ogni nodo del vettore. Da ogni nodo lanciamo una visita DFS vincolata: può esplorare solo i nodi che fanno parte del vettore. Se per ogni lancio della DFS riusciamo a visitare esattamente tutti i nodi del vettore, allora il sottoinsieme è fortemente connesso.

- **Nota sulla massimalità:** La definizione formale di SCC richiede anche che l'insieme sia "massimale" (non espandibile). In sede d'esame, per "algoritmo semplice", Cabodi si accontenta di verificare la forte connessione reciproca tra i nodi passati.
- **Complessità:** Tempo $O(K * (K + E_k))$, dove K è la dimensione del vettore ed E_k sono gli archi tra i nodi del vettore. Spazio $O(V)$ per gli array di supporto.

```
#include <stdbool.h>
#include <stdlib.h>

// Funzione ricorsiva di visita DFS vincolata al sottoinsieme
void dfs_subset(Graph* g, int v, bool* visited, bool* in_subset, int* count) {
    visited[v] = true; // Segno il nodo come visitato
    (*count)++;        // Incremento il contatore dei nodi raggiunti

    // Scorro i nodi adiacenti
    for (Node* adj = g->adjlists[v]; adj != NULL; adj = adj->next) {
        int w = adj->vertex;

        // Procedo solo se il vicino appartiene al vettore iniziale E non è ancora
        visitato
        if (in_subset[w] && !visited[w]) {
            dfs_subset(g, w, visited, in_subset, count);
        }
    }
}

bool is_strongly_connected_subset(Graph* g, int* subset, int size) {
    // Array O(1) lookup per sapere se un nodo del grafo fa parte del vettore
    bool* in_subset = calloc(g->numVertices, sizeof(bool));
    for (int i = 0; i < size; i++) {
        in_subset[subset[i]] = true;
    }

    // Lancio una DFS da ogni singolo nodo del vettore
    for (int i = 0; i < size; i++) {
        bool* visited = calloc(g->numVertices, sizeof(bool));
        int count = 0;

        dfs_subset(g, subset[i], visited, in_subset, &count);

        free(visited);

        // Se partendo da subset[i] non ho raggiunto tutti gli altri nodi del
        vettore, non è una SCC
        if (count != size) {
            free(in_subset);
            return false;
        }
    }
}
```

```
free(in_subset);  
return true; // Tutti i nodi si raggiungono reciprocamente  
}
```

Teoria (Camurati) :

● Partizioni di un insieme, che modello del calcolo combinatorio utilizza (nessuno)

1. Partizioni di un insieme e modello di calcolo combinatorio

- **Definizione:** Una partizione di un insieme è una suddivisione dei suoi elementi in sottoinsiemi non vuoti e disgiunti, la cui unione ricompone l'insieme di partenza.
- **Modello combinatorio (Nessuno):** Le partizioni non rientrano nei classici modelli del calcolo combinatorio (Disposizioni, Permutazioni, Combinazioni). Il motivo è che i modelli classici si basano sull'estrarre o ordinare k elementi da n . Le partizioni, invece, raggruppano tutti gli n elementi in un numero di sottoinsiemi non specificato a priori, e l'ordine dei sottoinsiemi (o degli elementi al loro interno) non ha importanza.
- **Numeri di Bell e Stirling:** Per contare le partizioni si usano successioni specifiche. I Numeri di Stirling di seconda specie contano le partizioni di n elementi in esattamente k sottoinsiemi. I Numeri di Bell contano il totale assoluto di tutte le partizioni possibili di un insieme di n elementi (ottenuti sommando i numeri di Stirling su tutti i k).

● Come si rappresentano i grafi (matrice e lista delle adiacenze) e complessità spaziale

2. Rappresentazione dei grafi e complessità spaziale

- **Matrice di adiacenza:** È una matrice quadrata $V \times V$ (dove V è il numero dei vertici). La cella $[i][j]$ vale 1 (o il peso dell'arco) se esiste un arco dal nodo i al nodo j , 0 altrimenti.
 - **Complessità Spaziale:** $O(V^2)$. È inefficiente per la memoria se il grafo è "sparso" (pochi archi rispetto al massimo possibile), perché memorizza esplicitamente anche l'assenza di archi.
 - **Vantaggio:** Sapere se esiste l'arco (i, j) ha costo temporale $O(1)$.
- **Lista di adiacenza:** È un array di V elementi (uno per ogni vertice). Ogni elemento contiene un puntatore a una lista concatenata che memorizza solo i nodi direttamente adiacenti a quel vertice.
 - **Complessità Spaziale:** $O(V + E)$, dove E è il numero degli archi. È la rappresentazione ottimale e più usata, perché alloca memoria solo per gli archi che esistono realmente.

- **Vantaggio:** Scorrere tutti i vicini di un nodo è molto più veloce rispetto alla matrice, rendendo visite come DFS e BFS molto più efficienti.
-

Data : 25 (+1.75 lab) -> 30 (26/01/2026)

Programmazione (Cabodi) :

● Una griglia è rappresentata mediante una matrice in cui se la casella i,j è piena allora $matr[i][j]=1$ e se $matr[i][j]=0$ la casella è vuota. Se ci si trova in una casella piena → posso spostarmi verso le altre caselle a N,S,E,O (se i vertici in quelle direzioni esistono, ovvero nella matrice il loro valore è 1 e se rispettano le dimensioni della matrice). Scrivere una funzione che effettui una dfs completa della griglia tenendo presente questo modo per rappresentare i 'vertici' e gli archi (che sono impliciti).

1. DFS completa su una griglia con archi impliciti

- **Concetto:** La griglia è vista come un grafo dove i vertici sono le celle con valore 1. Gli archi non sono salvati in liste o matrici di adiacenza, ma calcolati al volo: da una cella (i, j) ci si muove verso Nord $(i-1, j)$, Sud $(i+1, j)$, Est $(i, j+1)$, Ovest $(i, j-1)$, a patto che non si esca dai bordi e che la cella di destinazione contenga 1.
- **Logica dell'algoritmo:** Serve una matrice booleana visited delle stesse dimensioni della griglia per non visitare la stessa cella due volte (evitando loop infiniti). Poiché il "grafo-griglia" potrebbe essere disconnesso (isole di 1 separate da 0), la DFS completa deve iterare su tutta la matrice e far partire una visita ricorsiva ogni volta che trova un 1 non ancora visitato.
- **Complessità:** Tempo $O(R * C)$, dove R sono le righe e C le colonne, poiché ogni cella viene visitata al massimo una volta. Spazio $O(R * C)$ per la matrice dei visitati e per lo stack delle chiamate ricorsive (nel caso pessimo di un percorso a serpentina che copre tutta la griglia).

```
// Array direzionali per muoversi agilmente a N, S, E, O senza scrivere 4 if
int row_dir[] = {-1, 1, 0, 0};
int col_dir[] = {0, 0, 1, -1};

// Funzione ricorsiva che esplora la singola componente connessa (l'isola di 1)
void dfs_visit(int** grid, int R, int C, int r, int c, bool** visited) {
    // Segno la cella corrente come visitata
    visited[r][c] = true;

    // Provo a muovermi nelle 4 direzioni cardinali
```

```

    for (int i = 0; i < 4; i++) {
        int next_r = r + row_dir[i];
        int next_c = c + col_dir[i];

        // Controllo validità: non devo uscire dai confini della griglia
        if (next_r >= 0 && next_r < R && next_c >= 0 && next_c < C) {
            // Mi sposto solo se la cella adiacente è piena (1) e non è mai stata
            visitata
            if (grid[next_r][next_c] == 1 && !visited[next_r][next_c]) {
                dfs_visit(grid, R, C, next_r, next_c, visited);
            }
        }
    }
}

// Funzione principale per la DFS completa (gestisce grafi/griglie disconnessi)
void dfs_complete_grid(int** grid, int R, int C) {
    // Alloco dinamicamente la matrice dei visitati e la inizializzo a false
    bool** visited = (bool**)malloc(R * sizeof(bool*));
    for (int i = 0; i < R; i++) {
        visited[i] = (bool*)calloc(C, sizeof(bool));
    }

    // Scorro l'intera griglia per assicurarmi di visitare ogni componente
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            // Faccio partire la DFS solo se trovo una cella piena e non ancora
            visitata
            if (grid[i][j] == 1 && !visited[i][j]) {
                dfs_visit(grid, R, C, i, j, visited);
            }
        }
    }

    // Libero la memoria allocata
    for (int i = 0; i < R; i++) {
        free(visited[i]);
    }
    free(visited);
}

```

Teoria (Camurati) :

● Cos'è un arco bridge? Cos'è un punto di articolazione e quali sono le condizioni che devono verificarsi affinché un vertice lo sia?

1. Archi bridge e Punti di articolazione

- **Arco bridge (Ponte):** È un arco di un grafo non orientato la cui rimozione aumenta il numero di componenti connesse del grafo. In parole povere, è l'unica via di collegamento tra due parti del grafo.
- **Punto di articolazione (Cut vertex):** È un vertice la cui rimozione (assieme a tutti gli archi ad esso incidenti) aumenta il numero di componenti connesse del grafo. È un "collo di bottiglia" vitale per la connessione.
- **Condizioni per essere punto di articolazione (tramite DFS):** Durante una visita DFS, si calcolano per ogni nodo u il tempo di scoperta $pre[u]$ e il tempo minimo raggiungibile $low[u]$ (il nodo scoperto prima raggiungibile dal sottoalbero di u usando al massimo un back-edge, ovvero un arco che torna indietro verso la radice).
 - **Condizione per la radice della DFS:** La radice dell'albero DFS è un punto di articolazione se e solo se ha almeno due figli indipendenti nell'albero DFS (se ne ha uno solo, rimuoverla non disconnette nulla).
 - **Condizione per i nodi interni:** Un nodo u (non radice) è un punto di articolazione se ha almeno un figlio v nell'albero DFS tale che dal sottoalbero radicato in v non è possibile risalire a un antenato di u . Matematicamente, si verifica quando $low[v] \geq pre[u]$.
- **Complessità:** L'algoritmo basato su DFS (Tarjan) trova tutti i bridge e punti di articolazione in Tempo $O(V + E)$ e Spazio $O(V)$.

● Powerset: definizione e spiegazione rapida dell'algoritmo divide et impera per calcolarlo.

2. Powerset: definizione e algoritmo Divide et Impera

- **Definizione:** Il Powerset (Insieme delle parti) di un insieme S è l'insieme di tutti i possibili sottoinsiemi di S , inclusi l'insieme vuoto e S stesso. Se S ha n elementi, il suo Powerset contiene 2^n sottoinsiemi.
- **Algoritmo Divide et Impera:** Per calcolare il Powerset di un insieme di n elementi (es. $\{A, B, C\}$), si divide il problema in due:
 - **Divide:** Si separa un elemento (es. C) e si calcola ricorsivamente il Powerset del sottoinsieme rimanente di $n-1$ elementi ($\{A, B\}$). Il risultato (chiamiamolo P_{meno}) sarà $\{\{\}, \{A\}, \{B\}, \{A, B\}\}$.
 - **Impera (Combina):** Il Powerset totale si ottiene unendo P_{meno} con una sua copia esatta a cui, però, viene aggiunto l'elemento isolato (C) in ogni suo set. La copia diventa $\{\{C\}, \{A, C\}, \{B, C\}, \{A, B, C\}\}$.
 - **Risultato finale:** Si uniscono le due metà per avere tutti gli 8 sottoinsiemi.

- **Complessità:** Tempo $O(n * 2^n)$ perché ci sono 2^n sottoinsiemi da generare, e la copia/inserimento di ogni sottoinsieme costa in media $O(n)$. Spazio $O(n * 2^n)$ per memorizzare tutti gli insiemi generati, oppure $O(n)$ (per il call stack) se i sottoinsiemi vengono solo stampati a video man mano.
-

Data : 23 (+1.5 lab) -> 29 (26/01/2026)

Programmazione (Cabodi) :

● Implementazione di un BST con duplicati (funzione di inserimento)

1. Implementazione di un BST con duplicati (funzione di inserimento)

- **Concetto:** Per gestire i duplicati in un Albero Binario di Ricerca (BST) ci sono due approcci. Il primo (meno efficiente) è inserire i duplicati nel sottoalbero destro (o sinistro). Il secondo (consigliato e più elegante) è aggiungere un campo count (contatore) nel nodo: se si inserisce un valore già presente, si incrementa semplicemente il contatore.
- **Logica dell'algoritmo:** Navighiamo l'albero ricorsivamente sfruttando la proprietà del BST (minori a sinistra, maggiori a destra). Se troviamo un nodo con la stessa chiave, incrementiamo il campo count. Se arriviamo a una foglia nulla, allochiamo un nuovo nodo inizializzando count a 1.
- **Complessità:** Tempo $O(h)$, dove h è l'altezza dell'albero ($O(\log n)$ nel caso medio, $O(n)$ nel caso pessimo di albero sbilanciato). Spazio $O(h)$ per lo stack delle chiamate ricorsive (o $O(1)$ se si usa la versione iterativa).

```
// Definizione del nodo del BST con gestione duplicati
typedef struct Node {
    int key;
    int count; // Contatore per gestire le occorrenze dei duplicati
    struct Node* left;
    struct Node* right;
} Node;

// Funzione ricorsiva di inserimento
Node* insert_bst_duplicate(Node* root, int key) {
    // Caso base: abbiamo raggiunto la posizione vuota dove inserire il nodo
    if (root == NULL) {
        Node* new_node = (Node*)malloc(sizeof(Node));
```

```

    new_node->key = key;
    new_node->count = 1; // Prima occorrenza di questa chiave
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

// La chiave esiste già: non creo nuovi nodi, aggiorno solo il contatore
if (key == root->key) {
    root->count++;
}
// La chiave è minore: mi sposto nel sottoalbero sinistro
else if (key < root->key) {
    root->left = insert_bst_duplicate(root->left, key);
}
// La chiave è maggiore: mi sposto nel sottoalbero destro
else {
    root->right = insert_bst_duplicate(root->right, key);
}

// Ritorno il puntatore alla radice (aggiornata) del sottoalbero corrente
return root;
}

```

● (a voce) Cosa restituirebbe la search in questa situazione (voleva sapere la gestione dei duplicati)

2. Cosa restituirebbe la search in questa situazione (a voce)

- **Se usiamo l'approccio con il contatore (come nel codice sopra):** La funzione di ricerca restituisce semplicemente il puntatore al nodo cercato. Dal nodo, l'utente può leggere il campo count per sapere esattamente quanti duplicati di quel valore sono presenti nell'albero, con una singola ricerca in tempo $O(h)$.
- **Se usassimo l'approccio con i nodi replicati (es. duplicati sempre a destra):** La search standard restituirebbe il *primo* nodo trovato con quella chiave. Per trovare *tutti* i duplicati, la funzione di ricerca dovrebbe continuare a scendere nel sottoalbero destro finché la chiave non cambia, restituendo magari una lista di puntatori o stampandoli. È un approccio peggiore perché spreca memoria (nodi extra) e rende l'albero più profondo e sbilanciato.

Teoria (Camurati) :

● Equazione alle ricorrenze divide and conquer

1. Equazione alle ricorrenze Divide and Conquer

- **Definizione:** Il paradigma Divide et Impera divide un problema in sotto-problemi più piccoli, li risolve ricorsivamente e ne combina le soluzioni. Il suo tempo di esecuzione è descritto dall'equazione generale: $T(n) = a * T(n/b) + f(n)$.
- **Significato dei termini:** * $T(n)$: Tempo totale per risolvere un problema di dimensione n .
 - **a:** Numero di sotto-problemi in cui il problema originale viene diviso ($a \geq 1$).
 - **n/b:** Dimensione di ciascun sotto-problema ($b > 1$).
 - **f(n):** Costo (tempo) necessario per dividere il problema iniziale e per combinare le soluzioni dei sotto-problemi.
- **Risoluzione:** Si risolve tipicamente applicando il **Master Theorem**, che confronta il costo delle foglie dell'albero di ricorsione ($O(n^{\log_b(a)})$) con il costo della combinazione $f(n)$, determinando la complessità asintotica finale.

● Perché non si può usare la programmazione dinamica nei cammini massimi

2. Perché non si può usare la programmazione dinamica nei cammini massimi

- **Sottostruttura ottima:** La programmazione dinamica funziona solo se il problema gode della proprietà di "sottostruttura ottima", ovvero se la soluzione ottima globale può essere costruita a partire dalle soluzioni ottime dei suoi sotto-problemi.
- **Il problema dei cammini massimi:** Nei cammini minimi la sottostruttura ottima vale: un sottocammino di un cammino minimo è a sua volta un cammino minimo. Nel caso dei cammini massimi (semplici, senza cicli), questo **non è vero**. Se il cammino massimo da A a C passa per B, il sottocammino da A a B potrebbe non essere il cammino massimo tra A e B, perché la scelta di un percorso più lungo tra A e B potrebbe "consumare" vertici necessari per raggiungere poi C, violando la regola di non avere cicli.
- **Complessità:** Trovare il cammino semplice massimo in un grafo generale è un problema **NP-arduo**, impossibile da risolvere in tempo polinomiale con la programmazione dinamica.

● Programmazione dinamica in Bellman Ford (spiegare perché si può usare e spiegare in breve l'algoritmo)

3. Programmazione dinamica in Bellman-Ford

- **Perché si può usare:** Bellman-Ford calcola i cammini minimi da una singola sorgente in presenza di pesi negativi. Si può usare la programmazione dinamica perché sfrutta la sottostruttura ottima dei cammini minimi, costruendo la soluzione dal basso verso

l'alto (bottom-up): calcola prima i cammini minimi composti da 1 arco, poi da 2 archi, fino ad arrivare a $V-1$ archi.

- **Spiegazione rapida dell'algoritmo:**

1. Inizializza le distanze di tutti i vertici a infinito, tranne la sorgente a 0.
2. Esegue il **rilassamento** (aggiornamento della distanza minima se passando per un nodo intermedio si trova un percorso più breve) su *tutti* gli archi del grafo.
3. Ripete il passaggio due per **$V-1$ volte** (dove V è il numero dei vertici), perché in un grafo senza cicli negativi il cammino minimo più lungo possibile ha esattamente $V-1$ archi.
4. Fa un ultimo giro di controllo: se è possibile rilassare ancora un arco, significa che esiste un ciclo di peso negativo (e il problema non ha soluzione finita).

- **Complessità:** Tempo $O(V * E)$ poiché rilassa E archi per $V-1$ volte. Spazio $O(V)$ per l'array delle distanze ottime parziali.

● Perché i cammini minimi sono semplici

4. Perché i cammini minimi sono semplici

- **Cammino semplice:** Un cammino si dice semplice se non passa mai due volte per lo stesso vertice (ovvero, non contiene cicli).
 - **Assenza di cicli positivi:** Se un cammino minimo contenesse un ciclo di peso positivo, basterebbe rimuovere quel ciclo dal percorso per ottenere un cammino ancora più corto, generando una contraddizione.
 - **Assenza di cicli di peso zero:** Se contenesse un ciclo di peso zero, si potrebbe rimuovere il ciclo ottenendo un cammino dello stesso peso globale ma con meno archi. Pertanto, possiamo sempre scegliere di prendere la versione "semplice" di quel cammino.
 - **Assenza di cicli negativi:** Se il cammino includesse un ciclo di peso negativo, la distanza minima diverrebbe $-\infty$ (percorrendo il ciclo infinite volte), rendendo il cammino minimo non ben definito.
 - **Conclusione:** Dato che i cicli positivi e nulli possono essere scartati, e quelli negativi invalidano il problema, un cammino minimo ben definito non contiene mai cicli ed è quindi, per definizione, **semplice**. Ha al massimo $V-1$ archi.
-

Data : 28→30 (0 lab) (26/01/2026)

Programmazione (Cabodi) :

● Scrivere una funzione che dato un grafo rappresentante un insieme di amici ed un vertice v , determinare il numero di amici di amici del vertice v , senza contare gli amici a distanza 1, solo quelli a distanza 2 (ha voluto usassi un doppio ciclo iterativo)

1. Trovare il numero di amici a distanza 2 (Amici di amici)

- **Concetto:** Dobbiamo contare i vertici raggiungibili con un percorso di lunghezza esattamente 2 partendo da v .
- **Logica (Doppio ciclo):** Il primo ciclo scorre gli amici diretti di v . Il secondo ciclo (annidato) scorre gli amici di questi amici.
- **Condizioni di validità:** Un nodo trovato al secondo ciclo è valido se: non è il nodo di partenza v , non è un amico diretto (distanza 1), e non lo abbiamo già contato (due amici diretti potrebbero avere lo stesso amico in comune).
- **Complessità:** Tempo $O(\text{grado}(v) * \text{grado_massimo})$, nel caso pessimo limitato superiormente da $O(V + E)$. Spazio $O(V)$ per mantenere in memoria chi è amico diretto e chi è già stato contato.

```
// Assumo le solite strutture Node e Graph
int count_friends_distance_2(Graph* g, int v) {
    // Array O(1) per sapere subito se un nodo è amico diretto (distanza 1)
    bool* is_direct_friend = calloc(g->numVertices, sizeof(bool));
    // Array per evitare di contare due volte lo stesso amico a distanza 2
    bool* is_counted = calloc(g->numVertices, sizeof(bool));
    int count = 0;

    // Pre-processamento: segno tutti gli amici diretti di v
    for (Node* adj = g->adjLists[v]; adj != NULL; adj = adj->next) {
        is_direct_friend[adj->vertex] = true;
    }

    // Primo ciclo iterativo: scorro gli amici diretti (distanza 1)
    for (Node* adj1 = g->adjLists[v]; adj1 != NULL; adj1 = adj1->next) {
        int direct_friend = adj1->vertex;

        // Secondo ciclo iterativo: scorro gli amici dell'amico (potenziale
        // distanza 2)
        for (Node* adj2 = g->adjLists[direct_friend]; adj2 != NULL; adj2 = adj2->next) {
            int friend_of_friend = adj2->vertex;

            // Filtro: deve essere diverso da v, non deve essere un amico diretto,
            // e non già contato
```

```

        if (friend_of_friend != v && !is_direct_friend[friend_of_friend] &&
!is_counted[friend_of_friend]) {
            count++;
            is_counted[friend_of_friend] = true; // Segno come contato per non
sdoppiarlo
        }
    }
}

// Pulizia della memoria
free(is_direct_friend);
free(is_counted);

return count;
}

```

Teoria (Camurati) :

● Discorso generale su heap, con complessità e relazione padre figlio tramite indice del vettore

1. Discorso generale su Heap, complessità e indici

- **Definizione:** Un Heap è un albero binario quasi completo (tutti i livelli sono pieni tranne eventualmente l'ultimo, riempito da sinistra verso destra) che rispetta la proprietà dell'ordinamento parziale. In un Max-Heap, il valore di ogni nodo padre è maggiore o uguale a quello dei suoi figli; nel Min-Heap è minore o uguale.
- **Rappresentazione:** Grazie alla sua struttura completa, non usa puntatori ma viene memorizzato in un semplice vettore (array).
- **Relazione Padre-Figlio (indici):** Se la radice è all'indice 0, per un nodo all'indice i :
 - Il figlio sinistro si trova all'indice $2i + 1$.
 - Il figlio destro si trova all'indice $2i + 2$.
 - Il padre si trova all'indice $(i - 1) / 2$ (troncato all'intero inferiore).
- **Complessità Temporale:**
 - **Trovare il massimo/minimo:** $O(1)$ perché è sempre la radice (indice 0).
 - **Inserimento:** $O(\log n)$. Si inserisce in fondo al vettore e si fa *Heapify-Up* (si fa risalire il nodo scambiandolo col padre finché non si ripristina la proprietà).
 - **Estrazione (Cancellazione):** $O(\log n)$. Si toglie la radice, si sposta l'ultimo elemento del vettore al posto della radice e si fa *Heapify-Down* (lo si fa scendere scambiandolo col figlio maggiore/minore).

- **Costruzione (Build-Heap):** $O(n)$ partendo da un vettore non ordinato, partendo dalle foglie verso la radice.
- **Complessità Spaziale:** $O(n)$ per memorizzare l'array.

● **Matrice e liste di adiacenza, cosa cambia se si usa vettore di liste o lista di liste (con il vettore dobbiamo conoscere in anticipo il numero dei vertici, con lista di liste no, basta aggiungere nodo)**

2. Rappresentazione grafi: Matrice, Vettore di liste e Lista di liste

- **Matrice di Adiacenza:** Una matrice $V \times V$. Occupa spazio $O(V^2)$, pesantissima per grafi sparsi (con pochi archi). Il check dell'esistenza di un arco costa $O(1)$.
- **Vettore di Liste (Lista di adiacenza classica):** Si usa un array statico o allocato dinamicamente di dimensione V , dove ogni cella contiene la testa di una lista concatenata di archi.
 - **Vantaggio:** Accesso in tempo $O(1)$ alla lista dei vicini di un nodo.
 - **Svantaggio:** Il vettore "spina dorsale" obbliga a conoscere in anticipo il numero esatto dei vertici V (o richiede costose operazioni di realloc se si superano i limiti). Spazio $O(V + E)$.
- **Lista di Liste:** La "spina dorsale" non è un array, ma una lista concatenata di vertici. Ogni vertice della spina ha poi il suo puntatore alla lista concatenata degli archi.
 - **Vantaggio:** Totalmente dinamica. Si possono aggiungere nuovi vertici in qualsiasi momento a costo $O(1)$ senza riallocare nulla.
 - **Svantaggio:** Per trovare i vicini del vertice k , bisogna prima scorrere la spina dorsale, peggiorando il tempo di accesso iniziale da $O(1)$ a $O(V)$.

Data : 21→25 (1,5 lab) (26/01/2026)

Programmazione (Cabodi) :

● **Implementa una funzione che dato un BST calcola a quale profondità sono presenti il maggior numero di nodi**

1. Calcolare la profondità di un BST con il maggior numero di nodi

- **Concetto:** Dobbiamo trovare il livello dell'albero con la larghezza massima. Aniché usare una coda per una visita in ampiezza (BFS) – che in C standard è lunga da implementare da zero – l'approccio più elegante all'orale è usare una visita in profondità (DFS) e un array di contatori.
- **Logica dell'algoritmo:** Prima calcoliamo l'altezza dell'albero per sapere quanto deve essere grande il nostro array. Creiamo un array counts di dimensione pari all'altezza. Lanciamo una DFS tenendo traccia della profondità corrente: ogni volta che visitiamo un nodo, incrementiamo counts[profondita]. Alla fine, cerchiamo l'indice dell'array con il valore massimo.
- **Complessità:** Tempo $O(n)$ perché visitiamo ogni nodo due volte (una per l'altezza, una per il conteggio). Spazio $O(h)$, dove h è l'altezza dell'albero, necessario per l'array dei contatori e lo stack ricorsivo.

```
#include <stdlib.h>

typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
} Node;

// Funzione ausiliaria per calcolare l'altezza dell'albero (necessaria per l'array)
int get_height(Node* root) {
    if (root == NULL) return 0;

    int left_h = get_height(root->left);
    int right_h = get_height(root->right);

    return (left_h > right_h ? left_h : right_h) + 1;
}

// Visita DFS che mappa ogni nodo al suo livello corrispondente
void count_levels(Node* root, int depth, int* counts) {
    if (root == NULL) return; // Caso base: foglia nulla

    counts[depth]++; // Trovato un nodo a questa profondità, incremento il
    // contatore

    // Scendo nei sottoalberi aumentando la profondità di 1
    count_levels(root->left, depth + 1, counts);
    count_levels(root->right, depth + 1, counts);
}

// Funzione principale richiesta all'esame
int max_nodes_depth(Node* root) {
    if (root == NULL) return -1; // Gestione albero vuoto
```



```

int h = get_height(root);

// calloc inizializza in automatico tutti i contatori a zero
int* counts = (int*)calloc(h, sizeof(int));

// Popolo l'array tramite la DFS partendo dalla radice (profondità 0)
count_levels(root, 0, counts);

int max_nodes = 0;
int best_depth = 0;

// Scorro l'array per trovare il livello più popolato
for (int i = 0; i < h; i++) {
    if (counts[i] > max_nodes) {
        max_nodes = counts[i];
        best_depth = i; // Salvo l'indice, che corrisponde alla profondità
    }
}

free(counts);
return best_depth;
}

```

Teoria (Camurati) :

●Indica il funzionamento di una tabella ad accesso diretto e la complessità delle operazioni che si possono svolgere

1. Tabella ad accesso diretto: funzionamento e complessità

- **Funzionamento:** È la struttura dati più basilare per implementare un dizionario quando l'universo delle chiavi U è piccolo. Se le chiavi sono interi compresi tra 0 e $m-1$, si crea un array (la tabella) di dimensione m . L'elemento con chiave k viene memorizzato esattamente nell'indice k dell'array. Se nessuna chiave vale k , l'indice k conterrà un valore nullo.
- **Complessità Temporale:** Ricerca (Search), Inserimento (Insert) e Cancellazione (Delete) costano tutte rigorosamente **$O(1)$** , poiché si accede direttamente all'indice dell'array senza fare confronti.
- **Complessità Spaziale:** **$O(m)$** , dove m è la dimensione dell'universo delle chiavi. Questo è il suo grande difetto: se l'universo è immenso (es. tutti i possibili codici fiscali) ma memorizziamo solo 10 elementi, allochiamo un array gigantesco sprecando quasi tutta la memoria.

●Spiega il funzionamento della BSTdelete()

2. Funzionamento della BSTdelete()

- **Concetto:** La cancellazione di un nodo Z da un Albero Binario di Ricerca (BST) deve mantenere valida la proprietà fondamentale dell'albero (minori a sinistra, maggiori a destra). Si divide in 3 casi distinti in base al numero di figli del nodo Z.
 - **Caso 1 (Z non ha figli):** Z è una foglia. Viene semplicemente deallocato, e il puntatore del padre di Z che puntava a lui viene impostato a NULL.
 - **Caso 2 (Z ha un solo figlio):** Z viene "bypassato". Si prende l'unico figlio di Z e lo si collega direttamente al padre di Z. Poi si dealloca Z.
 - **Caso 3 (Z ha due figli):** È il caso complesso. Non possiamo rimuovere Z direttamente. Dobbiamo trovare il **successore o predecessore** di Z (il nodo con il valore minimo nel sottoalbero destro di Z). Si copia il valore del successore dentro il nodo Z. Ora l'albero è valido, ma abbiamo il successore duplicato: lanciamo quindi ricorsivamente la BSTdelete() sul nodo successore originale. Siccome il successore è il minimo del suo sottoalbero, non ha un figlio sinistro, quindi la sua cancellazione ricadrà sempre nel Caso 1 o Caso 2, risultando facile da rimuovere.
 - **Complessità:** Il tempo è dominato dalla ricerca del nodo Z e (nel Caso 3) dalla ricerca del suo successore. Entrambe le operazioni richiedono di scendere l'albero, quindi la complessità temporale è **O(h)**, dove h è l'altezza dell'albero ($O(\log n)$ nel caso medio, $O(n)$ nel caso pessimo).
-

Data : 19 -> 20 (no lab) (26/01/2026)

Programmazione (Cabodi) :

- Implementare una funzione che dato un BST trovi la profondità minima tra le foglie
- Poi a voce mi ha chiesto cosa avrei dovuto fare se avessi anche voluto contare le foglie (mettere un contatore)

Programmazione (Cabodi)

- **Concetto:** Dobbiamo trovare il cammino più breve dalla radice a un nodo foglia (un nodo senza figli, ovvero con puntatori left e right a NULL).
- **Logica dell'algoritmo:** Si utilizza una visita ricorsiva in profondità (DFS). Se l'albero è vuoto, restituiamo 0. Se siamo su una foglia, restituiamo 1. Se il nodo ha un solo figlio, siamo costretti a scendere in quell'unico ramo (ignorando il puntatore a NULL che

restituirebbe scorrettamente 0). Se ha due figli, calcoliamo ricorsivamente la profondità minima di entrambi i rami e restituiamo la minore, sommandole 1 (per contare il nodo corrente).

- **Complessità:** Tempo $O(n)$ nel caso pessimo (albero sbilanciato in cui dobbiamo esplorare tutti i nodi per trovare la foglia più vicina). Spazio $O(h)$, con h altezza dell'albero, per lo stack delle chiamate ricorsive.

Cosa fare per contare anche le foglie (risposta a voce): Aggiungerei un parametro puntatore alla funzione, ad esempio `int* leaf_count`. All'interno del "Caso base 2" (quando riconosco di essere su una foglia), inserirei l'istruzione `(*leaf_count)++`; prima di fare il `return 1`; In questo modo, sfruttando la stessa identica visita in tempo $O(n)$, aggiornare il contatore ogni volta che tocco una foglia.

```
#include <stdio.h>

// Definizione del nodo del BST
typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
} Node;

// Funzione ricorsiva per la profondità minima
int min_leaf_depth(Node* root) {
    // Caso base 1: albero vuoto
    if (root == NULL) {
        return 0;
    }

    // Caso base 2: siamo arrivati a una foglia effettiva
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }

    // Se manca il figlio sinistro, l'unica via possibile è esplorare a destra
    if (root->left == NULL) {
        return 1 + min_leaf_depth(root->right);
    }

    // Se manca il figlio destro, l'unica via possibile è esplorare a sinistra
    if (root->right == NULL) {
        return 1 + min_leaf_depth(root->left);
    }

    // Il nodo ha entrambi i figli: calcolo le profondità di ambo i rami
    int left_depth = min_leaf_depth(root->left);
    int right_depth = min_leaf_depth(root->right);
```

```
// Restituisco il ramo più corto, più 1 per il livello attuale
return 1 + (left_depth < right_depth ? left_depth : right_depth);
}
```

Teoria (Camurati) :

- Algoritmo MCD caso peggiore e complessità
- Teorema e corollario degli archi sicuri.

Teoria (Camurati)

- **Algoritmo MCD (Euclide):** Si basa sulla formula matematica $MCD(a, b) = MCD(b, a \% b)$. Si applica iterativamente o ricorsivamente la divisione modulare finché il resto non diventa 0. Quando il resto è 0, l'ultimo divisore non nullo è il Massimo Comun Divisore.
- **Caso peggiore:** Il caso pessimo si verifica quando gli input a e b sono due numeri consecutivi della sequenza di Fibonacci. In questo scenario, il quoziente della divisione è sempre 1, e l'operazione di modulo ($a \% b$) degenera in una banale sottrazione ($a - b$). Questo produce la riduzione numerica più lenta possibile ad ogni iterazione, massimizzando il numero totale di passi necessari.
- **Complessità:** Per il Teorema di Lamé, il numero di passi non supera mai 5 volte il numero delle cifre decimali dell'input più piccolo. Pertanto, la complessità Temporale è logaritmica: $O(\log(\min(a, b)))$. La complessità Spaziale è $O(1)$ se l'algoritmo è implementato in modo iterativo, o $O(\log(\min(a, b)))$ se implementato ricorsivamente (per via dello stack).
- **Definizioni preliminari agli archi sicuri:**
 - **Taglio (S, V-S):** Una qualsiasi partizione dei vertici del grafo in due insiemi disgiunti.
 - **Rispettare un taglio:** Un insieme di archi A "rispetta" un taglio se nessun arco di A collega un vertice di S a un vertice di $V-S$.
 - **Arco leggero:** Tra tutti gli archi che attraversano il taglio, è quello con il peso (costo) minimo.
- **Teorema degli archi sicuri:** Sia A un sottoinsieme di archi incluso in un qualche Minimum Spanning Tree (MST). Sia $(S, V-S)$ un qualsiasi taglio che rispetta A , e sia (u, v) un arco leggero che attraversa questo taglio. Allora l'arco (u, v) è un "arco sicuro" per A , ovvero l'insieme A unito all'arco (u, v) farà ancora parte di un MST valido.
- **Corollario (alla base di Prim e Kruskal):** Dato un grafo, se l'insieme di archi A forma una foresta (un insieme di alberi disgiunti) che è un sottoinsieme di un MST, allora l'arco di peso minimo che collega una di queste componenti connesse a qualsiasi

altro vertice al di fuori di essa è sempre un arco sicuro. (Prim usa questo corollario accrescendo un singolo albero centrale, Kruskal unendo gli alberi sparsi della foresta).

Data : 24.75->27 (1.25 Lab) (26/01/2026)

Programmazione (Cantoro) :

● Alberi binari estesi

1. Alberi binari estesi

- **Concetto:** Un albero binario esteso (o 2-albero) è un albero in cui ogni puntatore a NULL (figlio vuoto) dell'albero originale viene sostituito con un nodo speciale detto "nodo esterno" o "foglia estesa". I nodi originali diventano tutti "nodi interni".
- **Proprietà:** Ogni nodo interno ha esattamente 0 o 2 figli. Se un albero ha n nodi interni, avrà sempre $n+1$ nodi esterni.
- **Codice:** L'implementazione pratica spesso non alloca fisicamente i nodi esterni, ma li tratta come casi base (NULL) nelle visite, restituendo valori fittizi. Ecco un esempio per contare i nodi esterni:
- **Complessità:** Tempo $O(n)$ per visitare tutto l'albero, Spazio $O(h)$ per lo stack ricorsivo.

● Programmazione dinamica e greedy (esempio problema degli zaini con implementazione)

2. Programmazione dinamica e greedy (Problema dello Zaino)

- **Zaino Frazionario (Greedy):** Si possono prendere frazioni degli oggetti. La strategia Greedy (golosa) ordina gli oggetti per rapporto valore/peso decrescente e riempie lo zaino. Funziona e trova l'ottimo globale perché non si spreca mai spazio.
- **Zaino 0/1 (Programmazione Dinamica):** L'oggetto si prende intero o si lascia. Greedy fallisce perché potrebbe lasciare spazi vuoti sub-ottimali. Serve la Programmazione Dinamica: si usa una matrice per memorizzare le soluzioni ottime di sottoproblemi più piccoli (zaini di capienza minore con meno oggetti disponibili).
- **Complessità (Zaino 0/1):** Tempo $O(n * W)$ dove n è il numero di oggetti e W la capacità. Spazio $O(n * W)$ per la matrice di memoizzazione (ottimizzabile a $O(W)$ usando un solo array).

● Ricerca in profondità di un grafo con implementazione usando le liste di adiacenza

3. Ricerca in profondità (DFS) di un grafo con liste di adiacenza

- **Concetto:** Si visita un nodo, lo si marca come visitato, e si lancia ricorsivamente la funzione su tutti i suoi vicini non ancora visitati.
- **Complessità:** Tempo $O(V + E)$ perché ogni vertice ed ogni arco vengono esplorati al massimo una volta. Spazio $O(V)$ per l'array dei visitati e lo stack di ricorsione.

Teoria (Ferrero) :

● Algoritmi di ordinamento ricorsivi e le loro proprietà (stabile, in loco)

1. Algoritmi di ordinamento ricorsivi e proprietà (Merge Sort, Quick Sort)

- **In Loco (In-place):** Un algoritmo è in loco se ordina i dati usando solo una quantità costante $O(1)$ di memoria extra (o $O(\log n)$ per lo stack ricorsivo), sovrascrivendo l'array originale.
- **Stabile (Stable):** Un algoritmo è stabile se preserva l'ordine relativo degli elementi con chiavi uguali (es. se ho due '5', quello che era a sinistra rimane a sinistra dopo l'ordinamento).
- **Merge Sort:** Divide l'array a metà, ordina le metà ricorsivamente e le fonde.
 - **Proprietà:** È **Stabile** (nella fase di merge, tra uguali si prende prima quello di sinistra). **NON è in loco** perché la fase di merge richiede un array di appoggio di dimensione $O(n)$.
 - **Complessità:** Tempo $O(n \log n)$ sempre. Spazio $O(n)$.
- **Quick Sort:** Sceglie un pivot, partiziona l'array (minori a sx, maggiori a dx) e ricorre sulle due metà.
 - **Proprietà:** È **In loco** (spazio extra solo per lo stack $O(\log n)$). **NON è stabile** perché i frequenti scambi a lunga distanza durante il partizionamento distruggono l'ordine originale dei duplicati.
 - **Complessità:** Tempo $O(n \log n)$ caso medio/ottimo, $O(n^2)$ caso pessimo (pivot scelto male). Spazio $O(\log n)$.

● Linked list e le loro proprietà (estrazione di un elemento e come si realizza, come si possono ordinare e perché non funzionano gli algoritmi ricorsivi)

2. Linked list: proprietà, estrazione e ordinamento

- **Estrazione di un elemento:** A differenza degli array, non si può accedere all'elemento in $O(1)$. Bisogna scorrere la lista partendo dalla testa (Tempo $O(n)$). Durante la

scansione, si mantiene un puntatore al nodo corrente (curr) e uno al precedente (prev). Trovato il nodo, si scollega impostando $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$ e si libera la memoria di curr. L'operazione fisica di estrazione costa $O(1)$.

- **Perché gli algoritmi ricorsivi classici non funzionano bene:** Gli algoritmi come Quick Sort e Merge Sort (nella loro versione standard su array) si basano pesantemente sull'**accesso casuale** (es. `array[i]`). Su una lista concatenata, trovare l'elemento centrale (necessario per il Merge Sort) o accedere a indici arbitrari (necessario per il partizionamento rapido del Quick Sort) richiede una scansione lineare $O(n)$ ad ogni passo, distruggendo le performance temporali.
 - **Come si possono ordinare:** Il metodo più semplice ed efficiente per liste corte è l'**Insertion Sort** (costo $O(n^2)$), perché si scorre la lista linearmente manipolando i puntatori. Per liste lunghe si può adattare il **Merge Sort**, ma usando una logica iterativa bottom-up o trovando il centro con la tecnica dei "due puntatori" (uno lento e uno veloce), pagando però un costo maggiore nella gestione dei link rispetto agli array.
-

Data : 15->18 (no lab) (26/01/2026)

Programmazione (Cabodi) :

● Implementare una funzione che esegua la potatura di un albero dato un BST e un livello p (vale a dire eliminare tutti i valori di livello $> p$)

1. Potatura (pruning) di un BST a un livello p

- **Concetto:** Dobbiamo percorrere l'albero tenendo traccia della profondità corrente. Quando arriviamo al livello 'p', eliminiamo fisicamente tutti i nodi dei sottoalberi sinistro e destro, per poi impostare i puntatori dei figli a NULL.
- **Logica dell'algoritmo:** Usiamo una visita in profondità (DFS). Se il livello corrente è minore di p, procediamo ricorsivamente verso il basso. Se il livello corrente è uguale a p, invochiamo una funzione ausiliaria per deallocare la memoria di tutto ciò che sta sotto (tramite una visita post-ordine) e blocchiamo la ricorsione su quel ramo.
- **Complessità:** Tempo $O(n)$ nel caso pessimo, perché nel complesso visitiamo e/o deallociamo ogni nodo dell'albero al massimo una volta. Spazio $O(h)$, dove h è l'altezza dell'albero, per lo stack delle chiamate ricorsive.

```
#include <stdlib.h>
```

```

typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
} Node;

// Funzione ausiliaria per deallocare un intero sottoalbero
// Usa una visita post-ordine: prima i figli, poi il padre
void free_tree(Node* root) {
    if (root == NULL) return;

    free_tree(root->left); // Libero ricorsivamente il ramo sinistro
    free_tree(root->right); // Libero ricorsivamente il ramo destro
    free(root); // Infine libero il nodo corrente
}

// Funzione ricorsiva principale per la potatura
void prune_bst_util(Node* root, int current_level, int p) {
    if (root == NULL) return; // Caso base: albero vuoto o foglia superata

    // Se abbiamo raggiunto il livello limite p
    if (current_level == p) {
        // Deallochiamo interamente i sottoalberi pendenti
        free_tree(root->left);
        free_tree(root->right);

        // Impostiamo i figli a NULL per evitare dangling pointers (puntatori
appesi)
        root->left = NULL;
        root->right = NULL;

        return; // Interrompiamo la discesa, non c'è più nulla sotto da esplorare
    }

    // Se siamo sopra il livello p, continuiamo a scendere incrementando il livello
corrente
    prune_bst_util(root->left, current_level + 1, p);
    prune_bst_util(root->right, current_level + 1, p);
}

// Funzione wrapper per l'utente, assumendo che la radice sia a livello 0
void prune_bst(Node* root, int p) {
    prune_bst_util(root, 0, p);
}

```

Teoria (Camurati):

- Esplicare l'algoritmo di Karatsuba e calcolare l'equazione alle ricorrenze dell'algoritmo

1. Algoritmo di Karatsuba e sua equazione alle ricorrenze

- **Algoritmo Standard:** La moltiplicazione classica in colonna tra due numeri di n cifre richiede un tempo $O(n^2)$. Il Divide et Impera banale divide i numeri a metà, generando 4 moltiplicazioni ricorsive di dimensione $n/2$. La sua equazione è $T(n) = 4T(n/2) + O(n)$, che per il Master Theorem rimane $O(n^2)$.
- **Algoritmo di Karatsuba:** È un metodo Divide et Impera ottimizzato. Divide i due numeri a metà, ma usa un trucco algebrico: calcola il prodotto dei termini incrociati sottraendo i prodotti dei termini isolati da una singola moltiplicazione delle somme. Questo permette di scambiare una costosa moltiplicazione ricorsiva con delle banali addizioni (che costano solo $O(n)$).
- **Equazione alle ricorrenze:** Le moltiplicazioni ricorsive scendono da 4 a 3. L'equazione diventa rigorosamente $T(n) = 3T(n/2) + O(n)$.
- **Complessità (Master Theorem):** Si applica il Caso 1 del Master Theorem confrontando il costo delle ricorsioni $n^{\log_2(3)}$ con il costo di ricombinazione $O(n)$. Poiché $\log_2(3)$ è circa 1.58, le ricorsioni dominano. La complessità Temporale finale è $O(n^{\log_2(3)})$, circa $O(n^{1.58})$, asintoticamente molto più veloce di $O(n^2)$. La complessità Spaziale è $O(\log n)$ per lo stack ricorsivo.

● Funzioni tail recursive

2. Funzioni tail recursive

- **Definizione:** Una funzione ricorsiva si dice "tail recursive" (con ricorsione in coda) se la chiamata a se stessa è l'esatta, ultima istruzione eseguita prima di ritornare. Non deve esserci alcun calcolo pendente (es. somme o moltiplicazioni) da effettuare dopo che la chiamata ricorsiva è terminata.
 - **Meccanismo della Tail Call Optimization (TCO):** Se una funzione è tail recursive, il compilatore applica la TCO. Capendo che non c'è più nulla da fare nel contesto (frame) della funzione chiamante, il compilatore decide di non allocare un nuovo "Stack Frame" in memoria per la chiamata successiva, ma ricicla e sovrascrive quello corrente aggiornando solo i parametri.
 - **Complessità e Vantaggi:** A livello di linguaggio macchina, la ricorsione viene tradotta in un semplice e velocissimo ciclo iterativo (while/goto). La complessità Spaziale crolla drasticamente da $O(n)$ a $O(1)$, eliminando del tutto l'occupazione della memoria Stack e prevenendo matematicamente gli errori di Stack Overflow. La complessità Temporale rimane invariata.
-

Data : 18 -> 19 (0.0 Lab) (26/01/2026)

Programmazione (Cabodi) :

● Dato un vertice V ed un Grafo orientato, eliminare tutti gli archi entranti ed uscenti da V , usando il grafo con listaDiAdiacenze

- **Concetto:** In un grafo orientato con liste di adiacenza, l'eliminazione degli archi per un vertice V si divide in due fasi. Per gli archi **uscenti**, basta svuotare la lista di adiacenza associata all'indice V . Per gli archi **entranti**, non avendo puntatori all'indietro, è necessario scorrere le liste di adiacenza di *tutti* gli altri vertici del grafo e rimuovere i nodi che puntano a V .
- **Complessità:** Tempo $O(V + E)$, poiché nel caso pessimo si esplorano tutte le liste di adiacenza del grafo per scovare e rimuovere gli archi entranti. Spazio $O(1)$ perché l'operazione di rimozione e ricollegamento dei nodi avviene in loco.

```
#include <stdlib.h>

// Funzione principale per rimuovere archi entranti e uscenti
void remove_all_edges_of_vertex(Graph* g, int v) {
    // FASE 1: Rimuovere tutti gli archi USCENTI da v
    Node* curr_out = g->adjLists[v];
    while (curr_out != NULL) {
        Node* temp = curr_out;
        curr_out = curr_out->next;
        free(temp); // Dealloca fisicamente il nodo
    }
    // Setto la lista di v a NULL per indicare che non ha più archi uscenti
    g->adjLists[v] = NULL;

    // FASE 2: Rimuovere tutti gli archi ENTRANTI verso v
    // Scorro le liste di adiacenza di tutti gli altri vertici del grafo
    for (int i = 0; i < g->numVertices; i++) {
        // Ignoro il vertice v stesso (già svuotato al passo 1)
        if (i == v) continue;

        Node* curr_in = g->adjLists[i];
        Node* prev_in = NULL;

        // Esploro la lista di adiacenza del vertice i
        while (curr_in != NULL) {
            // Se trovo un arco che punta al vertice v
            if (curr_in->vertex == v) {
                // Caso in cui il nodo da eliminare è in testa alla lista
                if (prev_in == NULL) {
```

```

        g->adjlists[i] = curr_in->next;
    }
    // Caso in cui il nodo è in mezzo o in fondo: bypasso il nodo
    else {
        prev_in->next = curr_in->next;
    }

    // Salvo il puntatore, avanzo e dealloco per evitare memory leak
    Node* temp = curr_in;
    curr_in = curr_in->next;
    free(temp);
}
// Se non punta a v, avanzo semplicemente i puntatori
else {
    prev_in = curr_in;
    curr_in = curr_in->next;
}
}
}
}
}

```

Teoria (Camurati) :

- Definizione Arco completo ○ Definizione componenti connesse
- Fattore di carico tabella hash, valori che può assumere ○ Open addressing vs linear chaining ○ Pk double hashing (clustering)

Teoria (Camurati)

- **Grafo Completo (nota: la dicitura "arco completo" non è standard, si intende tipicamente "grafo completo"):** Un grafo non orientato si definisce completo se esiste un arco per ogni possibile coppia di vertici distinti. Il numero totale di archi è esattamente $V*(V-1)/2$. La complessità spaziale per rappresentarlo tramite matrice di adiacenza è $O(V^2)$, ed è una matrice interamente piena (senza zeri, a parte la diagonale principale).
- **Componenti Connesse:** In un grafo non orientato, una componente connessa è un sottografo "massimale" in cui esiste sempre un cammino tra ogni coppia di vertici. "Massimale" significa che non è possibile aggiungere alla componente alcun altro vertice del grafo originale senza rompere la proprietà di connessione. Nei grafi orientati si distingue tra componenti fortemente connesse (percorsi in entrambe le direzioni) e debolmente connesse (ignorando la direzione degli archi).
- **Fattore di carico tabella hash (alpha):** Definito come l'equazione $\alpha = n/m$, dove "n" è il numero di chiavi attualmente memorizzate e "m" è il numero totale di slot disponibili nella tabella. Indica il livello di riempimento medio.

- **Valori del fattore di carico:** Nella tecnica a concatenamento (chaining), alpha può assumere valori > 1 (più chiavi che slot disponibili). Nella tecnica a indirizzamento aperto (open addressing), alpha deve essere strettamente compreso tra 0 e 1 ($0 \leq \alpha \leq 1$), poiché non è fisicamente possibile inserire più di un elemento per slot.
 - **Open addressing vs Linear chaining:**
 - * **Linear Chaining (Concatenamento):** Le collisioni si risolvono creando liste concatenate all'esterno dell'array principale. Gli slot della tabella contengono solo i puntatori alle teste delle liste. È flessibile sui fattori di carico alti ma richiede memoria extra per i puntatori.
 - **Open Addressing (Indirizzamento aperto):** Tutti gli elementi sono memorizzati fisicamente dentro l'array della tabella. In caso di collisione, si ricalcola l'indice (probing) per cercare il primo slot vuoto. Evita l'overhead dei puntatori ma le prestazioni crollano quando la tabella è quasi piena.
 - **Perché Double Hashing (Clustering):** Le tecniche di ispezione standard nell'indirizzamento aperto creano "clustering" (ammassamenti). L'ispezione lineare crea il Primary Clustering (blocchi di celle adiacenti occupate si fondono). L'ispezione quadratica crea il Secondary Clustering (chiavi con lo stesso hash iniziale finiscono per seguire identici percorsi di ispezione). Il **Double Hashing** risolve entrambi usando una seconda funzione di hash indipendente per determinare il "passo" di salto. In questo modo, anche se due chiavi collidono al primo colpo, la seconda funzione genererà sequenze di salto completamente diverse.
-

Data : 23.75->25 (1.5 lab)(10/02/2026c)

Programmazione (Cabodi Cantoro) :

- Bellman Ford, rilassamento, Diskja
- heap ,heapify, heap built, heapsort.(con complessità)

1. Bellman-Ford, Rilassamento, Dijkstra

- **Rilassamento (Relaxation):** È l'operazione fondamentale degli algoritmi di cammino minimo. Dato un arco da u a v con peso $w(u,v)$, il rilassamento verifica se il cammino più breve noto per arrivare a v può essere migliorato passando per u . La condizione è: se $d[u] + w(u,v) < d[v]$, allora aggiorno $d[v] = d[u] + w(u,v)$.
- **Bellman-Ford:** Calcola i cammini minimi da una sorgente singola ed è in grado di gestire grafi con archi di peso negativo. L'algoritmo esegue il rilassamento di *tutti* gli

archi del grafo per $V-1$ volte. Esegue poi un ultimo ciclo di verifica: se un arco può essere ancora rilassato, significa che esiste un ciclo di peso negativo.

- **Complessità:** Tempo $O(V * E)$. Spazio $O(V)$ per memorizzare le distanze.
- **Dijkstra:** Calcola i cammini minimi da una sorgente singola, ma richiede che i pesi degli archi siano **tutti non negativi**. È un algoritmo *Greedy*: utilizza una Coda di Priorità (Min-Heap) per estrarre sempre il vertice non visitato con la distanza minima corrente, rilassando poi tutti i suoi archi uscenti.
 - **Complessità:** Tempo $O((V + E) \log V)$ implementato con un Min-Heap binario. Spazio $O(V)$.

2. Heap, Heapify, Build Heap, Heapsort

- **Heap:** È un albero binario quasi completo, rappresentato implicitamente tramite un array. In un Max-Heap, il valore di ogni nodo padre è sempre maggiore o uguale al valore dei suoi figli.
- **Max-Heapify:** Funzione che ripristina la proprietà del Max-Heap. Se un nodo è più piccolo di uno dei suoi figli, lo scambia con il figlio maggiore e viene richiamata ricorsivamente sul nuovo sottoalbero. Costo Temporale: $O(\log n)$. Costo Spaziale: $O(\log n)$ ricorsiva, $O(1)$ iterativa.
- **Build-Heap:** Costruisce un Max-Heap partendo da un array disordinato. Chiama Max-Heapify su tutti i nodi interni (non foglie), partendo dal basso verso l'alto (dall'indice $n/2 - 1$ fino a 0). Sebbene sembri costare $O(n \log n)$, un'analisi matematica rigorosa dimostra che il costo Temporale esatto è **$O(n)$** .
- **Heapsort:** Algoritmo di ordinamento in loco. Costruisce il Max-Heap, poi estrae ripetutamente la radice (il massimo), la scambia con l'ultimo elemento dell'array, riduce la dimensione dell'heap di 1 e chiama Max-Heapify sulla nuova radice.
 - **Complessità Heapsort:** Tempo $O(n \log n)$ sempre (caso ottimo, medio e pessimo). Spazio $O(1)$ perché non usa array ausiliari.

```
#include <stdio.h>

// Funzione per scambiare due valori
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Ripristina la proprietà del Max-Heap per il sottoalbero con radice 'i'
void max_heapify(int arr[], int n, int i) {
```

```

int largest = i;           // Inizializzo il massimo come radice
int left = 2 * i + 1;      // Indice del figlio sinistro
int right = 2 * i + 2;     // Indice del figlio destro

// Se il figlio sinistro esiste ed è maggiore della radice
if (left < n && arr[left] > arr[largest])
    largest = left;

// Se il figlio destro esiste ed è maggiore del massimo trovato finora
if (right < n && arr[right] > arr[largest])
    largest = right;

// Se il massimo non è la radice originale, violazione trovata!
if (largest != i) {
    swap(&arr[i], &arr[largest]); // Scambio la radice con il figlio maggiore

    // Ricorsione sul sottoalbero influenzato per propagare la correzione verso
    // il basso
    max_heapify(arr, n, largest);
}
}

// Funzione principale di ordinamento Heapsort
void heapsort(int arr[], int n) {
    // Fase 1: Build-Heap. Parto dall'ultimo nodo non foglia (n/2 - 1) e risalgo.
    // Questo trasforma l'array in un Max-Heap valido.
    for (int i = n / 2 - 1; i >= 0; i--) {
        max_heapify(arr, n, i);
    }

    // Fase 2: Estrazione degli elementi uno per uno
    for (int i = n - 1; i > 0; i--) {
        // Sposto l'elemento radice (il massimo attuale) alla fine dell'array
        swap(&arr[0], &arr[i]);

        // Richiamo heapify sulla nuova radice per ripristinare il Max-Heap,
        // riducendo virtualmente la dimensione dell'heap 'n' alla variabile 'i'
        max_heapify(arr, i, 0);
    }
}

```

Teoria (Camurati Ferrero) :

- Bst ,rotazioni,inserimento in foglia e in radice

1. BST, Rotazioni, Inserimento in foglia e in radice

- **BST (Binary Search Tree):** Albero in cui per ogni nodo x , i nodi nel sottoalbero sinistro hanno chiavi minori di x e quelli nel destro chiavi maggiori. L'altezza h determina i tempi delle operazioni ($O(\log n)$ se bilanciato, $O(n)$ se degenerare).
- **Rotazioni:** Operazioni locali di riorganizzazione dei puntatori che cambiano la struttura dell'albero senza violare la proprietà di ordinamento del BST. Esistono rotazioni a sinistra e a destra. Servono per bilanciare l'albero (es. AVL, Red-Black Trees) o per spostare nodi specifici verso l'alto. Tempo: $O(1)$. Spazio $O(1)$.
- **Inserimento in foglia:** È l'inserimento standard. Si scende a sinistra o a destra confrontando la chiave da inserire con i nodi attraversati, fino a trovare un puntatore a NULL. Si alloca il nuovo nodo e lo si aggancia lì come foglia. Tempo $O(h)$.
- **Inserimento in radice:** Si usa quando gli elementi inseriti più di recente hanno maggiore probabilità di essere cercati a breve. Si inserisce il nodo normalmente come foglia in tempo $O(h)$, poi si esegue una sequenza di rotazioni dal basso verso l'alto lungo il cammino di ricerca per spingere il nuovo nodo fino a farlo diventare la nuova radice dell'albero. Tempo totale $O(h)$.

● funzioni tail recursive

2. Funzioni tail recursive

- **Definizione:** Una funzione si dice tail recursive se la chiamata ricorsiva è rigorosamente l'ultima operazione effettuata prima del return. Non ci sono addizioni, moltiplicazioni o altre operazioni logiche in sospeso al ritorno della chiamata.
- **Tail Call Optimization (TCO):** Se una funzione è tail recursive, il compilatore "ricicla" l'area di memoria (Stack Frame) della funzione corrente, sovrascrivendola con i parametri della nuova chiamata, invece di impilare nuovi frame all'infinito.
- **Complessità:** Grazie alla TCO, la funzione viene tradotta a basso livello in un semplice ciclo iterativo. La complessità Temporale rimane invariata, ma la complessità Spaziale crolla da $O(n)$ a **$O(1)$** , eliminando il rischio di Stack Overflow.

● passaggio da funzione ricorsiva a iterativa

3. Passaggio da funzione ricorsiva a iterativa

- **Teorema fondamentale:** Qualsiasi funzione ricorsiva può essere sempre trasformata in una funzione iterativa equivalente.
- **Caso Tail Recursive:** La conversione è immediata. Si sostituisce la chiamata ricorsiva con un costrutto while, aggiornando i parametri della funzione alla fine di ogni iterazione del ciclo. Memoria aggiuntiva: $O(1)$.

- **Caso Ricorsione non-tail:** Se ci sono operazioni pendenti (come la discesa nell'albero in pre-ordine dove devo ricordarmi i nodi padri per esplorare i rami destri), per renderla iterativa bisogna simulare esplicitamente lo Stack di sistema. Si dichiara una struttura dati Stack e, invece di fare la chiamata ricorsiva, si esegue un push delle variabili locali e dello stato nello stack; al termine si fa il pop. La complessità Spaziale rimane $O(n)$ o $O(h)$, ma si sposta l'occupazione di memoria dallo Stack di sistema alla memoria Heap dell'applicazione.

Data : 19 -> 21 (0.5 Lab) (12/09/2025)

Programmazione (Cabodi) :

● Inserimento di item in tabella di hash con linear chaining + a voce come avrei potuto fare con linear probing

Inserimento in Hash Table con Linear Chaining (Concatenamento):

- **Logica:** La tabella di hash è un array di puntatori a nodi (liste concatenate). Calcoliamo l'indice tramite la funzione di hash. Creiamo un nuovo nodo e lo inseriamo in testa alla lista concatenata corrispondente a quell'indice, un'operazione istantanea che non richiede di scorrere la lista.
- **Complessità Temporale:** L'inserimento in testa costa rigorosamente $O(1)$. (Nota: se le specifiche richiedessero di vietare i duplicati, bisognerebbe prima scorrere la lista, costando $O(n)$ nel caso pessimo e $O(1)$ nel caso medio se il fattore di carico è tenuto basso).
- **Complessità Spaziale:** $O(m + n)$, dove m è la dimensione dell'array (gli slot) e n è il numero totale di elementi inseriti allocati dinamicamente.

```
// Struttura del nodo per la lista concatenata (chain)
typedef struct Node {
    int key;
    struct Node* next;
} Node;

// Struttura della Hash Table
typedef struct HashTable {
    int size;
    Node** table; // Array di puntatori a Node
}
```



```

} HashTable;

// Funzione di hash banalizzata (modulo della dimensione)
int hash_function(int key, int size) {
    return key % size;
}

// Inserimento con linear chaining (inserimento in testa)
void hash_insert(HashTable* ht, int key) {
    // Calcolo lo slot in cui inserire l'elemento
    int index = hash_function(key, ht->size);

    // Alloco memoria per il nuovo nodo e ne imposto il valore
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->key = key;

    // L'elemento successivo del nuovo nodo diventa l'attuale testa della lista
    new_node->next = ht->table[index];

    // Il nuovo nodo diventa ufficialmente la nuova testa della lista in quello
    slot
    ht->table[index] = new_node;
}

```

Teoria (Camurati) :

● Funzioni tail-recursive + stack frame

❓ Funzioni tail-recursive e Stack Frame:

- **Definizione Tail Recursive:** Una funzione ricorsiva è "tail-recursive" (ricorsione in coda) se la chiamata a se stessa è in assoluto l'ultima operazione eseguita. Nessun calcolo (come somme o moltiplicazioni) rimane in sospeso in attesa del ritorno della funzione.
- **Stack Frame:** È l'area di memoria allocata dinamicamente sullo Stack di sistema ogni volta che si invoca una funzione. Contiene l'indirizzo di ritorno, i parametri passati e le variabili locali.
- **Tail Call Optimization (TCO):** È un'ottimizzazione del compilatore. Quando una funzione è tail-recursive, il compilatore riconosce che lo Stack Frame corrente non ha più utilità. Invece di allocare un nuovo Stack Frame in cima allo Stack per la chiamata successiva, riutilizza e sovrascrive il frame attuale aggiornando solo i parametri.
- **Complessità e Vantaggi:** A livello macchina la ricorsione diventa un semplice ciclo iterativo. La complessità Temporale rimane invariata, ma la complessità Spaziale crolla da $O(n)$ a $O(1)$, scongiurando completamente il rischio di Stack Overflow.

● Rappresentazione di un grafo (lista/matrice delle adiacenze) con complessità

• Rappresentazione di un grafo (Lista vs Matrice delle adiacenze):

- **Matrice delle adiacenze:** È una matrice quadrata $V \times V$ in cui la cella $[i][j]$ vale 1 (o il peso dell'arco) se c'è un collegamento da i a j , e 0 altrimenti.
 - **Complessità Spaziale:** $O(V^2)$. Molto dispendiosa per grafi "sparsi" (pochi archi), poiché memorizza esplicitamente anche l'assenza di archi.
 - **Complessità Temporale:** Verificare se esiste l'arco (i, j) è immediato in tempo $O(1)$. Trovare tutti i vicini di un nodo richiede di scorrere un'intera riga, costando $O(V)$.
 - **Lista delle adiacenze:** È un array (o un vettore dinamico) di dimensione V , in cui ogni cella i contiene il puntatore alla testa di una lista concatenata che racchiude solo i nodi direttamente raggiungibili da i .
 - **Complessità Spaziale:** $O(V + E)$, dove E sono gli archi. È ottimale per la memoria perché alloca spazio solo per i collegamenti reali.
 - **Complessità Temporale:** Verificare l'esistenza di un arco (i, j) richiede di scorrere la lista di i , con costo $O(\text{grado_di_}i)$, nel caso pessimo $O(V)$. Trovare tutti i vicini è molto veloce, per questo è la struttura d'elezione per le visite (DFS e BFS).
-

Data : 19 -> 21 (No lab) (12/09/2025)

Programmazione (Cabodi) :

● Trovare chiave a valore minimo in heap (a voce)

Trovare il valore minimo in un Heap (a voce):

- **Caso 1: Min-Heap.** Se la struttura è un Min-Heap, per definizione l'elemento con il valore minimo si trova alla radice dell'albero, ovvero all'indice 0 dell'array. La complessità Temporale è immediata e vale **$O(1)$** .

- **Caso 2: Max-Heap (La vera domanda d'esame):** In un Max-Heap, il valore minimo non può avere figli (altrimenti violerebbe la regola che ogni padre è maggiore dei suoi figli). Quindi, il minimo deve trovarsi obbligatoriamente in uno dei nodi "foglia".
- **Dove sono le foglie:** In un heap rappresentato tramite array di dimensione n , le foglie occupano la seconda metà esatta dell'array, ovvero dagli indici da $(n/2)$ fino a $(n-1)$.
- **Logica dell'algoritmo:** Non avendo una regola di ordinamento orizzontale, siamo obbligati a scorrere linearmente tutte le foglie (la seconda metà dell'array) e confrontarle una a una per trovare il minimo globale.
- **Complessità (Max-Heap):** Tempo $O(n)$, perché dobbiamo ispezionare $n/2$ elementi. Spazio $O(1)$.

```
#include <limits.h>

// Funzione per trovare il minimo in un Max-Heap
int find_min_in_max_heap(int heap[], int n) {
    // Gestione array vuoto
    if (n <= 0) return INT_MIN;

    // Inizializzo il minimo al massimo valore possibile
    int min_val = INT_MAX;

    // Le foglie iniziano esattamente all'indice n/2 (troncato all'intero)
    // Scorro solo la seconda metà dell'array, risparmiando n/2 cicli inutili
    for (int i = n / 2; i < n; i++) {

        // Se trovo una foglia con un valore più basso, aggiorno il minimo
        // temporaneo
        if (heap[i] < min_val) {
            min_val = heap[i];
        }
    }

    return min_val;
}
```

Teoria (Camurati) :

- Tabella di Hash, delete in tab di Hash
- Grafo completo

Teoria (Camurati/Ferrero)

- **Tabella di Hash (Concetti Base):** È una struttura dati progettata per implementare un dizionario (operazioni di insert, search, delete) nel tempo medio più rapido possibile, idealmente $O(1)$. Utilizza una "funzione di hash" per trasformare la chiave di ricerca

nell'indice di un array. La complessità spaziale di base è $O(m)$, dove m è la dimensione dell'array.

- **Gestione delle Collisioni:** Poiché l'universo delle chiavi è molto più grande della dimensione dell'array, due chiavi diverse possono generare lo stesso indice (collisione). Si risolve in due modi: tramite liste concatenate esterne all'array (Chaining) o ispezionando altre celle vuote nell'array stesso (Open Addressing).
- **Cancellazione (Delete) con Chaining:** Molto semplice. Si calcola l'indice con l'hash, si scorre la lista concatenata associata a quello slot e, trovata la chiave, la si scollega aggiornando i puntatori (come in una normale Linked List) e si dealloca il nodo. Complessità Temporale: $O(1)$ in media, $O(n)$ nel caso pessimo se tutti gli elementi hanno colliso nello stesso slot.
- **Cancellazione (Delete) con Open Addressing (Il Trabocchetto):** In tecniche come il Linear Probing, l'elemento è salvato direttamente nell'array. Se troviamo la chiave da cancellare, **non possiamo semplicemente rimettere la cella a NULL**. Se lo facessimo, spezzerebbero la catena di ispezione per altri elementi che avevano colliso in precedenza e che sono stati inseriti più avanti nell'array: la loro ricerca si fermerebbe prematuramente a quel NULL.
- **Soluzione del Marker (Tombstone):** La cella cancellata viene sovrascritta con un valore speciale (un flag o un "marker" di stato DELETED). La funzione di search considererà la cella DELETED come occupata (e continuerà a ispezionare le celle successive), mentre la funzione di insert la considererà come vuota (e potrà sovrascriverla con una nuova chiave).
- **Grafo Completo:** È un grafo non orientato in cui esiste un arco per ogni singola coppia di vertici distinti. Il grado di ogni singolo vertice è esattamente $V-1$.
- **Numero di archi e Complessità:** Il numero totale di archi E è dato dalla formula combinatoria $V*(V-1)/2$. La complessità spaziale è $O(V^2)$ a prescindere dalla struttura usata (sia matrice di adiacenza che liste), poiché il numero di archi cresce quadraticamente rispetto ai vertici.

Data : 15 -> 18 (No Lab) (21/07/2025)

Programmazione (Cabodi) :

Implementare Linear Probing

❓ **Concetto:** Il Linear Probing (Ispezione Lineare) è una tecnica di indirizzamento aperto (Open Addressing) per risolvere le collisioni in una Tabella Hash. Gli elementi sono memorizzati direttamente nell'array.

❓ **Logica dell'algoritmo (Inserimento):** Si calcola l'indice di partenza tramite la funzione di hash. Se la cella è già occupata, si ispeziona linearmente la cella successiva aggiungendo 1 all'indice. Se si arriva alla fine dell'array, si riparte dall'inizio usando l'operatore modulo ($\% \text{size}$). Per farlo funzionare correttamente (specie in tabelle in cui si effettuano anche cancellazioni), ogni slot deve avere uno "Stato": VUOTO, OCCUPATO o CANCELLATO.

❓ **Complessità:** Tempo $O(1)$ nel caso medio se il fattore di carico è tenuto basso. Tempo $O(n)$ nel caso pessimo (tabella quasi piena) a causa del fenomeno del *Primary Clustering* (ammassamento primario, dove blocchi di celle occupate si fondono allungando drasticamente i tempi di ispezione). Spazio ausiliario $O(1)$.

```
#include <stdbool.h>
#include <stdio.h>

// Definisco i possibili stati di uno slot della tabella
typedef enum { EMPTY, OCCUPIED, DELETED } SlotState;

// Struttura dello slot della Hash Table
typedef struct {
    int key;
    SlotState state;
} HashSlot;

// Struttura della Hash Table
typedef struct {
    int size;
    HashSlot* table; // Array di HashSlot allocato dinamicamente
} HashTable;

// Inserimento con tecnica Linear Probing
bool hash_insert_linear(HashTable* ht, int key) {
    // Calcolo l'indice di partenza (hash della chiave)
    int index = key % ht->size;
    int start_index = index; // Lo salvo per capire se ho fatto un giro completo

    // Inizio l'ispezione lineare. Continuo finché trovo celle OCCUPATE
    while (ht->table[index].state == OCCUPIED) {

        // Se la chiave esiste già (evito duplicati), interrompo e ritorno false
        if (ht->table[index].key == key) {
            return false;
        }

        // Ispeziono la cella successiva. Il modulo 'size' mi fa ripartire da 0
        // se sforo il limite fisico dell'array (circolarità)
        index = (index + 1) % ht->size;
    }

    // Se siamo arrivati qui, la cella è vuota o cancellata.
    // In questo esempio non gestiamo la cancellazione, quindi è vuoto.
    ht->table[start_index].key = key;
    ht->table[start_index].state = OCCUPIED;
    return true;
}
```

```

        index = (index + 1) % ht->size;

        // Se l'indice corrente torna a essere uguale a quello di partenza,
        // significa che ho ispezionato tutta la tabella ed è piena. Inserimento
fallito.
        if (index == start_index) {
            return false;
        }
    }

    // Sono uscito dal while: ho trovato una cella EMPTY o DELETED.
    // Inserisco fisicamente la chiave e aggiorno lo stato dello slot a OCCUPIED.
    ht->table[index].key = key;
    ht->table[index].state = OCCUPIED;

    return true; // Inserimento avvenuto con successo
}

```

Teoria (Camurati) :

Cos'è un BST?

Definisci altezza di un albero

Cos'è il partizionamento?

❓ **Definizione di BST (Albero Binario di Ricerca):** È un albero binario che rispetta una rigorosa proprietà di ordinamento: per ogni nodo X, tutti i nodi presenti nel suo sottoalbero sinistro hanno una chiave minore di X, e tutti i nodi nel suo sottoalbero destro hanno una chiave maggiore di X. Questa proprietà permette di scartare metà dell'albero a ogni passo di ricerca.

❓ **Definizione di Altezza di un albero:** L'altezza di un nodo è la lunghezza (espressa in numero di archi) del cammino più lungo da quel nodo verso una foglia discendente. L'altezza dell'intero albero coincide con l'altezza della sua radice. Per convenzione, un albero vuoto ha altezza -1, e un albero con un solo nodo (la radice) ha altezza 0.

❓ **Complessità e legame con l'altezza:** Tutte le operazioni fondamentali del BST (ricerca, inserimento, cancellazione) hanno una complessità temporale di $O(h)$, dove h è l'altezza. Nel caso medio (albero bilanciato) h vale $O(\log n)$. Nel caso pessimo (albero degenerare, simile a una lista concatenata) h vale $O(n)$.

❓ **Cos'è il partizionamento di un BST:** È un'operazione che riorganizza l'Albero Binario di Ricerca in modo tale che un nodo specifico, tipicamente quello avente rango k (ovvero il k -esimo elemento più piccolo dell'albero), venga portato in cima e diventi la **nuova radice** dell'albero (o del sottoalbero corrente).

❓ **Meccanismo e logica (Rotazioni):** L'algoritmo si basa sulla conoscenza del numero di nodi presenti nei sottoalberi (spesso salvato in un campo count nei nodi). Funziona ricorsivamente:

- Se il k-esimo elemento si trova nel sottoalbero sinistro, si chiama il partizionamento ricorsivamente sul figlio sinistro. Quando la ricorsione ritorna (e il nodo target è diventato la radice del sottoalbero sinistro), si esegue una **rotazione a destra** sul nodo corrente per portare il target al posto della radice.
- Se si trova nel sottoalbero destro, si cerca nel figlio destro (aggiornando l'indice k) e, al ritorno, si applica una **rotazione a sinistra**.

❓ **Complessità:** L'operazione richiede di scendere lungo un cammino per trovare il k-esimo nodo e di eseguire una serie di rotazioni (costo $O(1)$ ciascuna) risalendo verso la radice. La complessità Temporale è quindi **$O(h)$** , dove h è l'altezza dell'albero. La complessità Spaziale è **$O(h)$** per via dello stack delle chiamate ricorsive.

Data : 17->19(Lab 0.75) (21/07/2025)

Programmazione (Cabodi) :

Q: implementare una funzione STdelete in una hashtable con Linear chaining

Implementazione STdelete in Hash Table con Linear Chaining:

- **Concetto:** In una tabella hash con chaining (concatenamento), ogni slot dell'array contiene il puntatore alla testa di una lista concatenata. La cancellazione (STdelete) consiste nel cercare il nodo in questa lista e rimuoverlo fisicamente riagganciando i puntatori.
- **Logica dell'algoritmo:** Si usa la funzione di hash per trovare l'indice (slot) corretto. Si scorre la lista concatenata associata allo slot usando due puntatori: curr (nodo corrente) e prev (nodo precedente). Se la chiave viene trovata, si scollega il nodo: se è il primo nodo (testa), si aggiorna il puntatore base dell'array; altrimenti si collega il prev al successivo di curr. Infine, si libera la memoria.
- **Complessità:** Tempo $O(1)$ nel caso medio (con fattore di carico basso, le liste sono molto corte). Tempo $O(n)$ nel caso pessimo (se tutte le chiavi hanno colliso nello stesso slot, degenerando in una singola lista lineare). Spazio ausiliario $O(1)$.

```
// Struttura del nodo della lista concatenata
typedef struct Node {
    int key;
```

```

    struct Node* next;
} Node;

// Struttura della Hash Table (Symbol Table)
typedef struct HashTable {
    int size;
    Node** table; // Array di puntatori alle teste delle liste
} HashTable;

// Funzione hash (modulo base)
int hash_function(int key, int size) {
    return key % size;
}

// Funzione di cancellazione (STdelete)
bool STdelete(HashTable* ht, int key) {
    // 1. Calcolo lo slot associato alla chiave
    int index = hash_function(key, ht->size);

    // 2. Inizializzo i puntatori per scorrere la lista
    Node* curr = ht->table[index];
    Node* prev = NULL;

    // 3. Scorro la lista finché non finisce o non trovo la chiave
    while (curr != NULL) {
        if (curr->key == key) {

            // Caso A: Il nodo da eliminare è la TESTA della lista
            if (prev == NULL) {
                // Faccio puntare lo slot dell'array al secondo elemento
                ht->table[index] = curr->next;
            }
            // Caso B: Il nodo è in mezzo o in fondo alla lista
            else {
                // Salto il nodo corrente, collegando il precedente al successivo
                prev->next = curr->next;
            }

            // 4. Libero fisicamente la memoria del nodo isolato
            free(curr);

            return true; // Cancellazione completata con successo
        }

        // Avanzamento dei puntatori per l'iterazione successiva
        prev = curr;
        curr = curr->next;
    }

    // Se arrivo qui, la chiave non era presente nella tabella

```



```
return false;
}
```

Teoria (Camurati) :

Q: cosa sono le partizioni di un insieme e come si trovano. Poi spiegare l'algoritmo di ER e se è un algoritmo derivato dalla combinatoria

Q: cosa sono i powerset(solo definizione)

Q: cosa è una coda a priorità e come posso implementarla

Q: quali altri metodi posso usare oltre all'heap e complessità delle funzioni PQInsert e PQchange in un vettore

? Partizioni di un insieme e Algoritmo ER (Equivalence Relations):

- **Definizione:** Una partizione è una suddivisione degli elementi di un insieme in sottoinsiemi (chiamati "blocchi") non vuoti e mutuamente disgiunti, la cui unione ricompona l'insieme originale. Non conta l'ordine dei blocchi né l'ordine degli elementi al loro interno.
- **Algoritmo ER:** Genera le partizioni sfruttando la biunivocità tra partizioni e relazioni di equivalenza (ER). Utilizza una ricorsione ad albero: per ogni nuovo elemento da inserire, l'algoritmo itera sui blocchi attualmente esistenti. L'elemento può essere aggiunto a uno qualsiasi dei blocchi già creati, oppure può formare un blocco completamente nuovo.
- **Modello combinatorio:** L'algoritmo **non utilizza** i 4 classici modelli del calcolo combinatorio (disposizioni, combinazioni, ecc.) perché in quei modelli si scelgono k elementi su n , mentre nelle partizioni si usano tutti gli n elementi raggruppandoli. Il modello matematico di riferimento si basa sui Numeri di Stirling di seconda specie (partizioni in esattamente k blocchi) e sui Numeri di Bell (totale di tutte le partizioni possibili).
- **Complessità:** Tempo $O(B_n)$ dove B_n è l' n -esimo Numero di Bell (crescita iperesponenziale). Spazio $O(n)$ per l'array di supporto e lo stack ricorsivo.

? Powerset (Insieme delle parti):

- **Definizione:** Il Powerset di un insieme S è l'insieme composto da tutti i possibili sottoinsiemi di S , includendo sempre l'insieme vuoto e l'insieme S stesso. Se l'insieme di partenza ha n elementi, il suo Powerset conterrà esattamente 2^n sottoinsiemi.

? Coda a priorità (Priority Queue) e implementazione:

- **Definizione:** È un Tipo di Dato Astratto (ADT) in cui ogni elemento inserito ha associato un valore di "priorità". A differenza di una coda standard (FIFO), l'operazione di

estrazione rimuove e restituisce sempre l'elemento con la priorità massima (o minima), indipendentemente dall'ordine di arrivo.

- **Implementazione standard:** Si implementa in modo ottimale utilizzando un **Heap** (Max-Heap o Min-Heap), ovvero un albero binario quasi completo memorizzato in un array.
- **Complessità Heap:** Inserimento (PQInsert) ed estrazione (PQExtract) costano Tempo $O(\log n)$ grazie alle operazioni di Heapify. La lettura del massimo costa $O(1)$. Spazio $O(n)$.

❓ Metodi alternativi all'Heap e complessità su Vettore:

- **Metodi alternativi:** Oltre all'Heap, una PQ può essere implementata usando un vettore non ordinato, un vettore ordinato, una lista concatenata (ordinata o non ordinata), oppure un Albero Binario di Ricerca (BST).
- **Vettore NON ordinato:** * **PQInsert:** Tempo **$O(1)$** . Basta inserire l'elemento in coda all'array (se c'è spazio).
 - **PQchange (modifica priorità):** Tempo **$O(n)$** . Non essendo ordinato, bisogna scorrere l'intero vettore per trovare l'elemento, e poi aggiornarlo (l'aggiornamento in sé costa $O(1)$, ma la ricerca domina il costo).
- **Vettore ORDINATO:**
 - **PQInsert:** Tempo **$O(n)$** . Si può trovare la posizione corretta in $O(\log n)$ con la ricerca binaria, ma l'inserimento fisico richiede di traslare (shiftare) tutti gli elementi successivi di una posizione verso destra, operazione che costa $O(n)$.
 - **PQchange:** Tempo **$O(n)$** . Anche conoscendo la posizione, cambiare la priorità rompe l'ordinamento: bisogna estrarre l'elemento e reinserirlo nella nuova posizione corretta, traslando gli elementi in mezzo.

Data : 16->19 (no lab) (21/07/2025)

Programmazione (Cabodi) :

● HeapInsert

❓ **Concetto HeapInsert:** L'inserimento in un Heap (consideriamo un Max-Heap) avviene aggiungendo fisicamente l'elemento in fondo all'array, per poi ripristinare la proprietà di ordinamento facendolo "galleggiare" verso l'alto (Heapify-Up).

❓ **Logica dell'algoritmo:**

1. Si posiziona la nuova chiave nell'ultima cella disponibile dell'array (incrementando la dimensione totale), il che preserva la struttura di albero quasi completo.
2. Si esegue l'operazione di **Heapify-Up** (o Fix-Up): si confronta il nuovo elemento con il proprio padre. Se il figlio è strettamente maggiore del padre, si scambiano. Il processo si ripete risalendo l'albero finché l'elemento non trova un padre maggiore o uguale, oppure fino a raggiungere la radice.

🔗 **Complessità:** Tempo $O(\log n)$ nel caso peggior, perché l'elemento può dover risalire dalla foglia fino alla radice, compiendo un numero di scambi pari all'altezza dell'albero. Spazio $O(1)$ perché le operazioni avvengono manipolando gli indici direttamente nell'array.

```
#include <stdio.h>
#include <stdbool.h>

// Struttura che incapsula l'array dell'Heap e le sue dimensioni
typedef struct {
    int* array;
    int size;      // Numero attuale di elementi
    int capacity;  // Dimensione massima allocata
} MaxHeap;

// Funzione per inserire un elemento nel Max-Heap
bool heap_insert(MaxHeap* heap, int key) {
    // Controllo preventivo per evitare overflow dell'array
    if (heap->size >= heap->capacity) {
        return false;
    }

    // Posiziono il nuovo elemento nella prima posizione libera in fondo all'array
    int i = heap->size;
    heap->array[i] = key;

    // Incremento la dimensione, certificando l'inserimento strutturale
    heap->size++;

    // Calcolo l'indice del nodo padre usando la formula per gli array 0-indexed
    int parent = (i - 1) / 2;

    // Avvio la procedura di Heapify-Up (Risolita iterativa)
    // Continuo a scambiare finché non sono alla radice (i > 0)
    // e finché il valore del nodo corrente è maggiore di quello del suo padre
    while (i > 0 && heap->array[i] > heap->array[parent]) {

        // Eseguo lo scambio (swap) per far salire l'elemento maggiore
        int temp = heap->array[i];
        heap->array[i] = heap->array[parent];
        heap->array[parent] = temp;
    }
}
```

```

    // Aggiorno l'indice corrente: ora mi trovo nella posizione del padre
    i = parent;

    // Ricalcolo il nuovo padre per il prossimo ciclo del while
    parent = (i - 1) / 2;
}

// Inserimento e riordino completati con successo
return true;
}

```

Teoria (Camurati) :

- Heap e domande riguardo la struttura e i vantaggi e gli svantaggi
- Come si può implementare un grafo e vantaggi e svantaggi (matrice di adiacenza e lista di adiacenza)

Teoria (Camurati)

- **Struttura dell'Heap:** Un Heap (solitamente un Binary Heap) è un albero binario "quasi completo", ovvero tutti i livelli sono completamente riempiti tranne al più l'ultimo, che viene riempito rigorosamente da sinistra verso destra. Questa regolarità permette di memorizzarlo implicitamente in un semplice array, senza l'uso di puntatori.
- **Relazione degli indici:** In un array a partire dall'indice 0, per un generico nodo all'indice i : il figlio sinistro si trova all'indice $2i + 1$, il figlio destro a $2i + 2$, e il padre all'indice $(i - 1) / 2$ (arrotondato per difetto).
- **Proprietà dell'ordinamento parziale:** In un Max-Heap, il valore di ogni nodo padre è sempre maggiore o uguale a quello dei suoi figli (la radice è il massimo assoluto). In un Min-Heap è minore o uguale (la radice è il minimo assoluto).
- **Vantaggi dell'Heap:**
 - **Efficienza spaziale (In-place):** Complessità spaziale $O(n)$ pura per memorizzare i dati, senza lo spreco di memoria (overhead) tipico dei puntatori usati negli alberi classici.
 - **Accesso all'estremo:** Trovare il massimo (nel Max-Heap) o il minimo (nel Min-Heap) costa tempo $O(1)$, poiché è sempre la radice all'indice 0.
 - **Prestazioni garantite:** Inserimento ed estrazione costano sempre Tempo $O(\log n)$ nel caso pessimo, perché l'albero è strutturalmente bilanciato per costruzione.
 - **Build-Heap efficiente:** La costruzione di un Heap partendo da un array completamente disordinato richiede tempo lineare $O(n)$, non $O(n \log n)$.

- **Svantaggi dell'Heap:**
 - **Ricerca generica lenta:** A differenza del BST (Binary Search Tree), non c'è ordinamento orizzontale. Cercare un elemento specifico richiede di ispezionare l'intero array, con tempo $O(n)$.
 - **Fusione costosa:** Unire due Heap distinti implementati su array richiede di creare un nuovo array e richiamare il Build-Heap, costando tempo $O(n)$.
- **Implementazione Grafo - Matrice di Adiacenza:** Si usa una matrice quadrata $V \times V$. Se c'è un arco dal vertice i al vertice j , la cella $[i][j]$ contiene 1 (o il peso dell'arco); altrimenti 0.
 - **Vantaggi:** Verificare se esiste uno specifico arco (i, j) è immediato (accesso diretto alla matrice in Tempo $O(1)$).
 - **Svantaggi:** Complessità Spaziale $O(V^2)$. Se il grafo è "sparso" (ha pochi archi rispetto al massimo possibile), gran parte della matrice sarà piena di zeri, spreco di enormi quantità di memoria. Inoltre, trovare tutti i vicini di un nodo richiede la scansione di un'intera riga, costando Tempo $O(V)$ indipendentemente da quanti vicini reali ci siano.
- **Implementazione Grafo - Lista di Adiacenza:** Si usa un array (o vettore) di dimensione V . Ogni cella i rappresenta un vertice e contiene il puntatore alla testa di una lista concatenata in cui sono salvati i nodi di destinazione degli archi uscenti da i .
 - **Vantaggi:** Complessità Spaziale $O(V + E)$. È estremamente efficiente per la memoria nei grafi sparsi, poiché alloca spazio solo per gli archi realmente esistenti. Perfetta per le visite (DFS, BFS), in quanto scorrere i vicini di un nodo i richiede un tempo strettamente proporzionale al suo numero di archi (grado del vertice).
 - **Svantaggi:** Per verificare se esiste l'arco (i, j) bisogna scorrere linearmente la lista concatenata del vertice i , il che costa Tempo $O(\text{grado}(i))$, declassando nel caso pessimo a $O(V)$. Presenta inoltre il piccolo overhead di memoria per i puntatori next delle liste.

Data : 16->20 (no lab) (04/07/2025)

Programmazione (Cabodi) :

● Dati due insiemi implementati come due liste ordinate, scrivere una funzione che ritorni l'intersezione degli insiemi

● Come sarebbe cambiata la funzione con insiemi implementati tramite vettore?

Programmazione (Cabodi)

- **Intersezione di due insiemi (liste ordinate):**
 - **Logica:** Sfruttiamo l'ordinamento. Usiamo due puntatori per scorrere parallelamente le due liste. Se il valore puntato nella prima lista è minore, avanziamo solo nella prima (perché potremmo trovare un match più avanti). Se è maggiore, avanziamo nella seconda. Se i valori coincidono, abbiamo trovato un'intersezione: creiamo un nuovo nodo per la lista risultato e avanziamo entrambi i puntatori.
 - **Complessità:** Tempo $O(n + m)$, dove n e m sono le lunghezze delle liste. Ogni elemento viene ispezionato al massimo una volta in una singola passata lineare. Spazio $O(\min(n, m))$ per l'allocazione della nuova lista risultante.

Come sarebbe cambiata la funzione con vettori (array):

- **Logica di attraversamento:** Sostanzialmente identica. Invece di usare puntatori ai nodi (`list1->next`), userei due indici interi (i e j) che partono da 0 e vengono incrementati ($i++$, $j++$).
- **Gestione della memoria:** Invece di allocare singoli nodi di volta in volta, dovrei pre-allocare un array dinamico (es. con `malloc`) di dimensione pari al più piccolo tra i due insiemi (dimensione massima possibile dell'intersezione). Terrei traccia di un contatore k per inserire gli elementi (`risultato[k++] = arr1[i]`).
- **Complessità e Performance:** Le complessità asintotiche rimangono identiche: Tempo $O(n + m)$ e Spazio $O(\min(n, m))$. Tuttavia, a livello pratico, i vettori risulterebbero significativamente più veloci in esecuzione grazie al principio di **località spaziale** (Memory Caching), che evita continui e frammentati accessi alla memoria Heap tipici delle liste concatenate.

```
#include <stdio.h>
#include <stdlib.h>

// Definizione del nodo della lista
typedef struct Node {
    int val;
    struct Node* next;
} Node;

// Funzione per intersecare due insiemi ordinati
Node* intersect_sorted_lists(Node* list1, Node* list2) {
    // Nodo "dummy" (fittizio) per semplificare l'inserimento in coda alla nuova lista
    // Evita di dover gestire a parte il caso speciale del primo inserimento (testa = NULL)
```

```

Node dummy;
dummy.next = NULL;
Node* tail = &dummy; // Puntatore di coda per inserire in O(1)

// Scorro finché non esaurisco almeno una delle due liste
while (list1 != NULL && list2 != NULL) {

    // Il valore di list1 è più piccolo: avanzo list1 per cercare un valore più
grande
    if (list1->val < list2->val) {
        list1 = list1->next;
    }
    // Il valore di list2 è più piccolo: avanzo list2
    else if (list1->val > list2->val) {
        list2 = list2->next;
    }
    // I valori sono uguali: elemento trovato in entrambi gli insiemi
    else {
        // Alloco il nuovo nodo per la lista intersezione
        Node* new_node = (Node*)malloc(sizeof(Node));
        new_node->val = list1->val;
        new_node->next = NULL;

        // Lo aggancio in coda alla lista risultato
        tail->next = new_node;
        tail = new_node; // Aggiorno la coda

        // Avanzo entrambi i puntatori originali per continuare la ricerca
        list1 = list1->next;
        list2 = list2->next;
    }
}

// Ritorno la testa reale della nuova lista (salto il nodo dummy)
return dummy.next;
}

```

Teoria (Camurati) :

- Complessita' caso peggiore del Quicksort, in che situazione capita la complessita' di caso peggiore? (Vettore ordinato in maniera opposta a quella desiderata e pivot ad un estremo)
- Cosa significa N nell'equazione alle ricorrenze del Quicksort?
- Definizione di kernel DAG
- Legame tra kernel DAG e componenti fortemente connesse

Teoria (Camurati)

- **Complessità caso peggiore del Quicksort e quando si verifica:**

- **Complessità:** Tempo $O(n^2)$. Spazio $O(n)$ per lo stack delle chiamate ricorsive (poiché l'albero di ricorsione degenera in una lista).
- **Situazione scatenante:** Il caso pessimo si verifica quando la funzione di partizionamento divide l'array nel modo più sbilanciato possibile: una parte con $n-1$ elementi e l'altra con 0 elementi. Questo accade sistematicamente se l'array è già ordinato (in modo crescente o decrescente) e l'algoritmo sceglie sempre come **pivot** il primo o l'ultimo elemento dell'array, che in quel caso corrisponde al minimo o al massimo assoluto.

- **Significato di 'n' nell'equazione alle ricorrenze del Quicksort:**

- L'equazione nel caso medio/ottimo è $T(n) = 2T(n/2) + O(n)$, nel caso pessimo è $T(n) = T(n-1) + T(0) + O(n)$.
- Il termine **n** rappresenta la dimensione del sottoproblema corrente, ovvero il numero di elementi del sotto-array che si sta analizzando in quella specifica chiamata ricorsiva.
- Il termine lineare **$O(n)$** rappresenta il costo della procedura di **partizionamento**: per dividere i valori minori e maggiori del pivot, l'algoritmo deve obbligatoriamente scansionare ed effettuare confronti su tutti gli 'n' elementi dell'attuale sotto-array.

- **Definizione di Kernel DAG:**

- Un Kernel DAG (Grafo Diretto Aciclico Condensato) è un meta-grafo derivato da un grafo orientato di partenza. Si ottiene "collassando" (condensando) intere porzioni del grafo in singoli macro-nodi. Per definizione, non contiene alcun ciclo.
- **Complessità costruttiva:** Costruire il Kernel DAG richiede di identificare prima le componenti e poi unire gli archi, operazione eseguibile in Tempo $O(V + E)$ e Spazio $O(V + E)$.

- **Legame tra Kernel DAG e Componenti Fortemente Connesse (SCC):**

- Le SCC (Componenti Fortemente Connesse) sono i sottografi massimali in cui ogni nodo è raggiungibile da ogni altro nodo (presenza di cicli chiusi).
- Il legame è strutturale: i macro-nodi del Kernel DAG sono **esattamente** le SCC del grafo originale. Gli archi del Kernel DAG rappresentano i collegamenti a senso unico tra una SCC e un'altra. Poiché tutte le relazioni cicliche sono state inglobate all'interno delle singole SCC, il meta-grafo risultante che le collega è matematicamente garantito essere un DAG (Aciclico).

Data : 15-> 19 (0 lab) (4/07/2025)

Programmazione (Cabodi) :

● **BST: Inserzione in radice di un nodo**

Inserimento in radice di un BST:

- **Concetto:** È una tecnica utile quando gli elementi inseriti più di recente hanno un'alta probabilità di essere cercati subito dopo. Anziché lasciare il nuovo nodo sul fondo (come foglia), lo si forza a diventare la nuova radice dell'albero.
- **Logica dell'algoritmo:** Si tratta di un processo ricorsivo in due fasi. Nella fase di "discesa", si naviga l'albero come in un inserimento normale e si posiziona il nodo come foglia. Nella fase di "risalita" (quando i return si srotolano), si applicano delle **Rotazioni**: se l'inserimento è avvenuto nel sottoalbero sinistro, si fa una rotazione a destra sul nodo corrente; se è avvenuto nel destro, si fa una rotazione a sinistra. Questo "tira su" il nuovo nodo passo dopo passo fino alla radice.
- **Complessità:** Tempo $O(h)$, dove h è l'altezza dell'albero ($O(\log n)$ caso medio, $O(n)$ caso pessimo sbilanciato) per scendere e risalire effettuando rotazioni a costo $O(1)$. Spazio $O(h)$ per lo stack delle chiamate ricorsive.

```
#include <stdlib.h>

// Definizione del nodo
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione ausiliaria per creare fisicamente il nuovo nodo foglia
Node* new_node(int key) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->key = key;
    n->left = n->right = NULL;
    return n;
}

// Rotazione a DESTRA: il figlio sinistro sale e prende il posto della radice
// corrente
Node* right_rotate(Node* root) {
    Node* new_root = root->left;
    root->left = new_root->right; // Il sottoalbero destro del figlio diventa
// sinistro del padre
    new_root->right = root;      // Il vecchio padre scende a destra
```

```

    return new_root;
}

// Rotazione a SINISTRA: il figlio destro sale e prende il posto della radice
corrente
Node* left_rotate(Node* root) {
    Node* new_root = root->right;
    root->right = new_root->left; // Il sottoalbero sinistro del figlio diventa
destro del padre
    new_root->left = root;      // Il vecchio padre scende a sinistra
    return new_root;
}

// Funzione ricorsiva di inserimento in radice
Node* insert_root(Node* root, int key) {
    // Caso base: arrivato nella posizione corretta vuota, creo il nodo
    if (root == NULL) {
        return new_node(key);
    }

    // Se la chiave è minore, scendo ricorsivamente a sinistra
    if (key < root->key) {
        root->left = insert_root(root->left, key);

        // Al ritorno dalla ricorsione, il nuovo nodo è appena diventato il figlio
sinistro.
        // Faccio una rotazione a destra per farlo salire al posto del 'root'
corrente.
        return right_rotate(root);
    }

    // Se la chiave è maggiore, scendo ricorsivamente a destra
    else if (key > root->key) {
        root->right = insert_root(root->right, key);

        // Al ritorno, il nuovo nodo è il figlio destro.
        // Faccio una rotazione a sinistra per farlo salire.
        return left_rotate(root);
    }

    // Se la chiave è già presente (duplicato), non faccio nulla e ritorno il nodo
intatto
    return root;
}

```

Teoria (Camurati) :

● Perchè nella tabella di hash è più efficiente il double hashing rispetto agli altri (ho detto perchè in caso di collisione la somma aggiunta potrebbe non essere lineare ma non credo sia corretta)

- **Tabella di hash con open addressing: eliminazione (utilizzare un vettore per tenere traccia per eventuali collisioni)**

Gemini ha detto

Teoria (Camurati)

- **Perché il Double Hashing è più efficiente:** L'intuizione sulla somma non lineare è sulla strada giusta, ma il termine tecnico corretto riguarda il problema del **Clustering (Ammassamento)**.
- **Il problema degli altri metodi:** Nell'ispezione lineare (Linear Probing, salto di 1), si crea il *Primary Clustering*: blocchi di celle occupate si fondono, creando lunghe sequenze da scorrere. Nell'ispezione quadratica (salto di i^2), si evita il primary ma si crea il *Secondary Clustering*: se due chiavi diverse generano lo stesso hash iniziale, seguiranno esattamente lo stesso e identico percorso di ispezione.
- **Il meccanismo del Double Hashing:** Risolve entrambi i problemi usando una seconda funzione di hash per calcolare il "passo" (lo step di salto). La formula è $h(k, i) = (h_1(k) + i * h_2(k)) \% m$. Poiché $h_2(k)$ dipende dalla chiave stessa, due chiavi che collidono al primo tentativo (avendo lo stesso h_1) avranno quasi certamente un h_2 diverso. Questo garantisce che le loro sequenze di ispezione divergano immediatamente.
- **Complessità:** Tempo $O(1)$ nel caso medio per ricerca e inserimento (con fattore di carico basso), $O(n)$ nel caso pessimo. Spazio $O(m)$ dove m è la dimensione della tabella.
- **Cancellazione in Open Addressing (Tombstone):** Se eliminiamo fisicamente una chiave da una cella rimettendola a NULL (o vuota), spezziamo irreversibilmente la catena di ispezione. Le chiavi inserite successivamente, che avevano colliso in quello slot ed erano state spostate più avanti, diventerebbero irraggiungibili perché la ricerca si fermerebbe al primo NULL incontrato.
- **La soluzione degli stati:** Si implementa aggiungendo un array di supporto (o un campo nella struct) per tracciare lo stato di ogni cella: EMPTY, OCCUPIED, o DELETED (Tombstone).
 - Quando si elimina un elemento, la cella non diventa EMPTY, ma viene marcata come DELETED.
 - Durante una **Ricerca**, la cella DELETED viene trattata come se fosse occupata: l'algoritmo non si ferma, ma continua a saltare.
 - Durante un **Inserimento**, la cella DELETED viene trattata come vuota: l'algoritmo la sovrascrive col nuovo elemento, ottimizzando lo spazio.

- **Complessità della cancellazione:** Tempo $O(1)$ nel caso medio, $O(n)$ nel caso pessimo (se bisogna scorrere lunghe catene di collisioni). Spazio extra $O(m)$ per mantenere l'array degli stati.
-

Data : 15-> 19 (0 lab) (28/05/2025)

Programmazione (Cabodi) :

● Realizza l'operazione di estrazione del massimo da una coda a priorità realizzata mediante lista non ordinata

Programmazione (Cabodi)

- **Estrazione del massimo da coda a priorità (lista non ordinata):**
 - **Concetto:** Poiché la lista non è ordinata, l'elemento con priorità massima può trovarsi ovunque. È obbligatorio scorrere l'intera lista per trovarlo.
 - **Logica dell'algoritmo:** Usiamo due puntatori per l'iterazione (curr e prev) e due puntatori per salvare la posizione del nodo massimo e del suo nodo precedente (max_node e max_prev). Lo scorrimento parallelo serve perché, una volta trovato il massimo, dobbiamo scollegarlo usando il nodo precedente per non spezzare la lista.
 - **Complessità:** Tempo $O(n)$, dove n è il numero di elementi, per scorrere linearmente l'intera struttura. Spazio $O(1)$ poiché la rimozione dei collegamenti e la deallocazione avvengono in loco.

```
#include <stdlib.h>
#include <limits.h>

// Definizione del nodo della coda a priorità
typedef struct Node {
    int val; // Il valore rappresenta la priorità
    struct Node* next;
} Node;

// Passo il puntatore alla testa per riferimento (Node**) perché la testa
// potrebbe cambiare se il nodo con priorità massima è proprio il primo
int extract_max_unordered_pq(Node** head_ref) {
    // Gestione della coda vuota: ritorno il valore minimo rappresentabile
    if (*head_ref == NULL) return INT_MIN;

    // Puntatori per l'iterazione corrente
    Node* curr = *head_ref;
```

```

Node* prev = NULL;

// Puntatori per memorizzare la posizione del massimo assoluto
Node* max_node = *head_ref;
Node* max_prev = NULL;

// Ispezione linearmente tutti i nodi della lista
while (curr != NULL) {
    // Se trovo una priorità strettamente maggiore, aggiorni i "salvataggi"
    if (curr->val > max_node->val) {
        max_node = curr;
        max_prev = prev; // Salvo il precedente per poterlo poi ricollegare
    }

    // Avanzo con i puntatori di iterazione
    prev = curr;
    curr = curr->next;
}

// Estraggo il valore da ritornare prima di deallocare il nodo
int max_val = max_node->val;

// Fase di rimozione (scollegamento):
// Caso A: il nodo con priorità massima era il primo della lista
if (max_prev == NULL) {
    // Sposto la testa della lista sul secondo elemento
    *head_ref = max_node->next;
}
// Caso B: il nodo con priorità massima era in mezzo o in fondo
else {
    // Bypasso il nodo massimo, collegando il suo precedente al suo successivo
    max_prev->next = max_node->next;
}

// Libero la memoria fisica del nodo estratto
free(max_node);

return max_val;
}

```

Teoria (Camurati) :

- Definizione di successore in un bst: spiega come si ottiene e la complessità dell'algoritmo
- Definizione di albero ricoprente minimo

Teoria (Camurati)

- **Definizione di successore in un BST:** Il successore di un nodo X è il nodo che contiene la chiave immediatamente successiva a quella di X in ordine di grandezza

(ovvero la più piccola tra tutte le chiavi maggiori di X). Corrisponde al nodo visitato subito dopo X in una visita simmetrica (in-order).

- **Algoritmo per trovare il successore:** L'algoritmo si divide in due casistiche dipendenti dalla struttura locale:
 - **Caso 1 (X ha un sottoalbero destro):** Il successore è il minimo di questo sottoalbero destro. Si ottiene spostandosi nel figlio destro di X e scendendo continuamente verso il figlio sinistro finché non si trova un nodo senza figlio sinistro (che sarà il minimo cercato).
 - **Caso 2 (X non ha un sottoalbero destro):** Il successore si trova risalendo l'albero. Si segue il puntatore al padre finché non si incontra un nodo che è figlio sinistro del proprio padre. Quel padre è il "primo antenato destro" ed è il successore di X.
 - **Complessità del successore:** Tempo $O(h)$, dove h è l'altezza dell'albero ($O(\log n)$ caso medio bilanciato, $O(n)$ caso pessimo sbilanciato), perché si percorre al massimo una singola volta un ramo verso il basso o verso l'alto. Spazio $O(1)$ in implementazione iterativa (si usano solo puntatori temporanei).
 - **Definizione di albero ricoprente minimo (MST - Minimum Spanning Tree):** Dato un grafo non orientato, connesso e pesato, l'albero ricoprente minimo è un sottografo che tocca (ricopre) tutti i vertici V del grafo originale usando esattamente $V-1$ archi, senza formare cicli (è un albero), tale per cui la somma totale dei pesi di questi $V-1$ archi sia la minima in assoluto tra tutti i possibili alberi ricoprenti estraibili da quel grafo.
 - **Complessità e algoritmi MST:** Si trova utilizzando algoritmi di natura "Greedy" (ingordi, fanno la scelta localmente ottima ad ogni passo). Kruskal ordina gli archi e li aggiunge evitando cicli; Prim fa crescere un singolo albero partendo da un nodo ed espandendosi verso l'arco adiacente più leggero. Entrambi hanno complessità Temporale $O(E \log V)$ e Spazio $O(V + E)$.
-

Data : 16->19 (0 lab) (28/05/2025)

Programmazione (Cabodi) :

● Realizzare una PQinsert con la PQ realizzata tramite lista

Programmazione (Cabodi)

- Implementazione di PQinsert con lista ordinata:

- **Concetto:** Per ottimizzare l'estrazione del massimo (che deve costare $O(1)$), implementiamo la Priority Queue mantenendo la lista sempre ordinata in senso decrescente. Il nodo con la priorità più alta si troverà sempre in testa.
- **Logica dell'algoritmo:** Creiamo il nuovo nodo. Se la lista è vuota o se la nuova priorità è maggiore di quella dell'attuale testa, effettuiamo un inserimento in testa aggiornando il puntatore principale. Altrimenti, usiamo due puntatori ausiliari (curr e prev) per scorrere la lista finché non troviamo un nodo con priorità strettamente inferiore alla nostra, e inseriamo il nuovo nodo esattamente in mezzo.
- **Complessità:** Tempo $O(n)$ nel caso pessimo, perché la nuova priorità potrebbe essere la più bassa in assoluto, costringendoci a scorrere l'intera lista per fare l'inserimento in coda. Spazio $O(1)$ perché l'allocazione del singolo nodo avviene in loco manipolando solo puntatori.

```
#include <stdlib.h>
#include <stdbool.h>

// Definizione del nodo della Coda a Priorità
typedef struct Node {
    int val; // Il valore numerico rappresenta il livello di priorità
    struct Node* next;
} Node;

// Passo il puntatore alla testa per riferimento (Node**) per poterlo modificare
// nel caso in cui l'inserimento avvenga proprio in prima posizione
bool PQinsert_ordered_list(Node** head_ref, int val) {

    // Alloco fisicamente la memoria per il nuovo elemento da accodare
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (new_node == NULL) {
        return false; // Gestione del fallimento di allocazione
    }
    new_node->val = val;

    // Caso A: La lista è vuota, OPPURE la nuova priorità sconfigge l'attuale
    massimo
    // Il nuovo nodo diventa obbligatoriamente la nuova testa della lista
    if (*head_ref == NULL || (*head_ref)->val < val) {
        new_node->next = *head_ref;
        *head_ref = new_node;
        return true;
    }

    // Caso B: Il nuovo nodo va inserito in mezzo o in fondo alla coda
```

```

// Prendo due puntatori per "inseguire" la posizione corretta
Node* curr = *head_ref;
Node* prev = NULL;

// Scorro la lista in avanti finché non arrivo alla fine (NULL)
// o finché non trovo un elemento con priorità strettamente MINORE della mia.
// L'uso di >= garantisce che a parità di priorità, l'ultimo arrivato si accodi
(stabilità)
while (curr != NULL && curr->val >= val) {
    prev = curr;
    curr = curr->next;
}

// Ho trovato il punto di rottura. Inserisco 'new_node' tra 'prev' e 'curr'
prev->next = new_node;
new_node->next = curr;

return true;
}

```

Teoria (Camurati) :

- **Equazioni alle ricorrenze del divide and conquer con particolare accezione al MergeSort**
- **Definizione di componente fortemente connessa (mi ha fatto una domanda extra riguardo il significato di “massimale” della definizione)**
 - **Equazione alle ricorrenze del Divide et Impera:** Il paradigma Divide et Impera spezza un problema in sotto-problemi più piccoli, li risolve e ne combina i risultati. L'equazione generale che ne descrive il tempo di esecuzione è $T(n) = a * T(n/b) + f(n)$, dove 'a' è il numero di sotto-problemi, 'n/b' è la dimensione di ciascun sotto-problema, e 'f(n)' è il costo per dividere il problema e combinare le soluzioni.
 - **Equazione del MergeSort:** Il MergeSort divide sempre l'array esattamente in due metà ($a = 2$) di dimensione dimezzata ($b = 2$). La divisione logica costa tempo $O(1)$, mentre la fusione (il merge) delle due metà ordinate costa tempo lineare $O(n)$.
 - **Calcolo della complessità (MergeSort):** L'equazione diventa rigorosamente $T(n) = 2T(n/2) + O(n)$. Applicando il Master Theorem (secondo caso), il costo delle chiamate ricorsive ($n^{\log_2(2)} = n^1 = n$) bilancia esattamente il costo di ricombinazione $f(n) = O(n)$. La complessità Temporale totale è quindi $O(n \log n)$ in ogni caso (ottimo, medio, pessimo). La complessità Spaziale è $O(n)$ per l'allocazione dell'array ausiliario durante la fase di fusione.
 - **Componente Fortemente Connessa (SCC):** In un grafo orientato, una SCC è un sottoinsieme di vertici nel quale, presa qualsiasi coppia di nodi u e v, esiste sempre un

cammino orientato per andare da u a v , e un cammino orientato per tornare da v a u . Di fatto, racchiude uno o più cicli interconnessi.

- **Significato di "Massimale":** La parola "massimale" nella definizione indica che la componente non può essere espansa ulteriormente. Significa che non è possibile prendere nessun altro vertice dal resto del grafo e aggiungerlo a questo sottoinsieme senza distruggere la proprietà di forte connessione (ovvero, il nuovo vertice inserito non sarebbe in grado di raggiungere tutti gli altri, o non potrebbe essere raggiunto da tutti gli altri).
 - **Complessità SCC:** Trovare tutte le componenti fortemente connesse di un grafo (usando algoritmi basati su DFS come Kosaraju o Tarjan) richiede Tempo $O(V + E)$ e Spazio $O(V)$.
-

Data : 17 -> 20 (0 lab) (28/05/2025) || QUA

Programmazione (Cabodi) :

Teoria (Camurati) :

- Dag e ordinamento topologico
- Cammini massimi (NP e su DAG)

Teoria (Camurati)

- **DAG (Directed Acyclic Graph):** È un grafo orientato che non contiene alcun ciclo orientato. Partendo da qualsiasi vertice e seguendo la direzione degli archi, è fisicamente impossibile tornare al vertice di partenza.
- **Ordinamento Topologico:** È un ordinamento lineare di tutti i vertici di un DAG tale che, per ogni arco orientato da u a v , il vertice u compare prima del vertice v nella sequenza. Modella perfettamente i problemi di propedeuticità (es. esami da superare o task da eseguire). È fattibile se e solo se il grafo è un DAG (un ciclo creerebbe un paradosso di precedenza).
- **Algoritmo (basato su DFS):** Si lancia una visita DFS (Depth-First Search, esplorazione ricorsiva in profondità fino a un vicolo cieco prima di tornare indietro). Ogni volta che un vertice ha terminato di esplorare tutti i suoi vicini (visita conclusa), viene inserito in

testa a una lista concatenata (o spinto in uno stack). Al termine della DFS su tutti i nodi, la lista/stack contiene l'ordinamento topologico corretto.

- **Complessità Ordinamento Topologico:** Tempo $O(V + E)$ in quanto si esegue una singola passata di DFS. Spazio $O(V)$ per tracciare i nodi visitati e memorizzare la struttura dati del risultato (lista o stack).
 - **Cammini massimi (Problema Generale NP-Arduo):** Trovare il cammino semplice (senza nodi ripetuti) di costo massimo in un grafo generico è un problema intrattabile in tempo polinomiale. Fallisce perché manca la **sottostruttura ottima**: un sottocammino di un cammino massimo non è necessariamente un cammino massimo locale, in quanto la scelta di un percorso più lungo localmente potrebbe consumare vertici necessari per raggiungere la destinazione finale senza chiudere un ciclo.
 - **Cammini massimi su DAG:** Sui DAG il problema diventa magicamente facile. Poiché per definizione non esistono cicli, ogni cammino è implicitamente "semplice". Questo ripristina la proprietà di sottostruttura ottima e permette di risolvere il problema in tempo lineare sfruttando l'ordinamento topologico.
 - **Algoritmo su DAG:** 1. Si calcola l'ordinamento topologico del grafo. 2. Si inizializza un array delle distanze tutte a $-\infty$, tranne il nodo sorgente impostato a 0. 3. Si scorrono i vertici sequenzialmente seguendo l'ordinamento topologico. 4. Per ogni vertice 'u' elaborato, si esegue il **rilassamento** per i cammini massimi su tutti i suoi archi uscenti verso 'v': se $d[u] + \text{peso}(u, v) > d[v]$, allora aggiorno $d[v] = d[u] + \text{peso}(u, v)$. L'ordine topologico garantisce che quando elaboro 'u', la sua distanza massima reale dalla sorgente è già stata calcolata in modo definitivo.
 - **Complessità Cammini massimi su DAG:** Tempo $O(V + E)$ per ottenere l'ordinamento topologico e per l'unica passata di rilassamento su nodi e archi. Spazio $O(V)$ per memorizzare le distanze parziali.
-

Data : 16 -> 19 (0 lab) (28/05/2025)

Programmazione (Cabodi) :

● dato un vertice v di un grafo, contare quanti sono i vertici raggiungibili da esso in due passi, non serve la ricorsione, occhio a non contare due volte lo stesso vertice

Programmazione (Cabodi)

- **Logica dell'algoritmo:** Il problema chiede di trovare i nodi raggiungibili in esattamente due passi. Bisogna prima esplorare la lista di adiacenza del vertice 'v' (nodi a distanza 1). Per ognuno di questi nodi, si esplora la rispettiva lista di adiacenza (nodi a distanza 2).
- **Gestione duplicati:** Nodi diversi a distanza 1 potrebbero puntare allo stesso nodo a distanza 2. Per evitare di contare la destinazione due volte, si alloca un array di booleani counted grande quanto il numero di vertici del grafo.
- **Complessità Temporale:** $O(\text{grado}(v) * \text{max_grado})$, che nel caso pessimo analizzando un grafo intero è limitato da $O(V + E)$.
- **Complessità Spaziale:** $O(V)$ per l'allocazione dell'array di supporto counted.

```
#include <stdlib.h>
#include <stdbool.h>

// Struttura per il nodo della lista di adiacenza
typedef struct EdgeNode {
    int v; // Indice del vertice destinazione
    struct EdgeNode* next; // Puntatore al prossimo arco
} EdgeNode;

// Struttura del grafo basata su liste di adiacenza
typedef struct {
    int V; // Numero totale di vertici nel grafo
    EdgeNode** adj; // Array di puntatori alle teste delle liste di adiacenza
} Graph;

// Funzione per contare i vertici raggiungibili in esattamente due passi
int count_two_steps(Graph* g, int start_vertex) {
    // Array di appoggio per segnare i vertici già contati ed evitare duplicati
    // calloc alloca la memoria e inizializza automaticamente tutti i valori a false (0)
    bool* counted = (bool*)calloc(g->V, sizeof(bool));
    if (counted == NULL) return -1; // Ritorno codice di errore se l'allocazione fallisce

    int count = 0; // Contatore finale dei vertici univoci trovati a distanza 2

    // step1_node scorre i vicini diretti del vertice di partenza (i nodi a 1 passo)
    EdgeNode* step1_node = g->adj[start_vertex];

    while (step1_node != NULL) {
        // Prendo l'indice del vicino attuale a distanza 1
        int intermediate_vertex = step1_node->v;
```

```

        // step2_node scorre i vicini del nodo intermedio (i nodi a 2 passi dalla
sorgente)
        EdgeNode* step2_node = g->adj[intermediate_vertex];

        while (step2_node != NULL) {
            int destination_vertex = step2_node->v;

            // Se questo nodo destinazione non è ancora stato contato da un altro
percorso
            if (counted[destination_vertex] == false) {
                // Lo marco come contato per ignorarlo se lo ritrovo da un altro
ramo
                counted[destination_vertex] = true;
                // Incremento il totale dei nodi unici raggiungibili
                count++;
            }

            // Avanzo per esplorare il prossimo nodo a distanza 2
            step2_node = step2_node->next;
        }

        // Avanzo per esplorare il prossimo nodo intermedio a distanza 1
        step1_node = step1_node->next;
    }

    // Libero la memoria dell'array di appoggio per evitare memory leak
    free(counted);

    return count;
}

```

Teoria (Camurati) :

- algoritmo di Bellman-Ford e la sua complessità
- trovare il successore e il predecessore in un BST e la complessità

Gemini ha detto

Teoria (Camurati)

- **Algoritmo di Bellman-Ford:**
 - **Obiettivo:** Trovare i cammini minimi da un vertice sorgente verso tutti gli altri vertici di un grafo orientato pesato. A differenza dell'algoritmo di Dijkstra, supporta pesi negativi.

- **Rilassamento:** L'operazione base. Dato un arco (u, v) con peso w , se la distanza per arrivare a u più w è strettamente minore della distanza attuale per arrivare a v , si aggiorna la distanza di v con questo nuovo valore minore.
 - **Logica:** In un grafo con V vertici, un cammino minimo semplice (senza cicli) contiene al massimo $V-1$ archi. L'algoritmo esegue un ciclo esterno esattamente $V-1$ volte. In ogni iterazione, scorre e "rilassa" tutti gli E archi del grafo.
 - **Rilevamento cicli negativi:** Terminata la fase principale, esegue un'ultima scansione di tutti gli archi. Se un arco può ancora essere rilassato, significa che esiste un ciclo di peso totale negativo raggiungibile dalla sorgente (il problema dei cammini minimi non ammette quindi soluzione).
 - **Complessità Temporale:** $O(V * E)$, poiché esegue V iterazioni su E archi.
 - **Complessità Spaziale:** $O(V)$ per memorizzare gli array delle distanze stimate e dei padri.
- **Successore e Predecessore in un BST:**
 - **Successore:** È il nodo che contiene la chiave immediatamente superiore a quella del nodo corrente X .
 - **Ricerca del Successore:**
 - Se X ha un figlio destro: Il successore è il minimo assoluto del sottoalbero destro (si va a destra una volta e poi tutto a sinistra finché il figlio sinistro è NULL).
 - Se X non ha figlio destro: Si risale verso l'alto tramite il puntatore al padre. Il successore è il primo antenato di cui il nodo (o un suo avo) è figlio sinistro.
 - **Predecessore:** È il nodo che contiene la chiave immediatamente inferiore a quella del nodo corrente X .
 - **Ricerca del Predecessore:**
 - Se X ha un figlio sinistro: Il predecessore è il massimo assoluto del sottoalbero sinistro (si va a sinistra una volta e poi tutto a destra).
 - Se X non ha figlio sinistro: Si risale verso l'alto. Il predecessore è il primo antenato di cui il nodo (o un suo avo) è figlio destro.
 - **Complessità (per entrambi):** Tempo $O(h)$, dove h è l'altezza dell'albero, poiché si percorre al massimo un ramo verticale ($O(\log n)$ se bilanciato, $O(n)$ nel caso pessimo degenerare). Spazio $O(1)$ se implementati iterativamente.

Data : 15 -> 19 (1.25 lab) (18/03/2025)

Programmazione (Cabodi) :

● in un grafo con liste delle adiacenze, isolare un vertice

Programmazione (Cabodi)

- **Logica dell'algoritmo (Isolare un vertice):** Isolare un vertice v significa lasciarlo esistere nel grafo (non ridimensioniamo l'array dei vertici), ma rimuovere qualsiasi arco che parta da v (archi uscenti) e qualsiasi arco che arrivi a v (archi entranti).
- **Gestione archi uscenti:** È immediato. Basta deallocare completamente la lista concatenata presente all'indice v dell'array delle adiacenze e impostare il puntatore a NULL.
- **Gestione archi entranti:** Poiché usiamo liste di adiacenza, non sappiamo a priori chi punta a v . Dobbiamo obbligatoriamente scorrere le liste di tutti gli altri vertici del grafo. Se troviamo un nodo che punta a v , lo scollegiamo (usando un puntatore al precedente) e lo deallociamo.
- **Complessità:** Tempo $O(V + E)$, perché nel caso pessimo dobbiamo visitare ogni vertice e scorrere tutti gli archi del grafo per scovare quelli entranti in v . Spazio $O(1)$, poiché la rimozione e deallocazione avvengono manipolando solo puntatori in loco.

```
#include <stdlib.h>

// Definizione del nodo della lista di adiacenza
typedef struct EdgeNode {
    int v; // Indice del vertice destinazione
    struct EdgeNode* next; // Puntatore al prossimo arco
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V; // Numero totale di vertici
    EdgeNode** adj; // Array di liste di adiacenza
} Graph;

// Funzione per isolare un vertice (rimuove archi uscenti ed entranti)
void isolate_vertex(Graph* g, int target_v) {
```

```

// Controllo di validità sul vertice richiesto
if (g == NULL || target_v < 0 || target_v >= g->V) return;

// 1. RIMOZIONE ARCHI USCENTI: Svuoto completamente la lista di target_v
EdgeNode* curr = g->adj[target_v];
while (curr != NULL) {
    EdgeNode* temp = curr;
    curr = curr->next;
    free(temp); // Libero la memoria di ogni arco uscente
}
// Segno che il vertice non ha più nessuna destinazione
g->adj[target_v] = NULL;

// 2. RIMOZIONE ARCHI ENTRANTI: Scorro tutti gli altri vertici del grafo
for (int i = 0; i < g->V; i++) {
    // Salto il vertice target_v stesso (gestito sopra, inclusi eventuali auto-
    anelli)
    if (i == target_v) continue;

    EdgeNode* edge = g->adj[i];
    EdgeNode* prev = NULL;

    // Scorro la lista delle adiacenze del vertice i
    while (edge != NULL) {
        // Se trovo un arco che punta al vertice da isolare
        if (edge->v == target_v) {

            // Scollego l'arco: caso in cui è la testa della lista
            if (prev == NULL) {
                g->adj[i] = edge->next;
            }
            // Scollego l'arco: caso in cui è in mezzo o in fondo
            else {
                prev->next = edge->next;
            }

            EdgeNode* to_delete = edge;

            // Avanzo il puntatore corrente per continuare l'ispezione
            edge = edge->next;

            // Dealloco il nodo dell'arco rimosso
            free(to_delete);
        }
        else {
            // Non era il vertice target, avanzo normalmente i puntatori
            prev = edge;
            edge = edge->next;
        }
    }
}

```

```
}  
}  
}
```

Teoria (Camurati) :

- cos'è uno heap? quale è la caratteristica principale dello heap?
- cos'è il rango in un bst? come faccio ad ottenerlo senza fare un ordinamento? (Ho risposto con una visita In-Order, ma ha detto che comunque è un ordinamento, quindi voleva sfruttare la cardinalità dell'albero radicato nel nodo corrente)

Teoria (Camurati)

- **Definizione di Heap:** Un Heap è un albero binario "quasi completo", ovvero tutti i livelli dell'albero sono totalmente pieni, tranne al più l'ultimo livello che deve essere riempito rigorosamente da sinistra verso destra.
- **Caratteristica principale (Ordinamento Parziale e Array):** La caratteristica fondamentale è la proprietà dell'ordinamento parziale: in un Max-Heap, il valore di ogni nodo padre è sempre maggiore o uguale a quello dei suoi figli; in un Min-Heap è minore o uguale. Grazie alla forma quasi completa, l'Heap si memorizza implicitamente in un array (senza puntatori), dove la radice è all'indice 0, i figli di i sono a $2i+1$ e $2i+2$, e il padre è a $(i-1)/2$.
- **Complessità Heap:** Spazio $O(n)$. Accesso all'estremo $O(1)$. Inserimento ed estrazione $O(\log n)$.
- **Cos'è il rango in un BST:** Il rango di una chiave all'interno di un Albero Binario di Ricerca indica la sua posizione nella sequenza ordinata di tutti gli elementi presenti (es. è il 1°, il 5°, il k -esimo elemento più piccolo).
- **Come ottenere il rango sfruttando la cardinalità (Senza In-Order):** Invece di scorrere linearmente l'albero (che costerebbe tempo $O(n)$), si modifica la struttura del nodo del BST aggiungendo un campo extra "N" (cardinalità), che memorizza il numero totale di nodi presenti nel sottoalbero radicato in quel nodo.
- **Meccanismo di calcolo:** Si scende nell'albero confrontando la chiave cercata K con il nodo corrente.
 - Se K è uguale alla radice corrente, il suo rango è esattamente la cardinalità del sottoalbero sinistro + 1.
 - Se K è minore, si cerca ricorsivamente il rango nel sottoalbero sinistro.
 - Se K è maggiore, il rango è pari alla cardinalità del sottoalbero sinistro + 1 + il rango di K calcolato ricorsivamente nel sottoalbero destro.

- **Complessità del Rango con cardinalità:** Tempo $O(h)$, dove h è l'altezza dell'albero ($O(\log n)$ nel caso medio bilanciato, $O(n)$ nel caso pessimo). Spazio $O(1)$ se implementato in modo iterativo. Questa tecnica sconfigge il tempo lineare della visita In-Order.
-

Data : 17,75->19 (0,5 lab(18/03/2025))

Programmazione (Cabodi) :

● in un bst date due chiavi k_1 e k_2 vedere se si trovano sullo stesso cammino radice-foglia

Programmazione (Cabodi)

- **Verificare se due chiavi k_1 e k_2 sono sullo stesso cammino (BST):**
 - **Logica dell'algoritmo:** Partendo dalla radice, il cammino verso k_1 e verso k_2 sarà identico finché le due chiavi non obbligano a prendere due rami diversi (una a destra, una a sinistra). Il nodo in cui le direzioni si separano è il *Lowest Common Ancestor* (LCA, l'antenato comune più profondo).
 - **Condizione chiave:** Affinché k_1 e k_2 si trovino sullo stesso *identico cammino* radice-foglia, una delle due chiavi deve obbligatoriamente essere un antenato dell'altra. Questo significa che nel punto esatto di divergenza (LCA), il valore del nodo corrente DEVE coincidere o con k_1 o con k_2 . Se coincide, verifichiamo semplicemente che l'altra chiave esista scendendo ulteriormente. Se non coincide con nessuna delle due, significa che k_1 e k_2 si sono "spartite" su due rami paralleli opposti.
 - **Complessità:** Tempo $O(h)$, dove h è l'altezza dell'albero ($O(\log n)$ nel caso medio, $O(n)$ nel caso pessimo degenerare), poiché si percorre un solo ramo in discesa. Spazio $O(1)$ utilizzando un approccio puramente iterativo.

```
#include <stdbool.h>
#include <stddef.h>

// Struttura del nodo del BST
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione ausiliaria per cercare una chiave scendendo nel BST
```

```

bool search_key(Node* root, int key) {
    while (root != NULL) {
        if (root->key == key) return true; // Chiave trovata
        if (key < root->key) root = root->left;
        else root = root->right;
    }
    return false; // Chiave non presente
}

// Funzione principale per verificare se k1 e k2 sono sullo stesso cammino
bool same_path(Node* root, int k1, int k2) {
    // Discendiamo nell'albero finché non troviamo il punto di divergenza (LCA)
    while (root != NULL) {

        // Se entrambe le chiavi sono minori del nodo corrente,
        // sono entrambe nel sottoalbero sinistro: il cammino non si è ancora
diviso
        if (k1 < root->key && k2 < root->key) {
            root = root->left;
        }
        // Se entrambe sono maggiori, il cammino prosegue unito verso destra
        else if (k1 > root->key && k2 > root->key) {
            root = root->right;
        }
        // Le direzioni divergono: abbiamo trovato l'LCA.
        else {
            // Affinché siano sullo stesso cammino, questo nodo di biforcazione
            // deve essere per forza una delle due chiavi (una è padre dell'altra).
            if (root->key == k1) {
                // k1 è l'antenato. Verifico che k2 esista scendendo da qui in giù.
                return search_key(root, k2);
            }
            else if (root->key == k2) {
                // k2 è l'antenato. Verifico che k1 esista nel suo sottoalbero.
                return search_key(root, k1);
            }
            else {
                // Il nodo di sdoppiamento non è né k1 né k2.
                // Significa che k1 e k2 si trovano su rami fratelli distinti.
                return false;
            }
        }
    }

    // Se arrivo qui, l'albero è vuoto o le chiavi mancano del tutto
    return false;
}

```

Teoria (Camurati) :

● Problema dei cammini massimi

● SCC

● definisci grafo completo

Teoria (Camurati)

- **Problema dei cammini massimi (Longest Path Problem):**

- **Grafo generico (NP-Arduo):** Trovare il cammino semplice (che non ripassa due volte sullo stesso nodo) di costo massimo tra due vertici è un problema intrattabile in tempo polinomiale. Fallisce perché manca la proprietà della *sottostruttura ottima*: il sottocammino di un cammino massimo potrebbe non essere un cammino massimo locale (scegliere l'arco più pesante a livello locale potrebbe costringerti a chiudere un ciclo o escludere nodi fondamentali per raggiungere la destinazione).
- **Risoluzione su DAG (Grafo Diretto Aciclico):** Se il grafo non ha cicli, ogni cammino è implicitamente semplice e il problema riacquista la sottostruttura ottima.
- **Meccanismo su DAG:** Si calcola l'ordinamento topologico dei nodi. Si inizializzano le distanze a $-\infty$ (tranne la sorgente a 0). Si ispezionano i nodi nell'ordine topologico e, per ogni arco uscente (u, v) , si applica un rilassamento "al contrario": se $\text{distanza}[u] + \text{peso}(u, v) > \text{distanza}[v]$, allora aggiorno $\text{distanza}[v]$.
- **Complessità (solo su DAG):** Tempo $O(V + E)$ per l'ordinamento e la successiva ispezione. Spazio $O(V)$.

- **SCC (Componenti Fortemente Connesse):**

- **Definizione:** In un grafo orientato, una SCC è un sottoinsieme *massimalmente connesso* di vertici. Al suo interno, presi due nodi qualsiasi u e v , esiste sempre un cammino diretto da u a v e contemporaneamente un cammino diretto da v a u .
- **Significato di "Massimale":** Significa che la componente è espansa al suo limite fisico: non è possibile aggiungere un altro nodo dal resto del grafo a questo sottoinsieme senza distruggere la garanzia di mutua raggiungibilità.
- **Algoritmi e Meccanismo:** Si usano algoritmi basati sulla DFS (Depth-First Search) come quello di Kosaraju o di Tarjan. Kosaraju fa una prima DFS per annotare i tempi di fine visita, inverte (traspone) tutti gli archi del grafo, e lancia una seconda DFS sui nodi seguendo l'ordine decrescente dei tempi di fine visita per isolare i cicli chiusi.

- **Complessità:** Tempo $O(V + E)$ in quanto si tratta di eseguire due normali visite DFS. Spazio $O(V)$ per mantenere lo stack di ricorsione e gli array dei tempi/colori.
 - **Grafo completo:**
 - **Definizione:** È un grafo (solitamente non orientato) in cui *ogni singola coppia* di vertici distinti è collegata direttamente da un arco univoco. In questo scenario estremo, ogni vertice del grafo ha grado massimo, pari a $V-1$.
 - **Numero di archi:** Il numero totale di archi E segue la formula combinatoria delle coppie: $E = V * (V - 1) / 2$.
 - **Complessità Spaziale:** Essendo un grafo di massima densità, memorizzarlo costa asintoticamente $O(V^2)$, rendendo la matrice delle adiacenze e la lista delle adiacenze equivalenti dal punto di vista dello spreco di memoria (poiché gli archi E coincidono con V^2).
-

Data : 15.5 -> ??? (0.75 lab) 18/03/2025

Programmazione (Cabodi) :

● in un albero contare quanti nodi hanno almeno tre nipoti (figli di figli)

Programmazione (Cabodi)

- **Conteggio nodi con almeno tre nipoti in un Albero Binario:**
 - **Logica dell'algoritmo:** Si utilizza una visita ricorsiva dell'albero (es. in pre-ordine). Per ogni nodo visitato, si contano i figli dei suoi figli (i nipoti diretti). Poiché in un albero binario ogni nodo ha al massimo 2 figli, i nipoti massimi possibili sono 4. Se il conteggio è maggiore o uguale a 3, il nodo corrente soddisfa la condizione e vale 1 nel conteggio totale, a cui si sommano i risultati delle chiamate ricorsive sui sottoalberi sinistro e destro.
 - **Prevenzione Segmentation Fault:** È fondamentale verificare rigorosamente che un figlio esista (non sia NULL) prima di tentare di accedere ai suoi rispettivi figli.
 - **Complessità Temporale:** $O(n)$, dove n è il numero di nodi dell'albero. Ogni nodo viene visitato e ispezionato un numero costante di volte.

- **Complessità Spaziale:** $O(h)$, dove h è l'altezza dell'albero (da $O(\log n)$ a $O(n)$), dovuta allo spazio occupato dallo stack delle chiamate ricorsive.

```
#include <stddef.h>

// Definizione del nodo di un classico albero binario
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione ricorsiva per contare i nodi con >= 3 nipoti
int count_three_grandchildren(Node* root) {
    // Caso base: se l'albero (o il sottoalbero) è vuoto, il conteggio è 0
    if (root == NULL) {
        return 0;
    }

    int grandchildren_count = 0;

    // Ispezione il ramo sinistro per cercare nipoti
    if (root->left != NULL) {
        // Se esiste il nipote a sinistra-sinistra, incremento
        if (root->left->left != NULL) grandchildren_count++;
        // Se esiste il nipote a sinistra-destra, incremento
        if (root->left->right != NULL) grandchildren_count++;
    }

    // Ispezione il ramo destro per cercare nipoti
    if (root->right != NULL) {
        // Se esiste il nipote a destra-sinistra, incremento
        if (root->right->left != NULL) grandchildren_count++;
        // Se esiste il nipote a destra-destra, incremento
        if (root->right->right != NULL) grandchildren_count++;
    }

    // Valuto se il nodo corrente rispetta la condizione richiesta (>= 3 nipoti)
    int is_valid_node = (grandchildren_count >= 3) ? 1 : 0;

    // Ritorno il risultato del nodo corrente sommato a quello dei sottoalberi
    return is_valid_node +
        count_three_grandchildren(root->left) +
        count_three_grandchildren(root->right);
}
```

Teoria (Camurati) :

- teorema e corollario archi sicuri

- come calcolare i cammini minimi utilizzando un modello del calcolo combinatorio

- come calcolare gli mst utilizzando un modello del calcolo combinatorio

Teoria (Camurati)

Teorema degli Archi Sicuri: Professore, il Teorema degli Archi Sicuri è il fondamento matematico che ci permette di usare algoritmi Greedy, come Prim e Kruskal, per trovare il Minimum Spanning Tree senza commettere errori.

Un arco si definisce matematicamente "sicuro" per l'insieme A se, aggiungendolo ad A, l'insieme risultante continua a essere un sottoinsieme valido di un Minimum Spanning Tree (MST). In parole povere: fare quella scelta non ti preclude la possibilità di arrivare alla soluzione ottima finale.

Il concetto di taglio: Per capirlo, immaginiamo di avere un insieme 'A' di archi che abbiamo già scelto e che sappiamo essere corretti. Ora tracciamo un "taglio", ovvero una linea immaginaria che divide tutti i vertici del grafo in due sottoinsiemi. L'unica regola imposta dal teorema è che questo taglio deve "rispettare A", cioè non deve mai spezzare in due nessun arco che abbiamo già inserito nel nostro insieme.

L'arco leggero e la sicurezza: A questo punto, osserviamo tutti gli archi che attraversano il taglio facendo da ponte tra i due sottoinsiemi. Quello che costa meno di tutti è chiamato "arco leggero". L'enunciato ci garantisce che questo arco è matematicamente "sicuro": possiamo aggiungerlo ad 'A' a occhi chiusi, con la certezza assoluta che farà parte dell'albero ottimo finale.

Corollario degli Archi Sicuri: Il corollario è l'applicazione pratica di questo teorema. Durante la costruzione dell'MST, i nostri archi scelti formano una specie di "foresta", cioè tanti piccoli alberelli staccati tra loro.

La regola pratica: Il corollario ci dice che se prendiamo uno qualsiasi di questi alberelli e cerchiamo l'arco più economico in assoluto che lo collega al resto del grafo, quell'arco è sicuramente un arco sicuro.

Perché giustifica Prim e Kruskal: È esattamente la logica usata dagli algoritmi Greedy. Prim fa crescere un solo albero isolandolo col taglio e pescando l'arco leggero verso l'esterno. Kruskal isola due alberelli e li unisce con l'arco globale più leggero. Grazie a questo teorema, evitano di calcolare tutte le combinazioni e trovano la soluzione perfetta in Tempo $O(E \log V)$ e Spazio $O(V)$.

Modello Combinatorio per i Cammini Minimi: Se volessimo calcolare i cammini minimi usando il calcolo combinatorio, staremmo di fatto usando la forza bruta pura, ignorando l'efficienza di algoritmi come Dijkstra.

Come funzionerebbe: L'idea è generare sistematicamente tutti i possibili cammini semplici tra il nodo di partenza e quello di destinazione. Siccome un cammino semplice non può avere cicli, toccherà al massimo V vertici. Quindi il modello calcola tutte le permutazioni possibili

dei vertici intermedi, somma i pesi per ogni percorso generato, e alla fine elegge banalmente quello che costa meno.

Perché non si usa: Questo approccio è impraticabile. Nel caso pessimo di un grafo completo, il numero di permutazioni esplode, portando la complessità Temporale a $O(V!)$. È un tempo fattoriale che renderebbe l'algoritmo infinito per grafi anche piccoli. La complessità Spaziale invece resta $O(V)$ per tracciare i nodi del cammino corrente.

Modello Combinatorio per l'MST: Anche qui, usare il modello combinatorio significa rinunciare alle scelte intelligenti e provare a generare tutti i possibili alberi ricoprenti che si possono estrarre dal grafo di partenza.

Enumerazione dei Cammini Semplici. È il modello matematico che genera in modo esaustivo tutte le permutazioni valide dei vertici per esplorare ogni singolo percorso privo di cicli. Questo approccio combinatorio esplode sempre a Tempo $O(V!)$ e Spazio $O(V)$.

Enumerazione degli Alberi Ricoprenti

Il meccanismo esaustivo: L'algoritmo prenderebbe tutte le possibili combinazioni di V vertici e $V-1$ archi. Per ognuna, deve prima verificare che sia tutto connesso e senza cicli. Poi somma i pesi degli archi di ogni albero valido trovato e, alla fine, sceglie quello con il peso totale minimo.

L'esplosione di Cayley: La complessità Temporale è un disastro. Secondo la formula di Cayley, in un grafo completo esistono esattamente $V^{(V-2)}$ alberi ricoprenti distinti. Valutarli tutti fa schizzare il Tempo a $O(V^V)$. È un'esplosione super-esponenziale (con Spazio $O(V)$) che dimostra alla perfezione perché abbiamo un disperato bisogno del Teorema degli Archi Sicuri per risolvere il problema in tempi reali.

Data : 15 -> 19 (no lab) 18/03/2025

Programmazione (Cabodi) :

- In un grafo orientato con lista di adiacenza trovare il vertice con la maggiore differenza tra archi entranti e uscenti (o, detto in altro modo, differenza tra `inDegree` e `outDegree`)
- (A voce) Come potresti modificare il codice se volessi fare la stessa cosa ma coi vertici distanti 2? (ho detto bfs ma mi ha detto che è troppo onerosa.. voleva un algoritmo iterativo con doppio ciclo in cui per ogni vertice vai a cercare quelli adiacenti)
- (A voce) Nel caso io non voglia ricontare i vertici già scoperti? (vettore di scoperta)

Programmazione (Cabodi)

- Vertice con massima differenza (`inDegree` - `outDegree`):

- **Logica dell'algoritmo:** In una lista di adiacenza, contare gli archi uscenti (outDegree) è banale (basta scorrere la lista del nodo), ma contare quelli entranti (inDegree) richiede di scansionare tutto il grafo per vedere chi punta al nodo.
- **Ottimizzazione:** Per evitare calcoli ridondanti, allochiamo un solo array diff grande V, inizializzato a 0. Scorriamo tutto il grafo una singola volta: per ogni arco diretto da u a v, il vertice u "perde" un punto (arco uscente, diff[u]--), mentre il vertice v "guadagna" un punto (arco entrante, diff[v]++). Finito il giro, cerchiamo linearmente il valore massimo nell'array.
- **Complessità:** Tempo $O(V + E)$ perché visitiamo ogni vertice e ogni arco esattamente una volta sola. Spazio $O(V)$ per allocare l'array delle differenze.

```
#include <stdlib.h>
#include <limits.h>

// Nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del Grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione per trovare il vertice con max (inDegree - outDegree)
int max_in_out_diff(Graph* g) {
    if (g == NULL || g->V == 0) return -1;

    // Alloco l'array delle differenze usando calloc per inizializzarlo a 0
    int* diff = (int*)calloc(g->V, sizeof(int));
    if (diff == NULL) return -1;

    // Scorro tutti i vertici del grafo (potenziali sorgenti degli archi)
    for (int u = 0; u < g->V; u++) {
        EdgeNode* edge = g->adj[u];

        // Scorro la lista di adiacenza del vertice u corrente
        while (edge != NULL) {
            int v = edge->v; // Nodo destinazione dell'arco

            // u ha un arco uscente, quindi la sua differenza scende
            diff[u]--;
            // v ha un arco entrante, quindi la sua differenza sale
            diff[v]++;
            edge = edge->next;
        }
    }

    // Troviamo il massimo valore in diff
    int max = INT_MIN;
    for (int i = 0; i < g->V; i++) {
        if (diff[i] > max) max = diff[i];
    }

    return max;
}
```



```

        edge = edge->next; // Avanzo al prossimo arco uscente di u
    }
}

// Variabili per tracciare il massimo trovato e il vertice associato
int max_diff = INT_MIN;
int best_vertex = -1;

// Scansione lineare finale per trovare il vertice vincente
for (int i = 0; i < g->V; i++) {
    if (diff[i] > max_diff) {
        max_diff = diff[i];
        best_vertex = i;
    }
}

free(diff); // Libero la memoria di supporto
return best_vertex; // Ritorno l'indice del vertice
}

```

? Risposta a voce: Modifica per calcolare l'in/out degree a distanza 2 (Senza BFS):

- **Logica del doppio ciclo:** La BFS è troppo onerosa (richiede di allocare code e gestire distanze). Si può risolvere in modo puramente iterativo. Per ogni vertice di partenza i , usiamo un puntatore per scorrere la sua lista di adiacenza (i vicini a distanza 1, chiamiamoli u).
- **Il passo successivo:** All'interno di questo ciclo, prendiamo l'indice u e apriamo un *secondo* ciclo while per scorrere la lista di adiacenza di u . I nodi trovati qui (chiamiamoli v) sono esattamente quelli a distanza 2 dal vertice di partenza i .
- **Calcolo:** A questo punto, applichiamo la stessa logica di prima: aggiorniamo $\text{diff}[i]--$ (perché da i esce un percorso di lunghezza 2) e $\text{diff}[v]++$ (perché a v entra un percorso di lunghezza 2).

? Risposta a voce: Uso del vettore di scoperta per non ricontare vertici a distanza 2:

- **Il problema:** Più nodi a distanza 1 potrebbero puntare allo stesso nodo a distanza 2 (es. $i \rightarrow u_1 \rightarrow v$ e $i \rightarrow u_2 \rightarrow v$). Senza controlli, conteremmo v due volte come destinazione di i .
- **Soluzione classica:** Si alloca un array booleano (o si resetta) per tenere traccia dei nodi già visitati dal vertice i corrente.
- **L'ottimizzazione per stupire Cabodi (Array di Marcatura a Interi):** Invece di resettare un intero array booleano a ogni giro (che costerebbe Tempo $O(V^2)$ totale), allochiamo un singolo array di interi visited inizializzato a -1.

- **Il meccanismo furbo:** Mentre calcoliamo i percorsi a distanza 2 per il vertice sorgente i , controlliamo se $visited[v] \neq i$. Se è diverso, significa che non abbiamo ancora contato v per l'attuale sorgente i . Lo contiamo e aggiorniamo $visited[v] = i$. Al giro successivo, con una nuova sorgente $i+1$, le vecchie marcature verranno automaticamente ignorate perché il valore atteso è cambiato, azzerando i tempi di reset. Complessità Spaziale $O(V)$, Tempo $O(V * max_grado^2)$.

Teoria (Camurati) :

Data : 15,75 -> 20 (1,75 lab) 14/03/2025

Programmazione (Cabodi) :

- Funzione che dati due vertici conti quanti sono i vertici raggiungibili da almeno uno di questi due dato un grafo non orientato
- (a voce) e se fossero stati adiacenti? (la risposta era che bastava usare una lista delle adiacenze o una matrice della adiacenze)

Programmazione (Cabodi)

- **Conteggio vertici raggiungibili da almeno uno tra $v1$ e $v2$:**
 - **Logica dell'algoritmo:** In un grafo non orientato, i vertici raggiungibili da un nodo costituiscono la sua intera componente connessa. L'obiettivo è contare l'unione dei vertici delle componenti connesse di $v1$ e $v2$.
 - **Meccanismo:** Alloco un array globale di visita. Lancio una visita (es. DFS) partendo da $v1$ e conto i nodi scoperti. Terminata la visita, controllo se $v2$ è stato visitato: se sì, significa che $v1$ e $v2$ appartengono alla stessa componente connessa e ho già contato tutto. Se $v2$ non è stato visitato, appartiene a una componente disgiunta: lancio una seconda DFS da $v2$ riutilizzando lo stesso array di visita e continuo a sommare al contatore.
 - **Complessità Temporale:** $O(V + E)$ nel caso in cui le due visite coprano l'intero grafo, visitando archi e nodi una sola volta.
 - **Complessità Spaziale:** $O(V)$ per l'array booleano dei visitati e lo stack di ricorsione della DFS.

```
#include <stdlib.h>
```

```

#include <stdbool.h>

// Nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione ricorsiva di supporto per esplorare la componente connessa
void dfs_count(Graph* g, int u, bool* visited, int* count) {
    // Segno il nodo corrente come visitato per non processarlo più
    visited[u] = true;
    // Incremento il contatore dei vertici totali raggiunti
    (*count)++;

    // Esploro tutti i vicini del nodo corrente
    EdgeNode* edge = g->adj[u];
    while (edge != NULL) {
        int neighbor = edge->v;

        // Se il vicino non è ancora stato scoperto, continuo la visita in
        // profondità
        if (!visited[neighbor]) {
            dfs_count(g, neighbor, visited, count);
        }
        // Passo al prossimo arco uscente
        edge = edge->next;
    }
}

// Funzione principale
int count_reachable_from_two(Graph* g, int v1, int v2) {
    if (g == NULL || v1 < 0 || v2 < 0 || v1 >= g->V || v2 >= g->V) return 0;

    // Array per tenere traccia dei nodi esplorati, inizializzato a false
    bool* visited = (bool*)calloc(g->V, sizeof(bool));
    if (visited == NULL) return -1;

    int total_reachable = 0;

    // 1. Lancio la prima esplorazione partendo da v1
    dfs_count(g, v1, visited, &total_reachable);

```

```

    // 2. Se v2 non è stato raggiunto dalla prima visita, è in una componente
    isolata da v1
    if (!visited[v2]) {
        // Lancio l'esplorazione da v2, sommando al conteggio precedente
        dfs_count(g, v2, visited, &total_reachable);
    }

    // Libero la memoria di supporto
    free(visited);

    return total_reachable;
}

```

Teoria (Camurati) :

● Componenti connesse e come si trovano

● Punto di articolazione

Teoria (Camurati)

- **Componenti Connesse:**

- **Definizione:** In un grafo non orientato, una componente connessa è un sottografo "massimale" in cui esiste sempre un cammino tra ogni coppia di vertici che ne fanno parte. Massimale significa che non è possibile aggiungere un ulteriore vertice a questo gruppo senza distruggere la proprietà di connessione.
- **Come si trovano:** Si alloca un array di visita azzerato. Si itera linearmente su tutti i vertici del grafo (da 0 a V-1). Quando si incontra un vertice non ancora visitato, si innesca una visita (DFS o BFS) a partire da esso. Tutti i nodi che questa singola visita riesce a marcare costituiscono esattamente una singola componente connessa. Il ciclo esterno prosegue finché tutti i nodi non sono stati processati.
- **Complessità:** Tempo $O(V + E)$ perché, pur essendoci un ciclo dentro un ciclo, ogni vertice viene scoperto una sola volta e ogni arco percorso al massimo due volte. Spazio $O(V)$.

- **Punto di Articolazione (Cut Vertex):**

- **Definizione:** In un grafo connesso, un punto di articolazione è un vertice critico la cui rimozione (insieme a tutti gli archi a esso collegati) spezza il grafo originale in due o più componenti connesse distinte, aumentando di fatto il numero totale di componenti.

- **Ricerca ottimizzata (Algoritmo di Tarjan/Hopcroft):** Si basa su una singola visita DFS modificata, evitando di simulare rimozioni multiple. Durante la discesa si assegnano due valori a ogni nodo: il "Tempo di Scoperta" (quando il nodo viene visitato la prima volta) e il "Low" (il tempo di scoperta più basso raggiungibile dal nodo o dai suoi discendenti sfruttando al massimo un *Back-edge*).
- **Back-edge (Arco all'indietro):** È un arco che congiunge un nodo a un suo antenato nell'albero di visita DFS, creando di fatto un percorso ciclico alternativo per "tornare su".
- **Condizione di Articolazione:** Un nodo 'u' (diverso dalla radice) è un punto di articolazione se ha un figlio 'v' nell'albero DFS tale per cui $Low(v) \geq Tempo_Scoperta(u)$. Questo significa che il figlio 'v' non possiede nessun Back-edge per aggirare 'u' e raggiungere gli antenati; se 'u' crolla, 'v' e tutta la sua discendenza rimangono isolati. (Eccezione: la radice della DFS è un punto di articolazione solo se ha due o più figli indipendenti nell'albero).
- **Complessità:** Tempo $O(V + E)$ in quanto si risolve tutto in un'unica scansione DFS profonda. Spazio $O(V)$ per i vettori di supporto dei tempi e lo stack ricorsivo.

Data : 15,5 -> 20 (1,25 lab) 14/03/2025

Programmazione (Cabodi) :

● Data una lista di stringhe, estrarre da questa lista le stringhe con lunghezza maggiore o uguale ad l e posizzionarle in un vettore di stringhe

Programmazione (Cabodi)

- **Logica dell'algoritmo (Estrazione stringhe lunghe):**
 - Non sapendo a priori quante stringhe rispetteranno la condizione (lunghezza $\geq l$), procediamo in due passate (due scansioni della lista).
 - **Passata 1 (Conteggio):** Scorriamo la lista per contare quanti nodi contengono una stringa di lunghezza adeguata. Questo ci serve per allocare esattamente l'array di stringhe (che è un array di puntatori a char).
 - **Passata 2 (Copia):** Scorriamo nuovamente la lista. Quando troviamo una stringa valida, allochiamo la memoria specifica per quella parola (+1 per il terminatore $\backslash 0$) e la copiamo nell'array, avanzando un indice dedicato all'array.

- **Complessità Temporale:** $O(N + S_{\text{tot}})$, dove N è il numero di nodi della lista e S_{tot} è la somma dei caratteri delle stringhe da copiare, poiché `strlen` e `strcpy` scorrono i caratteri.
- **Complessità Spaziale:** $O(K * L_{\text{med}})$ allocata sull'heap per il vettore risultato, dove K è il numero di stringhe estratte e L_{med} è la loro lunghezza media. Spazio ausiliario della funzione $O(1)$.

```
#include <stdlib.h>
#include <string.h>

// Definizione del nodo della lista concatenata di stringhe
typedef struct Node {
    char* str;           // Puntatore alla stringa allocata dinamicamente
    struct Node* next;   // Puntatore al prossimo elemento
} Node;

// Funzione che restituisce un array dinamico di stringhe e aggiorna out_size col
numero di elementi
char** extract_long_strings(Node* head, int l, int* out_size) {
    if (head == NULL) {
        *out_size = 0;
        return NULL; // Lista vuota, niente da estrarre
    }

    int count = 0;
    Node* current = head;

    // PRIMA PASSATA: Conto quante stringhe rispettano il criterio per dimensionare
    l'array
    while (current != NULL) {
        // Uso strlen per calcolare la lunghezza ignorando il terminatore '\0'
        if (strlen(current->str) >= l) {
            count++;
        }
        current = current->next;
    }

    // Comunico al chiamante quante stringhe ho trovato
    *out_size = count;

    // Se nessuna stringa rispetta il criterio, evito allocazioni inutili
    if (count == 0) return NULL;

    // Alloco dinamicamente l'array di puntatori a char (char**)
    char** result_array = (char**)malloc(count * sizeof(char*));
    if (result_array == NULL) return NULL; // Gestione fallimento malloc

    int index = 0;
```

```

    current = head; // Riporto il puntatore all'inizio della lista per la seconda
passata

    // SECONDA PASSATA: Popolo l'array con copie delle stringhe valide
    while (current != NULL) {
        int current_len = strlen(current->str);

        // Se la stringa è valida (stessa condizione di prima)
        if (current_len >= 1) {
            // Alloco lo spazio esatto per la stringa corrente (+1 per il carattere
'\0')
            result_array[index] = (char*)malloc((current_len + 1) * sizeof(char));

            // Se la malloc ha successo, copio fisicamente i caratteri
            if (result_array[index] != NULL) {
                strcpy(result_array[index], current->str);
            }

            // Avanzo la posizione disponibile nell'array di destinazione
            index++;
        }

        // Avanzo nella lista concatenata
        current = current->next;
    }

    // Ritorno l'array popolato. Il chiamante dovrà fare la free() di ogni stringa
e dell'array stesso.
    return result_array;
}

```

Teoria (Camurati) :

- Cosa sono e come funzionano i codici liberi da prefisso?
- Quicksort: caso peggiore. Cosa rende il caso peggiore, appunto peggiore? Come la scelta del pivot influenza il caso? Come posso scegliere il pivot per migliorarlo? quali sono le probabilità che avvenga il caso peggiore e perchè?

Teoria (Camurati)

- **Codici liberi da prefisso (Prefix-free codes):**
 - **Definizione:** Sono codici di codifica in cui nessuna parola del codice (sequenza di bit) è il prefisso iniziale di un'altra parola. Ad esempio, se la lettera 'A' è codificata come "01", nessun'altra lettera potrà avere un codice che inizia per "01" (come "010" o "011").

- **Meccanismo e Vantaggio:** Questa proprietà garantisce la **decodifica istantanea e univoca**. Ricevendo un flusso continuo di bit (es. "01100101"), il decodificatore legge un bit alla volta e, appena riconosce una sequenza valida, sa con certezza assoluta che la lettera è finita, senza bisogno di usare caratteri separatori (spazi o virgole) tra un simbolo e l'altro.
- **Implementazione pratica:** Si rappresentano facilmente tramite un albero binario in cui i simboli da codificare sono posizionati esclusivamente nelle foglie. Il percorso dalla radice alla foglia determina il codice (es. ramo sinistro = 0, ramo destro = 1). L'algoritmo di Huffman è il metodo standard per generare il codice libero da prefisso ottimo per un dato testo.
- **Quicksort e il Caso Peggior:**
 - **Cosa rende il caso peggiore:** Il Quicksort divide l'array in due partizioni basandosi su un elemento "pivot". Il caso peggiore si verifica quando si ha uno **sbilanciamento estremo e continuo** delle partizioni: a ogni chiamata ricorsiva, tutti gli elementi finiscono da una parte del pivot (sottoproblema di dimensione $n-1$) e nessuno dall'altra (sottoproblema di dimensione 0).
 - **Equazione di ricorrenza (Caso peggiore):** La formula diventa $T(n) = T(n-1) + O(n)$. Risolvendola, la complessità Temporale crolla inesorabilmente a $O(n^2)$. La complessità Spaziale, dovuta allo stack delle chiamate ricorsive che non si dimezzano ma scendono linearmente, degenera a $O(n)$.
 - **L'influenza del pivot:** Il disastro avviene se il pivot scelto si rivela essere costantemente il massimo o il minimo locale dell'array. Ad esempio, se l'algoritmo sceglie sempre ciecamente il primo (o l'ultimo) elemento e l'array in input è già perfettamente ordinato (o ordinato al contrario).
 - **Come migliorare la scelta del pivot:** * **Mediana di tre:** Scegliere come pivot il valore mediano tra il primo, il centrale e l'ultimo elemento della partizione corrente.
 - **Randomizzazione:** Scegliere l'indice del pivot in modo casuale ad ogni iterazione (Randomized Quicksort).
 - **Probabilità del caso peggiore e perché:** Nel Quicksort classico (pivot fisso al primo/ultimo elemento), se l'input è parzialmente o totalmente ordinato (evento frequentissimo nella realtà lavorativa), la probabilità di incappare nel caso peggiore è altissima, quasi certa. Usando il Quicksort randomizzato, il caso peggiore si verifica solo se il generatore casuale pesca ininterrottamente il minimo o il massimo per n volte di fila. La probabilità che questo accada è bassissima (asintoticamente $2/n!$), rendendo il caso peggiore $O(n^2)$ un evento teorico e garantendo un tempo $O(n \log n)$ praticamente certo nella realtà.

Data : 16,25 -> 20 (2 lab) 14/03/2025

Programmazione (Cabodi) :

● Data una lista concatenata semplice, non completamente ordinata, ritornare la lunghezza del più lungo sottovettore ordinato in ordine crescente o decrescente. Ho fatto una funzione che mi trovava la lunghezza in un sottovettore strettamente crescente/decrescente, mi ha chiesto come avrei potuto modificare la funzione in modo da trovare un ordinamento non stretto ($\geq$ o $\leq$).

Programmazione (Cabodi)

- **Logica dell'algoritmo (Sottovettore strettamente ordinato):**
 - Si scorre la lista una sola volta. Si usano due contatori temporanei: uno per la lunghezza della sequenza attualmente crescente e uno per quella decrescente.
 - Se il nodo successivo è strettamente maggiore, si incrementa il contatore crescente e si resetta brutalmente a 1 quello decrescente. Se è strettamente minore, si fa l'opposto. Se sono uguali, si resettano entrambi a 1 (perché l'ordinamento richiesto è stretto).
 - A ogni passo si aggiorna la lunghezza massima globale confrontandola con i contatori correnti.
- **Risposta a voce (Modifica per ordinamento NON stretto: $\geq$ o $\leq$):**
 - Se volessimo accettare valori uguali, la modifica è elegantissima e si fa nell'istruzione condizionale dell'uguaglianza. Se il nodo corrente è uguale al precedente, questo rispetta *contemporaneamente* sia la definizione di "non decrescente" ($\geq$) sia quella di "non crescente" ($\leq$).
 - Di conseguenza, al posto di resettare i contatori, si **incrementano entrambi** i contatori correnti di 1 e la visita prosegue.
- **Complessità Temporale:** $O(n)$, dove n è il numero di nodi, poiché la lista viene attraversata una volta sola linearmente.
- **Complessità Spaziale:** $O(1)$, poiché si allocano solo variabili intere di conteggio, senza strutture dati ausiliarie o chiamate ricorsive.

```
#include <stdlib.h>

// Definizione del nodo della lista concatenata semplice
typedef struct Node {
```

```

    int val;
    struct Node* next;
} Node;

// Funzione per trovare la sequenza strettamente crescente/decrescente più lunga
int longest_strict_sorted_sublist(Node* head) {
    // Se la lista è vuota, la lunghezza massima è 0
    if (head == NULL) return 0;

    int max_len = 1;        // Lunghezza massima globale trovata finora
    int curr_inc = 1;        // Contatore per la sequenza crescente corrente
    int curr_dec = 1;        // Contatore per la sequenza decrescente corrente

    Node* curr = head;

    // Scorro finché ho un nodo corrente e un nodo successivo da confrontare
    while (curr->next != NULL) {

        // CASO 1: Strettamente crescente
        if (curr->next->val > curr->val) {
            curr_inc++;        // La sequenza crescente continua
            curr_dec = 1;      // La sequenza decrescente viene rotta e si resetta
        }
        // CASO 2: Strettamente decrescente
        else if (curr->next->val < curr->val) {
            curr_dec++;        // La sequenza decrescente continua
            curr_inc = 1;      // La sequenza crescente viene rotta e si resetta
        }
        // CASO 3: Elementi uguali (rompe entrambi gli ordinamenti STRETTI)
        else {
            curr_inc = 1;
            curr_dec = 1;

            /* RISPOSTA ORALE (Ordinamento NON STRETTO):
               Se la traccia avesse chiesto >= o <=, due elementi uguali
               non avrebbero rotto la sequenza. Qui dentro avremmo scritto:
               curr_inc++;
               curr_dec++;
            */
        }

        // Aggiorno il massimo globale se uno dei due contatori lo ha superato
        if (curr_inc > max_len) max_len = curr_inc;
        if (curr_dec > max_len) max_len = curr_dec;

        // Avanzo al nodo successivo
        curr = curr->next;
    }

    return max_len;
}

```

}

Teoria (Camurati) :

- Cos'è un DAG? E' orientato? Perché? Come fare a trovare l'ordinamento topologico di un DAG? L'ordinamento topologico di un DAG è unico?
- Quale informazione bisogna aggiungere agli IBST? A cosa serve il numero di nodi nel sottoalbero di ogni nodo? Aggiungere il numero di nodi nei sottoalberi in un IBST ha senso? (Spoiler: no)

Teoria (Camurati)

- **Cos'è un DAG:** È l'acronimo di Directed Acyclic Graph (Grafo Diretto Aciclico). È un grafo che possiede archi con una direzione specifica ma che è strutturalmente privo di qualsiasi ciclo chiuso.
- **È orientato? Perché?** Sì, è intrinsecamente orientato. La direzione degli archi è fondamentale perché serve a modellare relazioni di dipendenza asimmetrica o precedenza (es. il task A deve essere completato rigorosamente prima del task B). Se non avesse le frecce (grafo non orientato), ogni arco A-B implicherebbe una relazione bidirezionale (da A a B e da B ad A), generando di fatto un ciclo continuo e distruggendo la gerarchia logica alla base del DAG.
- **Come trovare l'Ordinamento Topologico:**
 - **Meccanismo (basato su DFS):** Si lancia una visita DFS (Depth-First Search) sul grafo. Ogni volta che la visita ricorsiva di un nodo termina (cioè quando tutti i suoi vicini sono stati esplorati), si registra il suo "tempo di fine visita" e si inserisce il nodo in testa a una lista concatenata. L'ordinamento topologico è semplicemente l'elenco dei nodi ordinato in senso decrescente rispetto ai loro tempi di fine visita.
 - **Complessità:** Tempo $O(V + E)$ per esplorare l'intero grafo con la DFS. Spazio $O(V)$ per tracciare i nodi visitati e memorizzare la lista finale.
- **L'ordinamento topologico è unico?** No, in generale non è unico. Un DAG può ammettere molteplici ordinamenti topologici validi se ci sono nodi paralleli senza dipendenze reciproche. L'ordinamento è strettamente unico se e solo se all'interno del DAG esiste un **Cammino Hamiltoniano** (un percorso diretto che attraversa ogni singolo nodo del grafo esattamente una volta, imponendo una catena di dipendenze totale).
- **Alberi degli Intervalli (IBST - Interval Binary Search Tree):**
 - **Informazione da aggiungere:** In un IBST, la chiave di ordinamento è l'estremo inferiore dell'intervallo. L'informazione cruciale che DEVE essere aggiunta al

nodo è il valore del **massimo estremo superiore (max_upper_bound)** tra tutti gli intervalli presenti in quel nodo e nel suo intero sottoalbero.

- **A cosa serve il numero di nodi (Cardinalità):** L'informazione sulla cardinalità (quanti nodi ci sono nel sottoalbero) serve in un'altra struttura, chiamata Indexed BST (o Order-Statistic Tree), per calcolare in Tempo $O(\log n)$ il rango di un elemento o trovare il k-esimo elemento più piccolo.
 - **Ha senso aggiungere la cardinalità in un IBST? (Spoiler: No):** Assolutamente no. Lo scopo esclusivo di un IBST è trovare le intersezioni (overlap) tra intervalli. La ricerca di un'intersezione si basa puramente sui valori numerici dei limiti degli intervalli (scendo a sinistra se il max_upper_bound sinistro supera il limite inferiore che cerco). Sapere "quanti" intervalli ci sono in un ramo non fornisce alcuna informazione geometrica utile per decidere se c'è un'intersezione, sprecando solo memoria.
-

Data : 18-> 22 (1,25 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi Estrazione del massimo e del secondo massimo nell'heap

❓ **Logica dell'algoritmo in funzione singola (Inlining):** Per condensare tutto in un unico blocco di codice senza funzioni di supporto, eseguiamo un ciclo for che gira esattamente due volte. In ogni iterazione salviamo il valore in radice, lo sovrascriviamo con l'ultimo elemento dell'array, riduciamo la dimensione logica dell'heap e facciamo sprofondare il nuovo nodo radice (Sift-Down) tramite un ciclo while interno finché non ritrova la sua posizione corretta.

❓ **Considerazione didattica per l'orale:** Anche se tecnicamente fattibile e molto compatto, all'orale fai presente al Prof. Cabodi che questo approccio sacrifica la modularità del codice. Aver separato la funzione max_heapify è considerato lo standard di buona programmazione perché permette di riutilizzarla per altre operazioni (come la costruzione iniziale dell'heap o l'heapsort).

❓ **Complessità Temporale:** $O(\log n)$, poiché il ciclo esterno viene eseguito un numero costante di volte (esattamente 2), e il ciclo interno di ripristino scende al massimo per l'altezza dell'albero, che è proporzionale a $\log n$.

❓ **Complessità Spaziale:** $O(1)$, in quanto il processo iterativo alloca in memoria esclusivamente poche variabili intere di appoggio, indipendentemente dalla grandezza dell'array.

```

#include <stdlib.h>

// Funzione unica per estrarre il primo e il secondo massimo da un Max-Heap
int extract_two_maxes_inline(int* heap, int* size, int* out_first, int* out_second)
{
    // Verifico che l'heap esista e abbia almeno due elementi da poter estrarre
    if (heap == NULL || *size < 2) return 0;

    // Array di puntatori per assegnare comodamente i risultati nel ciclo senza
    // duplicare codice
    int* outputs[2] = {out_first, out_second};

    // Eseguo l'estrazione logica esattamente due volte (una per il 1° max, una per
    // il 2°)
    for (int i = 0; i < 2; i++) {

        // 1. Salvo il massimo corrente (che si trova sempre e obbligatoriamente
        // nella radice)
        *outputs[i] = heap[0];

        // 2. Prendo l'ultimo elemento in fondo all'heap e lo metto in radice per
        //appare il buco
        heap[0] = heap[*size - 1];

        // 3. Rimpicciolisco la dimensione ufficiale dell'heap, eliminando di fatto
        // l'ultimo nodo
        (*size)--;

        // 4. Avvio il processo di Sift-Down (Heapify) per far sprofondare la nuova
        // radice
        int index = 0;

        while (1) {
            // Calcolo gli indici teorici dei due figli del nodo corrente
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index; // Presumo inizialmente che il nodo corrente sia
            // il più grande

            // Se il figlio sinistro è dentro l'array ed è maggiore del nodo
            // corrente, lui è il nuovo candidato
            if (left < *size && heap[left] > heap[largest]) {
                largest = left;
            }

            // Se il figlio destro esiste ed è ancora più grande del candidato
            // attuale, vince lui
            if (right < *size && heap[right] > heap[largest]) {

```

```

        largest = right;
    }

    // Se il nodo corrente è maggiore o uguale a entrambi i figli, la
    proprietà è ripristinata
    if (largest == index) break;

    // Altrimenti, scambio fisicamente il valore del nodo corrente con
    quello del figlio maggiore
    int temp = heap[index];
    heap[index] = heap[largest];
    heap[largest] = temp;

    // Aggiorno l'indice per far continuare il ciclo while partendo dalla
    nuova posizione abbassata
    index = largest;
}
}

return 1; // Ritorno successo dopo aver estratto entrambi
}

```

Teoria (Camurati) :

Camurati: Bellman-ford, programmazione dinamica, double hashing

Teoria (Camurati)

- **Algoritmo di Bellman-Ford:**
 - **Scopo:** Risolve il problema dei cammini minimi a sorgente singola in un grafo orientato. La sua particolarità è che accetta archi con peso negativo (cosa che Dijkstra non può fare).
 - **Meccanismo (Rilassamento):** Sapendo che in un grafo di V vertici un cammino semplice ha al massimo $V-1$ archi, l'algoritmo esegue esattamente $V-1$ iterazioni. In ogni singola iterazione, scorre letteralmente *tutti* gli E archi del grafo e prova a "rilassarli" (se la distanza per arrivare al nodo di origine dell'arco più il peso dell'arco è minore della distanza attuale del nodo destinazione, si aggiorna la distanza).
 - **Rilevamento Cicli Negativi:** Finite le $V-1$ iterazioni, esegue un'ultima passata su tutti gli archi. Se riesce a rilassare ancora un arco, significa matematicamente che esiste un ciclo chiuso di peso totale negativo raggiungibile dalla sorgente (in questo caso, l'algoritmo segnala che il problema non ha soluzione divergendo a $-\infty$).

- **Complessità:** Tempo $O(V * E)$ per il doppio ciclo annidato. Spazio $O(V)$ per mantenere gli array delle distanze e dei predecessori.
- **Programmazione Dinamica:**
 - **Concetto:** È un paradigma di progettazione algoritmica usato per problemi di ottimizzazione. Consiste nel dividere un problema complesso in sottoproblemi più piccoli, risolverli una sola volta e memorizzarne il risultato per evitare di ricalcolarli in futuro.
 - **Requisiti fondamentali:** Per essere applicabile, il problema deve possedere la "Sottostruttura Ottima" (la soluzione globale ottima è formata dalle soluzioni ottime dei sottoproblemi) e la "Sovrapposizione dei Sottoproblemi" (l'algoritmo ricorsivo ingenuo continuerebbe a risolvere gli stessi identici sottoproblemi più e più volte, come nel calcolo di Fibonacci).
 - **Tecniche (Top-down vs Bottom-up):** Si applica in due modi. Con la "Memoization" (Top-down) si tiene la struttura ricorsiva ma si salva l'output in una cache. Con la "Tabulazione" (Bottom-up) si elimina la ricorsione e si riempie iterativamente una tabella (matrice o array) partendo dai casi base fino alla soluzione finale.
 - **Complessità:** Riduce drasticamente la complessità Temporale (spesso da tempi esponenziali $O(2^n)$ a polinomiali $O(n)$ o $O(n^2)$), al costo di aumentare la complessità Spaziale, solitamente $O(n)$ o $O(n^2)$ per memorizzare la tabella di supporto.
- **Double Hashing (Doppio Hashing):**
 - **Scopo:** È una tecnica di risoluzione delle collisioni nelle tabelle Hash a Indirizzamento Aperto (dove tutti gli elementi stanno direttamente nell'array principale).
 - **Meccanismo:** Quando l'inserimento di una chiave causa una collisione (la cella calcolata dalla prima funzione hash $h1$ è occupata), si usa una seconda funzione hash indipendente, $h2$, per calcolare la "lunghezza del passo" o "salto" da fare per i tentativi successivi. La formula per il tentativo 'i' diventa:

$$(h1(\text{chiave}) + i * h2(\text{chiave})) \text{ MOD } \text{dimensione_tabella}.$$
 - **Regole d'oro per $h2$:** Il risultato di $h2(\text{chiave})$ NON deve mai essere 0, altrimenti il passo è nullo e l'algoritmo si blocca in un loop infinito sulla stessa cella. Inoltre, $h2(\text{chiave})$ dovrebbe essere coprimo (primo relativo) rispetto alla dimensione della tabella, per garantire che la sequenza di salti visiti tutte le celle prima di ripetersi.
 - **Vantaggi rispetto ad altre tecniche:** Elimina sia il "Clustering Primario" (problema dell'ispezione lineare dove si formano blocchi contigui di dati) sia il

"Clustering Secondario" (problema dell'ispezione quadratica), poiché chiavi diverse che collidono allo stesso indice iniziale avranno salti h_2 diversi, disperdendosi in modo uniforme.

- **Complessità:** Ricerca, Inserimento e Cancellazione costano Tempo $O(1)$ nel caso medio. Degenerano a $O(n)$ nel caso pessimo se la tabella è piena e bisogna ispezionarla tutta. Spazio $O(n)$ per la tabella.
-

Data : 17,5 -> 22 (1.75 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi:

- Dato un grafo orientato liste adiacenza verifica se un elenco di vertici è un ciclo nel grafo int GRAPHcheckC(Graph g, int *p, int n)

Gemini ha detto

Programmazione (Cabodi)

- **Verifica se un elenco di vertici forma un ciclo (Grafo orientato con liste di adiacenza):**
 - **Logica dell'algoritmo:** Un ciclo è un cammino chiuso. L'array p di dimensione n rappresenta la sequenza dei vertici visitati. Affinché sia un ciclo valido, per ogni elemento $p[i]$ deve esistere un arco diretto verso $p[i+1]$. Inoltre, l'ultimo vertice $p[n-1]$ deve avere un arco di chiusura diretto verso il primo vertice $p[0]$.
 - **Meccanismo:** Scorro l'array con un ciclo for. Per gestire elegantemente la chiusura circolare (l'arco dall'ultimo al primo nodo) senza duplicare codice, uso l'operatore modulo: cerco sempre l'arco da $p[i]$ a $p[(i+1)\%n]$. La ricerca dell'arco si fa scorrendo la lista di adiacenza del nodo sorgente. Se anche un solo arco manca, la sequenza si interrompe e ritorno falso (0). Se il ciclo for termina indenne, tutti gli archi esistono e ritorno vero (1).
 - **Complessità Temporale:** $O(n * \text{grado_max})$, dove n è la lunghezza dell'array p e grado_max è il numero massimo di archi uscenti di un vertice (nel caso pessimo $O(V)$). Tempo totale approssimabile a $O(n * V)$.
 - **Complessità Spaziale:** $O(1)$, l'algoritmo non alloca strutture dati ausiliarie, usa solo indici e puntatori di scorrimento.


```

#include <stdlib.h>

// Definizione del nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione per verificare se l'array p di dimensione n è un ciclo
int GRAPHcheckC(Graph g, int *p, int n) {
    // Un ciclo deve avere senso logico (almeno 1 nodo per un auto-anello, o vuoto
    // gestito)
    if (g.adj == NULL || p == NULL || n <= 0) return 0;

    // Itero su tutti i vertici forniti nell'elenco
    for (int i = 0; i < n; i++) {
        // Identifico il nodo di partenza corrente
        int u = p[i];

        // Identifico il nodo di arrivo: uso il modulo per mappare p[n] di nuovo a
        // p[0]
        int v_next = p[(i + 1) % n];

        // Se uno dei vertici forniti non esiste nel grafo, la sequenza è invalida
        if (u < 0 || u >= g.V || v_next < 0 || v_next >= g.V) return 0;

        // Inizio a scorrere la lista degli archi uscenti dal nodo u
        EdgeNode* edge = g.adj[u];
        int edge_found = 0;

        // Cerco l'arco diretto verso v_next
        while (edge != NULL) {
            if (edge->v == v_next) {
                // Arco trovato, la continuità del ciclo è salva per questo passo
                edge_found = 1;
                break;
            }
            edge = edge->next; // Passo al prossimo arco uscente
        }

        // Se ho esplorato tutta la lista uscente di u senza trovare v_next, il
        // ciclo è rotto
        if (edge_found == 0) {
            return 0;
        }
    }
}

```

```

    }
}

// Se sono arrivato qui, tutti gli archi (compreso quello di chiusura) esistono
return 1;
}

```

Teoria (Camurati) :

Camurati:

- Heap: definizione e spiega implementazione con vettore
- Algoritmo di Er e partizionamento.
- Algoritmo di Er è considerato un modello di calcolo combinatorio?

Teoria (Camurati)

- **Definizione di Heap:** Un Heap è un albero binario "quasi completo", ovvero riempito totalmente in ogni suo livello, eccezion fatta al più per l'ultimo, che deve essere riempito rigorosamente da sinistra verso destra senza lasciare "buchi". In un Max-Heap, vige la proprietà di ordinamento parziale: il valore di ogni nodo padre è sempre maggiore o uguale a quello di entrambi i suoi figli. In un Min-Heap vale l'opposto (padre minore o uguale).
- **Implementazione con vettore (Array):**
 - **Meccanismo:** Grazie alla forma quasi completa e compatta, l'Heap non si implementa con i classici nodi e puntatori, ma si mappa direttamente su un array lineare. Questo elimina totalmente l'overhead di memoria dovuto ai puntatori left e right.
 - **Regole di navigazione matematiche:** Posizionando la radice all'indice 0 dell'array, la topologia dell'albero si ricava con semplici calcoli algebrici sugli indici. Dato un nodo all'indice i :
 - Il figlio sinistro si trova all'indice $2*i + 1$.
 - Il figlio destro si trova all'indice $2*i + 2$.
 - Il padre si trova all'indice intero $(i - 1) / 2$.
 - **Complessità:** Spazio $O(n)$ per l'array. Accesso al massimo/minimo in Tempo $O(1)$. Inserimento ed Estrazione in Tempo $O(\log n)$ a causa delle operazioni di risalita/discesa (sift-up/sift-down) lungo l'altezza dell'albero.
- **Algoritmo di Er e partizionamento:**

- **Scopo:** L'algoritmo di M.C. Er serve a generare in modo sistematico ed esaustivo tutte le possibili **partizioni di un insieme**. Una partizione è una suddivisione di un insieme di N elementi in sottoinsiemi non vuoti e mutuamente disgiunti (es. partizionare {A,B,C} in {{A,B}, {C}}).
 - **Stringhe a Crescita Vincolata (RGS - Restricted Growth Strings):** L'algoritmo non muove insiemi fisici, ma genera array di interi chiamati RGS. Un RGS è una sequenza in cui il primo elemento è sempre 0, e ogni elemento successivo può essere al massimo uguale al valore massimo visto finora più uno (es. $c[i] \leq \max(c[0] \dots c[i-1]) + 1$). Ogni valore numerico nell'array rappresenta l'ID del "blocco" a cui è assegnato l'elemento i-esimo.
 - **Meccanismo di Er:** Genera queste stringhe in ordine lessicografico, partendo da 000...0 (tutti gli elementi nello stesso blocco) fino a 012...(n-1) (ogni elemento in un blocco separato), garantendo di non generare partizioni duplicate equivalenti a meno di rinominare i blocchi.
 - **Complessità:** La complessità Spaziale è $O(n)$ per memorizzare la stringa e gli array di supporto dei massimi. La complessità Temporale scala proporzionalmente ai Numeri di Bell ($O(B_n)$), che crescono in modo super-esponenziale (più veloci dell'esponenziale classico ma meno del fattoriale).
 - **Algoritmo di Er considerato modello del calcolo combinatorio:**
 - **La risposta è assolutamente Sì.** L'algoritmo di Er è l'emblema del modello del calcolo combinatorio (o forza bruta / ricerca esaustiva).
 - **Perché:** Non cerca una singola soluzione ottima tramite scelte furbe o scorciatoie (come farebbe un algoritmo Greedy o di Programmazione Dinamica). Il suo scopo intrinseco è **enumerare alla cieca l'intero spazio delle configurazioni matematiche possibili** (tutte le partizioni). Poiché lo spazio delle partizioni cresce secondo i Numeri di Bell, l'algoritmo sconta una complessità Temporale intrattabile $O(B_n)$, caratteristica che definisce esattamente l'esplosione tipica dei modelli di calcolo combinatorio puro.
-

Data : 16 -> 18 (0lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi: - **data una lista concatenata dividerla in due liste, la prima degli elementi pari e la seconda degli elementi dispari dell'originale e ritornarla al chiamante**

Programmazione (Cabodi)

- **Logica dell'algoritmo (Divisione lista pari e dispari):**
 - In C, una funzione non può ritornare due valori (le due teste delle nuove liste) tramite un semplice return. Usiamo il passaggio per riferimento tramite **doppi puntatori** (Node**) passati come argomenti.
 - Invece di allocare nuovi nodi (che sprecherebbe Spazio $O(n)$), manipoliamo direttamente i puntatori next dei nodi esistenti, "scollegandoli" dalla lista originale e "riagganciandoli" a due nuove catene distinte.
 - **Trucco per l'orale (Nodi Dummy/Sentinella):** Per evitare di dover scrivere continui if per controllare se la testa della nuova lista pari o dispari è NULL (il caso di inserimento in testa), si usano due nodi fittizi locali. Questo rende il codice estremamente pulito ed elegante da scrivere alla lavagna.
- **Complessità Temporale:** $O(n)$, dove n è la lunghezza della lista originale. Attraversiamo la lista una sola volta.
- **Complessità Spaziale:** $O(1)$, poiché modifichiamo solo i puntatori sul posto senza fare alcuna chiamata a malloc.

```
#include <stdlib.h>

// Struttura del nodo della lista
typedef struct Node {
    int val;
    struct Node* next;
} Node;

// Funzione che divide la lista. Usa doppi puntatori per restituire le due nuove
// teste al chiamante.
void split_even_odd(Node* head, Node** out_even, Node** out_odd) {
    // Uso nodi fittizi allocati sullo stack per evitare if speciali sul primo
    // inserimento
    Node dummy_even = {0, NULL};
    Node dummy_odd = {0, NULL};

    // Puntatori "coda" per tenere traccia dell'ultimo nodo aggiunto alle
    // rispettive liste
    Node* tail_even = &dummy_even;
    Node* tail_odd = &dummy_odd;

    // Puntatore per scorrere la lista originale
    Node* curr = head;

    // Scorro l'intera lista una sola volta
    while (curr != NULL) {
```

```

    // Se il valore è pari, lo aggancio alla coda della lista pari
    if (curr->val % 2 == 0) {
        tail_even->next = curr;
        tail_even = tail_even->next; // Avanzo la coda
    }
    // Altrimenti lo aggancio alla coda della lista dispari
    else {
        tail_odd->next = curr;
        tail_odd = tail_odd->next; // Avanzo la coda
    }

    // Passo al nodo successivo della lista originale
    curr = curr->next;
}

// Fondamentale: chiudo le due nuove liste impostando l'ultimo next a NULL.
// Altrimenti, manterrebbero i vecchi collegamenti della lista originale,
creando cicli o errori.
tail_even->next = NULL;
tail_odd->next = NULL;

// Assegno i risultati bypassando i nodi fittizi (restituisco ciò che ci sta
attaccato dietro)
*out_even = dummy_even.next;
*out_odd = dummy_odd.next;
}

```

Teoria (Camurati) :

Camurati:

- paradigma greedy (cos'è e come funziona)
- divide and conquer (equazione alle ricorrenze) + moltiplicazione tra interi e karatsuba (descrivere e dire perché il secondo è migliore)

Teoria (Camurati)

- **Paradigma Greedy (Cos'è e come funziona):**
 - **Definizione:** È una tecnica di progettazione di algoritmi che costruisce la soluzione passo dopo passo, effettuando a ogni iterazione la scelta che appare "localmente ottima" in quel preciso istante, senza mai guardare al futuro o riconsiderare scelte passate.
 - **Meccanismo:** L'algoritmo mantiene un insieme di candidati. A ogni passo, una "funzione di selezione" sceglie il candidato migliore disponibile. Una "funzione di fattibilità" controlla se il candidato può essere aggiunto alla soluzione

parziale senza violare i vincoli del problema. Se è fattibile, viene aggiunto definitivamente.

- **Requisiti per il successo:** Funziona (cioè trova l'ottimo globale) solo se il problema gode della **Proprietà della scelta greedy** (una scelta localmente ottima porta sempre a una soluzione globalmente ottima) e della **Sottostruttura ottima** (la soluzione globale contiene le soluzioni ottime dei sottoproblemi).
- **Vantaggi e Svantaggi:** È estremamente veloce (spesso Tempo $O(n \log n)$ dovuto all'ordinamento iniziale dei candidati, Spazio $O(1)$ o $O(V)$), ma se il problema non rispetta le due proprietà matematiche, il Greedy fallisce producendo una soluzione sub-ottima (es. fallisce nel problema dello Zaino Intero).
- **Divide et Impera ed Equazione alle Ricorrenze:**
 - **Meccanismo:** Paradigma che divide il problema in sotto-problemi indipendenti e più piccoli, li risolve ricorsivamente, e infine combina le loro soluzioni per ottenere quella del problema originario.
 - **Equazione di Ricorrenza:** Il costo temporale si modella con la formula $T(n) = a * T(n/b) + f(n)$. Dove 'a' è il numero di chiamate ricorsive (sotto-problemi), 'n/b' è la dimensione di ciascun sotto-problema rispetto all'input 'n', e 'f(n)' è il costo (Tempo) per dividere l'input e combinare i risultati. Si risolve tipicamente applicando il **Master Theorem**.
- **Moltiplicazione tra interi (Standard vs Karatsuba):**
 - **Moltiplicazione Standard (Divide et Impera):** Divide due numeri X e Y di 'n' cifre a metà: $X = A10^{(n/2)} + B$ e $Y = C10^{(n/2)} + D$. Svolge il prodotto calcolando 4 moltiplicazioni ricorsive: AC, AD, BC, BD. L'equazione è $T(n) = 4T(n/2) + O(n)$. Per il Master Theorem, il Tempo è $O(n^2)$.
 - **L'intuizione di Karatsuba:** Osserva che il termine centrale (AD + BC) può essere calcolato senza fare due moltiplicazioni separate. Calcola il prodotto $(A+B)*(C+D)$ e da questo sottrae i termini AC e BD (che ha già calcolato a parte).
 - **Meccanismo di Karatsuba:** Esegue solo 3 moltiplicazioni ricorsive invece di 4: calcola AC, BD, e $(A+B)*(C+D)$.
 - **Perché Karatsuba è migliore:** L'equazione di ricorrenza diventa $T(n) = 3T(n/2) + O(n)$. Risolvendola con il Master Theorem, il Tempo crolla a $O(n^{\log_2(3)})$, ovvero circa $O(n^{1.58})$. La complessità Spaziale per entrambi rimane $O(\log n)$ per lo stack ricorsivo. Evitare una chiamata ricorsiva a ogni livello dell'albero riduce drasticamente il tempo di esecuzione per numeri con migliaia di cifre, passando da un tasso di crescita quadratico a uno sub-quadratico.

Data : 17.5 -> ?? (1.25 lab) 14/03/2025

Programmazione (Cabodi) :

- scrivere una funzione che: data una tabella di Hash con linear chaining calcoli la lunghezza media delle liste (si trascurino le liste vuote) (per il ritorno della media avevo fatto un ritorno ad intero: $\text{return len}/n$; e mi ha chiesto cosa avrei dovuto fare per ritornare un float, bastava dire di fare una cast di tipo float solo al dividendo o al divisore: $\text{return ((float) len)}/n$; - se dovessi modificare la dimensione di una tabella di Hash, cosa dovrei fare? (reinserire tutti gli elementi siccome la funzione di Hash è cambiata)

Programmazione (Cabodi)

- **Logica dell'algoritmo (Lunghezza media liste non vuote):**
 - Scorro l'intero array di puntatori della tabella Hash (da 0 alla dimensione della tabella).
 - Per ogni indice, se il puntatore alla testa della lista non è NULL, significa che la lista non è vuota. Incremento un contatore delle liste valide.
 - Inizio a scorrere la lista concatenata nodo per nodo, incrementando un contatore globale dei nodi totali.
 - Alla fine, restituisco il rapporto tra i nodi totali e le liste valide. Per ottenere un risultato decimale accurato in C, è fondamentale eseguire un cast esplicito a float del numeratore (o del denominatore), altrimenti il compilatore esegue una divisione intera troncando i decimali prima della restituzione.
- **Complessità Temporale:** $O(M + N)$, dove M è la dimensione dell'array (numero di "bucket") e N è il numero totale di elementi inseriti, poiché visito ogni cella e ogni nodo esattamente una volta.
- **Complessità Spaziale:** $O(1)$, poiché utilizzo solo variabili intere di conteggio.

```
#include <stdlib.h>

// Definizione del nodo per il linear chaining
typedef struct Node {
    int key;
    struct Node* next;
} Node;

// Struttura della Tabella Hash
typedef struct {
    int size;           // Dimensione dell'array (M)
```

```

    Node** table; // Array di puntatori alle teste delle liste
} HashTable;

// Funzione per calcolare la lunghezza media delle liste non vuote
float average_list_length(HashTable* ht) {
    // Controllo di sicurezza se la tabella non esiste
    if (ht == NULL || ht->size == 0 || ht->table == NULL) return 0.0;

    int total_nodes = 0; // Somma di tutti i nodi presenti
    int non_empty_lists = 0; // Contatore delle liste che hanno almeno un nodo

    // Scorro tutti i bucket dell'array della tabella Hash
    for (int i = 0; i < ht->size; i++) {
        Node* current = ht->table[i];

        // Se il bucket contiene qualcosa, lo conteggio tra le liste valide
        if (current != NULL) {
            non_empty_lists++;

            // Scorro la lista per contare fisicamente quanti elementi ci sono
            while (current != NULL) {
                total_nodes++;
                current = current->next; // Avanzo al nodo successivo
            }
        }
    }

    // Se tutte le liste sono vuote, evito l'errore matematico di divisione per
    zero
    if (non_empty_lists == 0) return 0.0;

    // Risposta Orale: Eseguo il cast a float del dividendo per forzare
    // l'aritmetica in virgola mobile, evitando il troncamento della divisione
    intera
    return ((float)total_nodes) / non_empty_lists;
}

```

Risposta a voce (Modifica dimensione tabella Hash):

- **L'operazione necessaria:** Non si può fare un semplice realloc o copiare i puntatori. Bisogna eseguire un'operazione chiamata **Rehashing**.
- **Il motivo:** La funzione di Hash tipica è $h(k) = k \bmod M$. Se cambio M (la dimensione della tabella), l'indice di destinazione di ogni chiave cambia radicalmente.
- **Meccanismo:** Alloco una nuova tabella con la nuova dimensione. Scorro l'intera vecchia tabella nodo per nodo e, per ogni elemento, ricalcolo il suo nuovo hash in base alla nuova dimensione e lo reinserisco da zero nella nuova tabella. Infine, libero la memoria della vecchia. Complessità Temporale: $O(N)$.

Teoria (Camurati) :

- enuncia il teorema e il corollario degli archi sicuri

- la ricerca del cammino a peso massimo è un problema di tipo dinamico? Perché?

Teoria (Camurati)

- **Teorema degli Archi Sicuri:**

- **Contesto:** Fondamento matematico degli algoritmi Greedy per il Minimum Spanning Tree (MST), come Prim e Kruskal.
- **Definizioni:** Immagina 'A' come un sottoinsieme di archi già confermati per l'MST. Un "taglio" divide i vertici in due insiemi. Il taglio "rispetta A" se non spezza nessun arco di 'A'. Un "arco leggero" è l'arco col peso minore tra quelli che attraversano il taglio.
- **Enunciato:** Dato un sottoinsieme 'A' incluso in un MST e un taglio che rispetta 'A', l'arco leggero che attraversa quel taglio è un arco **sicuro**. Significa che può essere aggiunto all'insieme 'A' con la garanzia matematica che farà parte dell'MST globale ottimo.

- **Corollario degli Archi Sicuri:**

- **Enunciato:** L'insieme 'A' di archi forma una foresta (componenti connesse disgiunte). Se si prende uno qualsiasi di questi alberi e si individua l'arco più leggero in assoluto che lo unisce al resto del grafo, quell'arco è sicuro.
- **Impatto:** Giustifica perché Prim (che espande un solo albero) e Kruskal (che unisce foreste) trovano la soluzione perfetta senza dover riconsiderare le scelte passate in Tempo $O(E \log V)$.

- **Cammino a peso massimo e Programmazione Dinamica:**

- **Il problema nei grafi generici:** In un grafo con cicli, trovare il "cammino semplice" (senza ripetizioni di nodi) di peso massimo è un problema NP-Arduo e **NON** è risolvibile con la Programmazione Dinamica.
- **Perché fallisce (Mancanza Sottostruttura Ottima):** La Programmazione Dinamica esige che la soluzione ottima globale sia formata da soluzioni ottime locali. Nel cammino massimo semplice, questo crolla: prendere il cammino locale più lungo tra due nodi intermedi potrebbe costringerti a "bruciare" (visitare) nodi cruciali, bloccandoti in un vicolo cieco e impedendoti di raggiungere la destinazione finale senza creare cicli vietati.
- **L'unica eccezione (I DAG):** La Programmazione Dinamica diventa utilizzabile **esclusivamente** sui DAG (Directed Acyclic Graphs). Essendo matematicamente vietati i cicli, il rischio di ripassare sugli stessi nodi

scompare, la Sottostuttura Ottima viene ripristinata e il problema si risolve in Tempo lineare $O(V + E)$ processando i nodi in Ordinamento Topologico.

Data : 15,50 -> 18 (0 lab) 14/03/2025 ||||| QUA

Programmazione (Cabodi) :

Cabodi: - PQchange lista ordinata

Programmazione (Cabodi)

- **Logica dell'algoritmo (PQchange su lista ordinata):**
 - **Obiettivo:** Modificare la priorità di un elemento in una Coda a Priorità (implementata come lista concatenata e ordinata per priorità decrescente) e ripristinare l'ordinamento.
 - **Fase 1 (Ricerca ed Estrazione):** Si scorre la lista per trovare il nodo bersaglio (tramite un identificatore). Tenendo traccia del nodo precedente, si "scollega" fisicamente il nodo bersaglio dalla lista bypassando i puntatori, senza deallocarlo.
 - **Fase 2 (Aggiornamento e Reinserimento):** Si aggiorna il valore della priorità nel nodo appena estratto. Si riparte dalla testa della lista per cercare la nuova posizione corretta (scorrendo finché si trovano priorità maggiori o uguali). Si riaggancia il nodo reinserendolo.
 - **Complessità Temporale:** $O(N)$, dove N è la lunghezza della lista. Nel caso peggiore, l'estrazione richiede di scorrere tutta la lista e il reinserimento richiede un ulteriore scorrimento.
 - **Complessità Spaziale:** $O(1)$, l'operazione avviene interamente "in-place" riorganizzando solo i puntatori, senza allocare nuova memoria.

```
#include <stdlib.h>

// Definizione del nodo della Coda a Priorità
typedef struct Node {
    int id;           // Identificativo univoco dell'elemento
    int priority;     // Valore di priorità (assumiamo ordinamento decrescente)
    struct Node* next; // Puntatore al prossimo elemento
} Node;

// Funzione che modifica la priorità e riordina. Ritorna la (nuova) testa della
// lista.
```

```

Node* PQchange(Node* head, int target_id, int new_priority) {
    if (head == NULL) return NULL; // Coda vuota

    Node *curr = head;
    Node *prev = NULL;

    // 1. RICERCA: Trovo il nodo da modificare e il suo predecessore
    while (curr != NULL && curr->id != target_id) {
        prev = curr;
        curr = curr->next;
    }

    // Se il nodo non esiste nella coda, ritorno la testa inalterata
    if (curr == NULL) return head;

    // 2. ESTRAZIONE: Scollego il nodo dalla sua posizione attuale
    if (prev == NULL) {
        // Il nodo era in testa, la nuova testa diventa il secondo elemento
        head = curr->next;
    } else {
        // Il nodo era in mezzo o in fondo, bypasso il nodo corrente
        prev->next = curr->next;
    }

    // 3. AGGIORNAMENTO: Modifico la priorità del nodo estratto
    curr->priority = new_priority;
    curr->next = NULL; // Pulisco il puntatore per evitare cicli

    // 4. REINSERIMENTO: Cerco la nuova posizione basata sulla nuova priorità
    (decescente)
    Node *ins_curr = head;
    Node *ins_prev = NULL;

    // Scorro finché trovo nodi con priorità maggiore o uguale a quella nuova
    while (ins_curr != NULL && ins_curr->priority >= new_priority) {
        ins_prev = ins_curr;
        ins_curr = ins_curr->next;
    }

    // Riaggancio il nodo nella nuova posizione trovata
    if (ins_prev == NULL) {
        // La nuova priorità è la massima assoluta, inserimento in testa
        curr->next = head;
        head = curr;
    } else {
        // Inserimento in mezzo o in coda
        curr->next = ins_curr;
        ins_prev->next = curr;
    }
}

```

```
// Ritorno il puntatore alla testa della lista aggiornata
return head;
}
```

Teoria (Camurati) :

Camurati:

- powerset cos'è, come si ottiene
- successore/predecessore BST cosa sono come si trovano

Teoria (Camurati)

- **Powerset (Insieme delle parti):**

- **Cos'è:** È l'insieme di tutti i possibili sottoinsiemi generabili a partire da un insieme di partenza S, includendo sempre l'insieme vuoto e l'insieme S stesso. Se l'insieme di partenza contiene N elementi, il suo Powerset conterrà esattamente 2^N sottoinsiemi.
- **Come si ottiene (Modello Binario):** Si assegna un bit a ogni elemento dell'insieme (0 = escluso, 1 = incluso). L'algoritmo si riduce a generare tutti i numeri binari da 0 fino a $(2^N)-1$. Ogni numero binario rappresenta un sottoinsieme univoco.
- **Come si ottiene (Modello Ricorsivo / Backtracking):** Il Backtracking esplora un albero delle decisioni in profondità. Per ogni singolo elemento, si effettuano due chiamate ricorsive diramate: una in cui l'elemento viene forzatamente aggiunto al sottoinsieme parziale, e una in cui viene ignorato. Quando si raggiunge la profondità N, si stampa il sottoinsieme trovato.
- **Complessità:** Tempo $O(N * 2^N)$ nel caso in cui si vogliano stampare o salvare tutti gli elementi di tutti i sottoinsiemi. Spazio $O(N)$ corrispondente all'altezza dello stack di ricorsione (o all'array binario di supporto).

- **Successore e Predecessore in un BST (Binary Search Tree):**

- **Cosa sono:** In un albero binario di ricerca, l'ordinamento naturale è dato dalla visita "In-Order" (simmetrica). Il Successore di un nodo X è il nodo con la chiave immediatamente superiore a quella di X. Il Predecessore è il nodo con la chiave immediatamente inferiore.
- **Come trovare il Successore (Caso 1 - Esiste sottoalbero destro):** Se il nodo X ha un figlio destro, il suo successore è banalmente il valore minimo di tutto il sottoalbero destro. L'algoritmo fa un passo a destra e poi scende tutto a sinistra finché trova un nodo senza figlio sinistro.

- **Come trovare il Successore (Caso 2 - Nessun sottoalbero destro):** Se X non ha un figlio destro, il successore è un suo antenato. L'algoritmo risale l'albero passando per i padri. Il successore è il primo antenato per cui si effettua una "svolta a destra" risalendo (ovvero, è il primo antenato di cui il nodo corrente, o un suo predecessore nel cammino di risalita, risulta essere figlio sinistro). Se si arriva alla radice senza mai fare questa svolta, il nodo X era il massimo assoluto e non ha successore.
 - **Come trovare il Predecessore:** La logica è speculare. Caso 1 (esiste sottoalbero sinistro): si cerca il massimo del sottoalbero sinistro (un passo a sinistra, poi tutto a destra). Caso 2 (nessun sottoalbero sinistro): si risale l'albero cercando il primo antenato di cui il percorso corrente risulta essere figlio destro.
 - **Complessità:** Entrambe le ricerche si sviluppano seguendo un cammino monodirezionale lungo l'altezza dell'albero 'h'. Tempo $O(h)$, che corrisponde a $O(\log N)$ in alberi bilanciati (AVL, Red-Black) ma degrada a $O(N)$ nel caso pessimo di un BST degenerare (una lista). Spazio $O(1)$ se la risalita si implementa iterativamente con i puntatori ai padri.
-

Data : 17 -> 19 (0 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi: - BSTpartition - Come si può implementare questa funzione con un BST senza parametro N a ogni nodo? (ho risposto con una visita in order e un contatore di tipo static che viene incrementato ogni volta che il nodo viene completato)

🔗 Logica dell'algoritmo (BSTpartition senza campo N):

- **Obiettivo:** La funzione partition ha lo scopo di trovare il k-esimo elemento più piccolo di un BST e portarlo alla radice tramite una serie di rotazioni ad albero.
- **L'implementazione standard:** Di solito, ogni nodo ha un campo N (numero di nodi nel sottoalbero) per calcolare il rango a ogni passo e scendere a destra o a sinistra in Tempo $O(h)$.
- **La tua risposta all'orale (Visita In-Order):** La tua intuizione è esatta. Senza il campo N, l'unico modo per trovare il k-esimo elemento è simulare un ordinamento tramite una visita In-Order (Sinistra-Radice-Destra), decrementando un contatore dei nodi visitati.

- **Correzione da fare all'esame (Perché NON usare static):** Evita di dire al Prof. Cabodi di usare una variabile static. In C, una variabile static mantiene il valore tra chiamate diverse. Se l'utente chiama la funzione partition due volte di fila nello stesso programma, il contatore non si azzerà e il codice fallisce. La soluzione robusta è passare il contatore per riferimento (tramite un puntatore `int* k`).
- **Meccanismo di risalita:** Quando il contatore arriva a zero, il nodo target è trovato. Impostiamo il contatore a un valore sentinella (es. -1) per avvisare le chiamate ricorsive precedenti. Durante la chiusura della ricorsione (il *backtracking*), applichiamo una rotazione destra (rotR) se il target emerge dal sottoalbero sinistro, e una rotazione sinistra (rotL) se emerge dal destro, facendolo "galleggiare" fino alla radice.

❓ **Complessità Temporale:** $O(N)$ nel caso pessimo (se il BST è degenere o se cerchiamo l'ultimo elemento), poiché l'assenza del campo taglia-sottoalbero ci costringe a visitare fisicamente i nodi in ordine.

❓ **Complessità Spaziale:** $O(h)$, dove h è l'altezza dell'albero, per via dello stack delle chiamate ricorsive della visita In-Order.

```
#include <stdlib.h>

// Definizione del nodo del BST
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione di supporto: Rotazione a Destra
Node* rotR(Node* h) {
    Node* x = h->left;
    h->left = x->right;
    x->right = h;
    return x;
}

// Funzione di supporto: Rotazione a Sinistra
Node* rotL(Node* h) {
    Node* x = h->right;
    h->right = x->left;
    x->left = h;
    return x;
}

// Funzione ricorsiva di partizionamento.
// k_ptr è un puntatore al rango cercato (0-indexed, 0 = minimo)
Node* BSTpartition(Node* root, int* k_ptr) {
    // Caso base: albero vuoto
```

```

if (root == NULL) return NULL;

// 1. Visita In-Order: scendo prima tutto a sinistra
root->left = BSTpartition(root->left, k_ptr);

// Se la chiamata a sinistra ha trovato il nodo (k_ptr è diventato -1),
// il nodo sta risalendo. Applico una rotazione destra per portarlo su.
if (*k_ptr == -1) {
    return rotR(root);
}

// 2. Visita In-Order: Analizzo il nodo corrente (la radice attuale)
if (*k_ptr == 0) {
    // Ho trovato il k-esimo elemento!
    *k_ptr = -1; // Imposto il flag sentinella per innescare le rotazioni in
risalita
    return root; // Inizio a ritornare il nodo bersaglio
}

// Non è questo il nodo, scalo il contatore prima di andare a destra
(*k_ptr)--;

// 3. Visita In-Order: scendo nel ramo destro
root->right = BSTpartition(root->right, k_ptr);

// Se la chiamata a destra ha trovato il nodo,
// applico una rotazione sinistra per farlo risalire.
if (*k_ptr == -1) {
    return rotL(root);
}

// Ritorno il nodo inalterato se il target non era nel mio sottoalbero
return root;
}

// Funzione wrapper pulita da chiamare nel main
Node* partition_wrapper(Node* root, int k) {
    // Passo l'indirizzo della variabile locale così si resetta a ogni nuova
esecuzione
    return BSTpartition(root, &k);
}

```

Teoria (Camurati) :

Camurati: - Algoritmo di Kosaraju (cosa riguarda), definizione di componenti fortemente connesse - Complessità dell'algoritmo che inverte il grafo

Teoria (Camurati)

- **Componenti Fortemente Connesse (SCC - Strongly Connected Components):**
 - **Definizione:** In un grafo orientato, una SCC è un sottografo "massimale" in cui, per ogni singola coppia di vertici u e v , esiste sempre un cammino diretto da u a v e un cammino di ritorno da v a u . "Massimale" significa che non è possibile aggiungere un altro vertice del grafo a questo gruppo senza distruggere la proprietà di forte connessione.
- **Algoritmo di Kosaraju:**
 - **Scopo:** Trova e isola tutte le Componenti Fortemente Connesse di un grafo orientato.
 - **Meccanismo (I 3 Passi):**
 1. Esegue una visita DFS completa sul grafo originale. Quando l'esplorazione ricorsiva di un vertice e di tutti i suoi discendenti è totalmente conclusa (Tempo di fine visita / in post-ordine), il vertice viene inserito in cima a uno Stack.
 2. Calcola il grafo trasposto (invertendo la direzione di tutti gli archi).
 3. Estrae i vertici dallo Stack uno alla volta. Se il vertice estratto non è ancora stato visitato, lancia una nuova DFS sul grafo trasposto partendo da esso. Tutti i nodi raggiunti in questa singola DFS formano esattamente una SCC.
 - **Complessità Temporale:** $O(V + E)$. L'algoritmo esegue due visite DFS (costo $V+E$ ciascuna) e un'operazione di inversione del grafo (costo $V+E$). Asintoticamente rimane lineare.
 - **Complessità Spaziale:** $O(V + E)$ per memorizzare il nuovo grafo trasposto, lo Stack di supporto e gli array dei visitati.
- **Complessità dell'algoritmo che inverte il grafo (Grafo Trasposto):**
 - **Logica:** Per invertire un grafo con liste di adiacenza, si alloca un nuovo grafo vuoto con V vertici. Si scorre l'intero array delle adiacenze del grafo originale. Per ogni arco diretto $u \rightarrow v$ trovato nella lista del nodo u , si inserisce un arco opposto $v \rightarrow u$ nella lista del nodo v del nuovo grafo.
 - **Complessità Temporale:** $O(V + E)$. Il ciclo esterno scorre i V vertici, il ciclo interno scorre gli E archi totali. Ogni nodo e ogni arco viene processato esattamente una volta.
 - **Complessità Spaziale:** $O(V + E)$ necessari per allocare fisicamente in memoria la struttura del nuovo grafo invertito con tutte le sue liste di adiacenza.

Data : 17.25 -> 19 (0 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi: - Fare una funzione che dato un HEAP estragga i due dati più grandi

Programmazione (Cabodi)

- **Logica dell'algoritmo (Estrazione dei due massimi da un Heap):**
 - In un Max-Heap rappresentato tramite array, il valore massimo assoluto si trova obbligatoriamente nella radice (indice 0).
 - La procedura più sicura e modulare prevede di estrarre la radice sovrascrivendola con l'ultimo elemento dell'array, per poi chiamare la funzione di ripristino (heapify o sift-down) per far sprofondare questo nuovo nodo fino alla sua posizione corretta.
 - Una volta completata la prima estrazione e ripristinato l'albero, il secondo massimo originario sarà automaticamente salito nella posizione di radice. Basterà chiamare la stessa funzione di estrazione una seconda volta.
- **Complessità Temporale:** $O(\log n)$. L'estrazione viene chiamata un numero costante di volte (2), e ogni estrazione esegue un heapify che costa $O(\log n)$ corrispondente all'altezza dell'albero.
- **Complessità Spaziale:** $O(1)$, implementando la discesa del nodo in modo iterativo senza usare lo stack di ricorsione.

```
#include <stdlib.h>

// Funzione iterativa per far scendere un nodo e ripristinare il Max-Heap (Spazio O(1))
void max_heapify(int* heap, int size, int index) {
    while (1) {
        int left = 2 * index + 1; // Calcolo matematico indice figlio sinistro
        int right = 2 * index + 2; // Calcolo matematico indice figlio destro
        int largest = index;       // Assumo inizialmente che il nodo corrente sia
        // il maggiore

        // Se il figlio sinistro è dentro l'array ed è più grande, lui è il nuovo
        // candidato
        if (left < size && heap[left] > heap[largest]) largest = left;

        // Se il figlio destro esiste ed è ancora più grande, vince lui
        if (right < size && heap[right] > heap[largest]) largest = right;

        // Se il nodo corrente non è il maggiore, scambiamolo con il maggiore
        if (largest != index) {
            swap(heap, index, largest);
            index = largest;
        }
    }
}
```

```

        if (right < size && heap[right] > heap[largest]) largest = right;

        // Se il nodo corrente è maggiore o uguale ai figli, l'heap è perfetto e mi
fermo
        if (largest == index) break;

        // Altrimenti, scambio il nodo corrente con il figlio maggiore per farlo
scendere fisicamente
        int temp = heap[index];
        heap[index] = heap[largest];
        heap[largest] = temp;

        // Aggiorno l'indice per continuare il controllo nel livello inferiore
        index = largest;
    }
}

// Funzione base per estrarre la radice corrente
int extract_max(int* heap, int* size) {
    int max_val = heap[0];          // Salvo il valore massimo da restituire

    heap[0] = heap[*size - 1];     // Tappo il buco in radice spostandoci l'ultima
foglia dell'albero
    (*size)--;                      // Riduco la dimensione ufficiale scartando
l'ultimo elemento

    max_heapify(heap, *size, 0);    // Faccio sprofondare la nuova radice per
ripristinare l'ordine

    return max_val;
}

// Funzione principale richiesta all'orale: estrae i due massimi e li salva nei
puntatori
int extract_two_max(int* heap, int* size, int* max1, int* max2) {
    // Controllo di sicurezza: l'heap deve esistere e avere almeno due elementi
    if (heap == NULL || *size < 2) return 0;

    // Estraggo il primo massimo (la radice originaria)
    *max1 = extract_max(heap, size);

    // Essendo stato chiamato heapify, il secondo massimo originario è ora in
radice. Lo estraggo.
    *max2 = extract_max(heap, size);

    return 1; // Ritorno 1 come segnale di successo
}

```

Teoria (Camurati) :

Camurati: - Memoization - Collisioni nelle tabelle di hash – Clustering

Teoria (Camurati)

- **Memoization (Memoizzazione):**

- **Cos'è:** È una tecnica di Programmazione Dinamica di tipo *Top-Down*. Mantiene la struttura intuitiva e ricorsiva del paradigma Divide et Impera, ma introduce una "cache" (di solito un array o una tabella hash) per memorizzare i risultati dei sottoproblemi già calcolati.
- **Meccanismo:** Prima di eseguire una chiamata ricorsiva, l'algoritmo controlla la cache. Se il risultato per quel sottoproblema è già presente, lo restituisce istantaneamente in Tempo $O(1)$. Se non c'è, esegue il calcolo, lo salva nella cache e poi lo restituisce.
- **Vantaggi e Complessità:** Evita il ricalcolo disastroso dei sottoproblemi sovrapposti (es. nel calcolo di Fibonacci). Trasforma complessità Temporali esponenziali come $O(2^n)$ in lineari $O(n)$. La complessità Spaziale aumenta a $O(n)$ per ospitare la struttura dati della cache e lo stack delle chiamate ricorsive.

- **Collisioni nelle tabelle di Hash:**

- **Cos'è:** Si verifica quando la funzione di Hash mappa due o più chiavi distinte esattamente nello stesso indice (bucket) dell'array della tabella. Per il principio dei cassetti, sono inevitabili poiché l'universo delle chiavi possibili è sempre enormemente più grande della dimensione limitata della tabella.
- **Tecnica 1 - Chaining (Liste di collisione):** Ogni cella dell'array non contiene direttamente il dato, ma un puntatore alla testa di una lista concatenata. Gli elementi che collidono vengono semplicemente accodati in questa lista.
- **Tecnica 2 - Open Addressing (Indirizzamento aperto):** Tutti gli elementi risiedono direttamente nell'array principale. Se una cella è occupata, l'algoritmo cerca la prima cella libera successiva seguendo una specifica sequenza di ispezione (lineare, quadratica o doppio hash).
- **Complessità:** Le collisioni degradano le prestazioni ottimali. Invece di avere ricerca/inserimento in Tempo $O(1)$, nel caso pessimo (tutte le chiavi nello stesso bucket) il Tempo degenera a $O(n)$, dovendo scorrere interamente la lista o ispezionare tutta la tabella.

- **Clustering:**

- **Cos'è:** È un fenomeno degenerativo tipico delle tabelle Hash a Indirizzamento Aperto. Consiste nella formazione di lunghi blocchi contigui di celle occupate.

Rallenta drasticamente le operazioni perché, per trovare una cella vuota, l'algoritmo è costretto a scavalcare l'intero blocco uno slot alla volta.

- **Clustering Primario:** Causato dall'ispezione lineare (si cerca la cella $i+1$, $i+2$, ecc.). Chiavi che hanno un hash di partenza vicino tendono a fondersi in un unico enorme agglomerato continuo.
 - **Clustering Secondario:** Causato dall'ispezione quadratica (si cerca la cella $i+1^2$, $i+2^2$, ecc.). Se due chiavi diverse generano lo stesso identico hash iniziale, seguiranno esattamente la stessa sequenza di salti, creando un cluster lungo quello specifico percorso.
 - **Soluzione:** Si risolve usando il *Double Hashing* (Doppio Hash), in cui il "salto" per cercare la cella successiva è calcolato da una seconda funzione hash indipendente. Chiavi che collidono all'inizio avranno salti di lunghezza diversa, disperdendosi uniformemente.
-

Data : 17 -> 19 (0 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi ● `GRAPHcnt2(Graph g, int v, int w)`: contare i vertici adiacenti a due vertici passati come parametro, il graph ha la lista di adiacenze.

Programmazione (Cabodi)

- **Logica dell'algoritmo (Contare vertici adiacenti a v e w):**
 - **Obiettivo:** Trovare la cardinalità dell'intersezione tra la lista di adiacenza del vertice v e quella del vertice w.
 - **L'approccio ingenuo da evitare:** Scorrere la lista di v e, per ogni nodo, scorrere tutta la lista di w. Costerebbe Tempo $O(\text{grado}(v) * \text{grado}(w))$, troppo lento.
 - **L'approccio ottimizzato (Vettore di marcatura):** Allochiamo un array temporaneo di supporto (mappa booleana) grande quanto il numero totale di vertici V, inizializzato a 0.
 - Scorriamo la lista di adiacenza di v una singola volta e "marchiamo" con un 1 l'array all'indice corrispondente a ogni vicino di v.

- Successivamente, scorriamo la lista di adiacenza di w . Per ogni vicino di w , controlliamo istantaneamente nell'array se quel nodo era stato marcato da v . Se sì, incrementiamo il contatore dei nodi in comune.
- **Complessità Temporale:** $O(\text{grado}(v) + \text{grado}(w))$, che nel caso peggiore è limitato a $O(V)$. Molto più efficiente del doppio ciclo annidato.
- **Complessità Spaziale:** $O(V)$ per l'allocazione dell'array di marcatura.

```
#include <stdlib.h>

// Nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione per contare i nodi adiacenti sia a v che a w
int GRAPHcnt2(Graph g, int v, int w) {
    // Controllo di validità: il grafo deve esistere e i vertici devono essere nei
    limiti
    if (g.adj == NULL || v < 0 || w < 0 || v >= g.V || w >= g.V) return 0;

    // Alloco un array di marcatura grande V e lo inizializzo a zero usando calloc
    int* is_adj_to_v = (int*)calloc(g.V, sizeof(int));
    if (is_adj_to_v == NULL) return 0; // Controllo fallimento memoria

    int common_count = 0;

    // 1. Mappatura dei vicini di v
    EdgeNode* curr_v = g.adj[v];
    while (curr_v != NULL) {
        // Marco l'indice corrispondente al vicino di v come "visto" (1)
        is_adj_to_v[curr_v->v] = 1;

        curr_v = curr_v->next; // Avanzo nella lista di v
    }

    // 2. Intersezione con i vicini di w
    EdgeNode* curr_w = g.adj[w];
    while (curr_w != NULL) {
        // Se il vicino di w ha un 1 nell'array di marcatura, significa che è anche
        vicino di v
        if (is_adj_to_v[curr_w->v] == 1) {
```

```

        common_count++; // Trovato un nodo in comune, incremento il contatore
    }

    curr_w = curr_w->next; // Avanzo nella lista di w
}

// Libero la memoria di supporto allocata per evitare memory leak
free(is_adj_to_v);

return common_count;
}

```

Teoria (Camurati) :

Camurati

- formula generale equazioni alle ricorrenze per il divide et impera
- equazione alle ricorrenze del caso peggiore del quicksort
- MST, premesse del grafo di partenza e caratteristiche

Teoria (Camurati)

- **Formula generale delle equazioni di ricorrenza (Divide et Impera):**
 - **La Formula:** $T(n) = a * T(n/b) + f(n)$
 - **Significato dei termini:** * 'n' è la dimensione totale dell'input.
 - 'a' è il numero di sotto-problemi in cui il problema originale viene diviso (deve essere $a \geq 1$).
 - 'n/b' è la dimensione di ogni singolo sotto-problema (con $b > 1$, indica il fattore di rimpicciolimento).
 - 'f(n)' è il costo (in termini di Tempo) necessario per dividere l'input iniziale e per ricombinare le soluzioni parziali alla fine.
 - **Risoluzione:** Si risolve analiticamente tramite il Master Theorem, che confronta il costo della divisione/combinazione $f(n)$ con il costo di risoluzione delle foglie dell'albero ricorsivo (calcolato come $n^{\log_b(a)}$).
- **Equazione di ricorrenza del caso peggiore del Quicksort:**
 - **Il Contesto:** Si verifica quando la scelta del pivot genera costantemente la partizione più sbilanciata possibile (es. il pivot è sempre il massimo o il minimo, lasciando $n-1$ elementi da una parte e 0 dall'altra).
 - **La Formula:** $T(n) = T(n-1) + T(0) + O(n)$. Sapendo che il caso base $T(0)$ costa $O(1)$, si semplifica in: $T(n) = T(n-1) + O(n)$.

- **Conseguenze:** Risolvendo questa ricorrenza (sviluppando la sommatoria di n), l'albero di ricorsione degenera in una lista lineare. La complessità Temporale esplode a $O(n^2)$. La complessità Spaziale, legata allo stack di chiamate ricorsive che non si dimezza ma scala di uno alla volta, degenera a $O(n)$.
 - **MST (Minimum Spanning Tree): premesse e caratteristiche:**
 - **Premesse del grafo di partenza:** Affinché il problema abbia senso, il grafo di partenza deve obbligatoriamente essere **Non Orientato** (senza frecce), **Connesso** (deve esistere un percorso tra ogni coppia di vertici, altrimenti parleremmo di Minimum Spanning Forest) e **Pesato** (ogni arco deve avere un costo associato).
 - **Caratteristiche topologiche dell'MST:** È un sottografo che forma un albero. Di conseguenza, per definizione matematica, non contiene alcun ciclo chiuso, include esattamente tutti i V vertici del grafo originale, e utilizza esattamente $V-1$ archi.
 - **Caratteristica di ottimalità:** La somma totale dei pesi dei $V-1$ archi scelti è la minima assoluta possibile tra tutti gli alberi ricoprenti estraibili da quel grafo.
 - **Proprietà di unicità:** L'MST estratto è rigorosamente unico se e solo se tutti i pesi degli archi nel grafo di partenza sono valori distinti tra loro. Se ci sono pesi uguali, potrebbero esistere più MST equivalenti.
-

Data : 16.25 -> 20 (1.5 lab) 14/03/2025

Programmazione (Cabodi) :

Cabodi - FIFOinsert con vettore -> gestisco la fifo come lista circolare

Programmazione (Cabodi)

- **Logica dell'algoritmo (FIFO con Vettore Circolare):**
 - **Il Problema del vettore lineare:** Se usassimo un array normale, dopo una serie di inserimenti in coda ed estrazioni in testa, la coda "scivolerebbe" verso la fine dell'array. L'array sembrerebbe pieno anche se all'inizio ci sono celle vuote.
 - **La Soluzione (Coda Circolare):** Manteniamo due indici (head per estrarre, tail per inserire) e un contatore degli elementi (count). L'array viene trattato come

un anello chiuso: quando un indice raggiunge la fine fisica dell'array, riparte da zero.

- **Il trucco matematico:** L'avvolgimento circolare si ottiene elegantemente (senza usare if) con l'operatore Modulo (%). La formula $\text{tail} = (\text{tail} + 1) \% \text{size}$ fa avanzare l'indice e lo riporta a 0 non appena uguaglia la dimensione massima.
- **Complessità Temporale:** $O(1)$, poiché l'inserimento richiede solo accessi diretti all'array tramite indice e semplici assegnazioni.
- **Complessità Spaziale:** $O(1)$ ausiliario, in quanto lo spazio del vettore è preallocato nella struttura e non creiamo nuovi nodi.

```
#include <stdlib.h>

// Definizione della struttura Coda (FIFO) implementata con vettore
typedef struct {
    int* array; // Puntatore al vettore allocato dinamicamente
    int size;   // Capacità massima totale del vettore
    int count;  // Numero attuale di elementi presenti nella coda
    int head;   // Indice da cui si estrae (il più vecchio)
    int tail;   // Indice in cui si inserisce il prossimo elemento
} CircularQueue;

// Funzione per inserire un elemento (Enqueue). Ritorna 1 se successo, 0 se piena.
int FIFOinsert(CircularQueue* q, int value) {
    // Controllo di sicurezza base sulla struttura
    if (q == NULL || q->array == NULL) return 0;

    // Se il conteggio è uguale alla capacità, la coda è completamente piena
    if (q->count == q->size) {
        return 0; // Overflow: impossibile inserire
    }

    // Inserisco il valore nella cella puntata attualmente dalla coda (tail)
    q->array[q->tail] = value;

    // Avanzo l'indice della coda. L'operatore modulo (%) crea l'effetto circolare:
    // se tail arriva a 'size', il resto della divisione è 0, e tail riparte
    // dall'inizio.
    q->tail = (q->tail + 1) % q->size;

    // Incremento il contatore degli elementi attualmente in coda
    q->count++;

    return 1; // Inserimento avvenuto con successo
}
```


Teoria (Camurati) :

Camurati - Funzioni tail recursive, come si possono trasformare in iterative (for/while + variabili d'appoggio) e se questo si può fare con tutte le ricorsive - Powerset, numero di powerset partendo da n elementi (2^n)

Teoria (Camurati)

- **Funzioni Tail Recursive (Ricorsione in coda):**

- **Definizione:** Una funzione si dice "tail recursive" quando la chiamata ricorsiva è l'assoluta e ultima istruzione eseguita prima del return. Non ci sono operazioni matematiche o logiche in sospeso da eseguire dopo che la chiamata ricorsiva è terminata (es. $\text{return } n * \text{fattoriale}(n-1)$ NON è tail recursive perché c'è una moltiplicazione in sospeso).
- **Trasformazione in iterativa:** Proprio perché non ci sono operazioni in sospeso, lo stato della funzione chiamante non ha bisogno di essere salvato. Si può convertire banalmente in un ciclo while. Le variabili d'appoggio (i parametri della funzione) vengono semplicemente aggiornati con i nuovi valori prima di ripetere il ciclo, eliminando il sovraccarico delle chiamate a funzione.
- **TCO (Tail Call Optimization):** È una tecnica dei compilatori moderni (come GCC) che rileva le funzioni tail recursive e le trasforma in automatico in cicli iterativi a basso livello. Questo fa crollare la complessità Spaziale da $O(n)$ (per lo stack dei frame) a $O(1)$.
- **Si può fare con TUTTE le ricorsive?** Sì, dal punto di vista teorico qualsiasi funzione ricorsiva può essere resa iterativa. Tuttavia, se la funzione NON è tail recursive (es. visite degli alberi, ricorsione multipla), la conversione non è banale: richiede l'allocazione esplicita di una struttura dati Stack in memoria (array o lista) per simulare a mano ciò che il sistema operativo fa con il call stack.

- **Powerset (Insieme delle parti):**

- **Cos'è:** Dato un insieme di partenza di n elementi, il Powerset è l'insieme che contiene letteralmente tutti i possibili sottoinsiemi generabili, includendo sempre l'insieme vuoto e l'insieme originale completo.
- **Numero di elementi (2^n):** La cardinalità del Powerset è esattamente 2^n .
- **Perché proprio 2^n (Spiegazione logica per l'orale):** Pensa a ogni elemento dell'insieme come a un interruttore indipendente. Per costruire un sottoinsieme, per ogni elemento hai esattamente due scelte: "lo includo" (1) o

"lo escludo" (0). Avendo due scelte per ciascuno degli n elementi, le combinazioni totali sono $2 \times 2 \times 2$ ripetuto n volte, ovvero 2^n .

- **Complessità:** Per generare e stampare tutto il Powerset, la complessità Temporale è $O(n * 2^n)$ (poiché ci sono 2^n insiemi e la stampa/copia di ognuno costa fino a n). La complessità Spaziale è $O(n)$ per l'array booleano di supporto o per l'altezza dell'albero di ricorsione (backtracking).
-

Data : 15,50->??(1,75 lab) 12/03/2025

Programmazione (Cabodi) :

Cabodi: - controllare se un bst è bilanciato in altezza

Programmazione (Cabodi)

- **Logica dell'algoritmo (Verifica BST bilanciato in altezza):**
 - **Definizione:** Un albero è bilanciato in altezza (proprietà degli alberi AVL) se, per ogni singolo nodo, la differenza in valore assoluto tra l'altezza del suo sottoalbero sinistro e quella del suo sottoalbero destro è al massimo 1.
 - **Approccio ingenuo (da evitare):** Calcolare l'altezza su ogni nodo durante la visita costa Tempo $O(n^2)$ nel caso peggiorativo, perché i nodi vengono riattraversati più volte.
 - **Approccio ottimizzato (Bottom-up):** Si usa una singola visita in post-ordine (Sinistra-Destra-Radice). La funzione ricorsiva calcola e restituisce l'altezza reale del sottoalbero. Se durante la risalita scopre che uno sbilanciamento supera 1, restituisce un valore sentinella (es. -1) che viene propagato istantaneamente fino alla radice, bloccando ulteriori calcoli inutili.
 - **Complessità Temporale:** $O(n)$, dove n è il numero di nodi, poiché ogni nodo viene visitato esattamente una sola volta.
 - **Complessità Spaziale:** $O(h)$, dove h è l'altezza dell'albero, dovuta allo stack delle chiamate ricorsive (da $O(\log n)$ se bilanciato fino a $O(n)$ se degenerare).

```
#include <stdlib.h>
#include <stdbool.h>

// Definizione del nodo dell'albero binario di ricerca
typedef struct Node {
```

```

    int key;
    struct Node *left, *right;
} Node;

// Funzione di supporto per calcolare il massimo tra due interi
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Funzione ricorsiva che ritorna l'altezza se bilanciato, o -1 se sbilanciato
int check_balance_height(Node* root) {
    // Caso base: un albero vuoto è bilanciato e ha altezza 0
    if (root == NULL) return 0;

    // Scendo ricorsivamente nel sottoalbero sinistro
    int left_height = check_balance_height(root->left);

    // Se il sottoalbero sinistro ha segnalato uno sbilanciamento (-1), lo propago
    if (left_height == -1) return -1;

    // Scendo ricorsivamente nel sottoalbero destro
    int right_height = check_balance_height(root->right);

    // Se il sottoalbero destro ha segnalato uno sbilanciamento (-1), lo propago
    if (right_height == -1) return -1;

    // Controllo la condizione di bilanciamento sul nodo corrente
    int diff = left_height - right_height;
    if (diff > 1 || diff < -1) {
        return -1; // Sbilanciamento rilevato, ritorno la sentinella
    }

    // Se è bilanciato, calcolo la sua altezza reale e la ritorno al padre
    return max(left_height, right_height) + 1;
}

// Funzione wrapper pulita per il chiamante
bool is_balanced(Node* root) {
    // Se la funzione di supporto ritorna diverso da -1, l'albero è bilanciato
    return check_balance_height(root) != -1;
}

```

Teoria (Camurati) :

- teorema e corollario sugli archi sicuri
- powerset (quali sono i tre metodi per farlo), spiegare il metodo del divide et impera per il powerset

Teoria (Camurati)

- **Teorema degli Archi Sicuri:**

- **Scopo:** Fornisce la giustificazione matematica per l'uso di algoritmi Greedy (come Prim e Kruskal) nella ricerca del Minimum Spanning Tree (MST), garantendo che le scelte locali non pregiudichino l'ottimo globale.
- **Definizioni per l'orale:** Un "taglio" è una partizione astratta che divide i V vertici del grafo in due sottoinsiemi disgiunti. Se possediamo già un insieme di archi A' parzialmente costruito per l'MST, un taglio "rispetta A' " se la sua linea di divisione non spezza nessun arco di A' . L'"arco leggero" è l'arco con il peso minore tra tutti quelli che attraversano il taglio.
- **Enunciato:** Dato un sottoinsieme A' incluso in un MST e un taglio qualsiasi che rispetta A' , l'arco leggero che attraversa il taglio è "sicuro" per A' (cioè, aggiungendolo ad A' , il nuovo insieme fa ancora parte di un MST valido).

- **Corollario degli Archi Sicuri:**

- **Enunciato:** Durante la costruzione, l'insieme A' forma una "foresta" (componenti connesse separate). Se si considera una singola componente di questa foresta, l'arco più leggero in assoluto che la collega a qualsiasi altro nodo fuori da essa è un arco sicuro.
- **Legame pratico:** Giustifica l'efficienza degli algoritmi: permettono di trovare la soluzione in Tempo $O(E \log V)$ ignorando le esplosioni combinatorie $O(V^V)$.

- **Powerset (Insieme delle parti - I 3 metodi):**

- **Definizione:** L'insieme di tutti i possibili sottoinsiemi generabili da un insieme di N elementi. Contiene esattamente 2^N elementi, incluso l'insieme vuoto e se stesso.
- **Metodo 1 (Iterativo / Binario):** Si mappa ogni elemento a un bit. Contando da 0 a $(2^N)-1$, ogni numero binario generato (es. 101) rappresenta univocamente un sottoinsieme (es. prendo il primo elemento, salto il secondo, prendo il terzo).
- **Metodo 2 (Backtracking / Albero delle decisioni):** Si esplora uno spazio di stati in profondità. Per l'elemento i -esimo, l'algoritmo effettua due diramazioni ricorsive: una in cui l'elemento viene aggiunto al sottoinsieme parziale in costruzione, e una in cui viene ignorato.
- **Metodo 3 (Divide et Impera):**
 - **Logica:** Divide il problema a metà. Il Powerset di un insieme di N elementi è matematicamente uguale all'unione di due blocchi distinti: il Powerset dei primi $N-1$ elementi, unito allo stesso identico Powerset dei

primi $N-1$ elementi in cui però l' N -esimo elemento è stato forzatamente inserito in ogni singolo sottoinsieme.

- **Meccanismo ricorsivo:** La funzione chiede a se stessa di calcolare il Powerset dell'array privato dell'ultimo elemento. Una volta ricevuto questo risultato parziale (che contiene $2^{(N-1)}$ insiemi), l'algoritmo lo duplica. Al secondo gruppo duplicato, aggiunge l'elemento che era stato tenuto da parte. Infine, fonde i due gruppi e li restituisce.
 - **Complessità Temporale:** $O(N * 2^N)$. Ci sono 2^N sottoinsiemi totali da generare, e copiare un sottoinsieme di dimensione media $N/2$ costa tempo proporzionale a N .
 - **Complessità Spaziale:** $O(N * 2^N)$ se dobbiamo memorizzare in memoria l'intero Powerset finale come matrice o lista di liste, più $O(N)$ per lo stack della ricorsione.
-

Data : 15,25 -> (no lab) 12/03/2025

Programmazione (Cabodi) :

Cabodi:

- Scrivere la funzione di insert in radice di un BST

Gemini ha detto

Programmazione (Cabodi)

- **Logica dell'algoritmo (Inserimento in Radice in un BST):**
 - **Obiettivo:** Inserire un nuovo nodo in un Albero Binario di Ricerca (BST) forzandolo a diventare la nuova radice, ma mantenendo rigorosamente la proprietà strutturale del BST (minori a sinistra, maggiori a destra).
 - **Meccanismo (Ricorsione + Rotazioni):** L'inserimento diretto in radice è complesso perché richiede di smembrare e ricollegare l'intero albero. Si usa quindi un trucco elegante e asintoticamente equivalente: si sfrutta la ricorsione.
 - **Fase 1 (Discesa):** Si confronta la chiave da inserire con il nodo corrente e si scende verso le foglie esattamente come in un normale inserimento.

- **Fase 2 (Risalita e Galleggiamento):** Una volta creato il nodo in fondo (nel caso base), il sistema inizia a chiudere le chiamate ricorsive (Backtracking). Durante questa risalita, applichiamo una **Rotazione a Destra** se il nuovo nodo arriva dal sottoalbero sinistro, o una **Rotazione a Sinistra** se arriva dal destro. Questo fa letteralmente "galleggiare" il nuovo nodo livello per livello fino a portarlo in cima.
- **Complessità Temporale:** $O(h)$, dove h è l'altezza dell'albero ($O(\log V)$ nel caso ottimo/bilanciato, $O(V)$ nel caso pessimo di albero degenerare a lista).
- **Complessità Spaziale:** $O(h)$ per lo stack delle chiamate ricorsive impiegate per la discesa e la risalita.

```
#include <stdlib.h>

// Definizione del nodo del BST
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione helper per creare un nuovo nodo foglia
Node* create_node(int key) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (new_node != NULL) {
        new_node->key = key;
        new_node->left = NULL;
        new_node->right = NULL;
    }
    return new_node;
}

// Funzione di supporto: Rotazione a Destra
// Si usa quando il nodo da far salire si trova nel ramo sinistro
Node* rotR(Node* h) {
    Node* x = h->left;    // Il figlio sinistro x diventerà il nuovo padre
    h->left = x->right;    // Il sottoalbero destro di x passa a sinistra del vecchio
    padre
    x->right = h;          // Il vecchio padre diventa il figlio destro di x
    return x;             // Ritorno x che è la nuova radice del sottoalbero
}

// Funzione di supporto: Rotazione a Sinistra
// Si usa quando il nodo da far salire si trova nel ramo destro
Node* rotL(Node* h) {
    Node* x = h->right;   // Il figlio destro x diventerà il nuovo padre
    h->right = x->left;    // Il sottoalbero sinistro di x passa a destra del vecchio
    padre
    x->left = h;           // Il vecchio padre diventa il figlio sinistro di x
}
```

```

    return x;          // Ritorno x che è la nuova radice del sottoalbero
}

// Funzione principale ricorsiva per l'inserimento in radice
Node* insert_root(Node* root, int key) {
    // Caso base: se arrivo in fondo (o l'albero è vuoto), creo e ritorno il nuovo
    nodo
    if (root == NULL) {
        return create_node(key);
    }

    // Se la chiave è minore della radice corrente, devo inserirla a sinistra
    if (key < root->key) {
        // Scendo ricorsivamente. La chiamata mi restituirà il nodo appena inserito
        root->left = insert_root(root->left, key);

        // Durante la risalita, il nodo è ora mio figlio sinistro.
        // Lo faccio "galleggiare" verso di me tramite una rotazione a destra.
        root = rotR(root);
    }

    // Se la chiave è maggiore, devo inserirla a destra
    else if (key > root->key) {
        // Scendo ricorsivamente
        root->right = insert_root(root->right, key);

        // Durante la risalita, il nodo è mio figlio destro.
        // Lo faccio "galleggiare" tramite una rotazione a sinistra.
        root = rotL(root);
    }

    // Se key == root->key (duplicato), salto gli if e ritorno il nodo inalterato.

    // Ritorno la nuova radice aggiornata al chiamante superiore
    return root;
}

```

Teoria (Camurati) :

Camurati

- qual è la complessità dell'algoritmo di visita in profondità di un grafo(n^2 per matrice e $n+e$ per lista di adiacenze)
- Definizione di grafo connesso

Teoria (Camurati)

- **Complessità Visita DFS (Depth-First Search):**

- **Con Matrice di Adiacenze:** Complessità Temporale $O(V^2)$ (dove V è il numero di nodi, indicati con 'n' nella tua domanda).
 - **Perché:** Per ogni vertice che visitiamo (costo V), dobbiamo obbligatoriamente scansionare interamente la sua riga nella matrice per scoprire quali sono i suoi vicini. Siccome una riga della matrice è lunga V , facciamo V operazioni per ciascuno dei V vertici. Il costo totale è $V * V = V^2$.
 - **Con Lista di Adiacenze:** Complessità Temporale $O(V + E)$ (dove E è il numero di archi).
 - **Perché:** Il ciclo principale della DFS si assicura di chiamare la ricorsione su ogni vertice una sola volta (grazie al vettore dei "visitati"), dando un costo di base V . All'interno di ogni visita, scorriamo solo la lista concatenata degli archi che esistono realmente per quel nodo. In tutto l'algoritmo, attraverseremo ogni arco esattamente una volta (o due in grafi non orientati), dando un costo aggiuntivo E . Totale: $V + E$.
 - **Complessità Spaziale:** Per entrambe le rappresentazioni è $O(V)$, in quanto dobbiamo mantenere l'array dei nodi visitati e lo stack di sistema per le chiamate ricorsive (che nel caso pessimo di un grafo fatto a linea retta, impila V chiamate).
 - **Definizione di Grafo Connesso:**
 - **Nei Grafi Non Orientati:** Un grafo si definisce "connesso" se, presa una qualsiasi coppia di vertici 'u' e 'v' appartenenti al grafo, esiste sempre almeno un cammino (una sequenza di archi) che permette di viaggiare da 'u' a 'v'. Visivamente, significa che il grafo è costituito da un unico blocco coeso, senza isole o vertici isolati.
 - **Nei Grafi Orientati (Dettaglio extra per l'orale):** La definizione si sdoppia.
 - **Fortemente Connesso:** Se per ogni coppia (u, v) esiste un cammino che rispetta il senso delle frecce da 'u' a 'v', E contemporaneamente un cammino di ritorno da 'v' a 'u'.
 - **Debolmente Connesso:** Se il grafo risulta connesso solo se decidiamo di ignorare il verso delle frecce (trattandolo di fatto come un grafo non orientato).
-

Data : 16,5 -> 21 (1.25 lab) 12/03/2025

Programmazione (Cabodi) :

Cabodi:

- Dato un BST, calcolare la profondità massima e minima delle foglie.

Programmazione (Cabodi)

- **Logica dell'algoritmo (Profondità massima e minima delle foglie):**
 - **Meccanismo:** Si utilizza una visita in profondità (DFS, tipicamente in pre-ordine). Durante la discesa ricorsiva, si passa come parametro la profondità del nodo corrente (partendo da 0 per la radice).
 - **Identificazione delle foglie:** Un nodo è una foglia se entrambi i suoi puntatori (sinistro e destro) sono NULL.
 - **Aggiornamento:** Quando si raggiunge una foglia, si confronta la sua profondità con i valori di min_depth e max_depth (passati per riferimento tramite puntatori) e, se necessario, li si aggiorna. min_depth va inizializzato a un valore altissimo (es. INT_MAX), mentre max_depth a un valore bassissimo (es. -1).
 - **Complessità Temporale:** $O(n)$, dove n è il numero di nodi dell'albero. Ogni nodo viene visitato esattamente una volta.
 - **Complessità Spaziale:** $O(h)$, dove h è l'altezza dell'albero, dovuta all'occupazione di memoria dello stack delle chiamate ricorsive ($O(\log n)$ se l'albero è bilanciato, $O(n)$ se degenera).

```
#include <stdlib.h>
#include <limits.h>

// Definizione del nodo del BST
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione ricorsiva di supporto per l'attraversamento
void calculate_depths_recursive(Node* root, int current_depth, int* min_depth, int* max_depth) {
    // Caso base: se arrivo a un puntatore nullo, torno indietro
    if (root == NULL) {
        return;
    }

    // Controllo se il nodo corrente è una foglia (nessun figlio)
    if (root->left == NULL && root->right == NULL) {
```

```

        // Aggiorno la profondità minima se quella corrente è strettamente minore
        if (current_depth < *min_depth) {
            *min_depth = current_depth;
        }
        // Aggiorno la profondità massima se quella corrente è strettamente
maggiore
        if (current_depth > *max_depth) {
            *max_depth = current_depth;
        }
        return; // Ottimizzazione: una foglia non ha figli, inutile scendere oltre
    }

    // Se non è una foglia, continuo la discesa incrementando la profondità di 1
    calculate_depths_recursive(root->left, current_depth + 1, min_depth,
max_depth);
    calculate_depths_recursive(root->right, current_depth + 1, min_depth,
max_depth);
}

// Funzione principale/wrapper da richiamare nel main
void find_leaf_depths(Node* root, int* out_min, int* out_max) {
    // Gestione del caso limite: albero completamente vuoto
    if (root == NULL) {
        *out_min = 0;
        *out_max = 0;
        return;
    }

    // Inizializzo i valori puntati prima di far partire la ricorsione
    *out_min = INT_MAX; // Valore massimo possibile per i calcolatori
    *out_max = -1;      // Valore impossibile per forzare il primo aggiornamento

    // Faccio partire la visita dalla radice (profondità 0)
    calculate_depths_recursive(root, 0, out_min, out_max);
}

```

Teoria (Camurati) :

Camurati:

- Selezione di attività, quali algoritmi e approcci si possono usare per risolvere il problema (ho detto greedy, e brute force tramite powerset con combinazioni semplici). –

Che cos'è una partizione di un insieme, vari algoritmi per trovare le partizioni di un set (er, disp, ..)

Teoria (Camurati)

- **Selezione di attività (Activity Selection Problem):**

- **Il problema:** Dato un insieme di attività, ciascuna con un orario di inizio e uno di fine, trovare il sottoinsieme di dimensione massima composto da attività mutuamente compatibili (che non si sovrappongono nel tempo).
- **Approccio Greedy (La soluzione ottima e pratica):**
 - **Meccanismo:** Si ordinano in modo crescente tutte le attività in base al loro orario di **fine**. Si seleziona la prima attività dell'elenco ordinato. Successivamente, si itera sulle restanti e si seleziona la prossima attività il cui orario di inizio è maggiore o uguale all'orario di fine dell'ultima attività inserita.
 - **Perché funziona:** Scegliere l'attività che finisce prima (Proprietà della scelta Greedy) libera il maggior tempo possibile per tutte le attività successive, massimizzando il numero di inserimenti.
 - **Complessità:** Tempo $O(n \log n)$ per l'ordinamento iniziale, seguito da una singola passata $O(n)$. Spazio $O(1)$ ausiliario (se ordinate in-place).
- **Approccio Forza Bruta (Powerset):**
 - **Meccanismo:** Si ignora la logica Greedy e si enumerano sistematicamente tutte le combinazioni possibili generando il Powerset (l'insieme delle parti) delle N attività. Per ogni sottoinsieme generato, si controlla se ci sono sovrapposizioni. Alla fine si seleziona il sottoinsieme valido col numero maggiore di elementi.
 - **Complessità:** Totalmente inefficiente. Essendo il Powerset formato da 2^n sottoinsiemi, il Tempo esplode a $O(n * 2^n)$. Spazio $O(n)$ per generare il sottoinsieme ricorsivamente.
- **Partizione di un insieme:**
 - **Definizione:** È una suddivisione degli elementi di un insieme di partenza in un numero di "blocchi" (sottoinsiemi). Due regole fondamentali: nessun blocco può essere vuoto, e l'intersezione tra due blocchi qualsiasi deve essere vuota (devono essere mutuamente disgiunti). L'unione di tutti i blocchi restituisce l'insieme di partenza.
- **Algoritmo di Er (Generazione ottimizzata):**
 - **Meccanismo:** Utilizza le RGS (Restricted Growth Strings). Una RGS è un array di N interi dove il valore all'indice i indica in quale blocco è stato posizionato l'elemento i . La stringa inizia sempre con 0, e ogni elemento successivo non può superare il valore massimo degli elementi precedenti più uno. L'algoritmo incrementa sistematicamente le cifre da destra verso sinistra per generare tutte e sole le partizioni valide senza duplicati.

- **Complessità:** Tempo $O(B_n)$, dove B_n è il numero di Bell (crescita super-esponenziale asintoticamente ingestibile per N grandi). Spazio $O(n)$ per memorizzare la stringa RGS.
 - **Algoritmo tramite Disposizioni Semplici (Forza bruta ingenua):**
 - **Meccanismo:** Si sceglie un numero di blocchi fissato K . Si genera il modello combinatorio delle Disposizioni Semplici con ripetizione per assegnare gli N elementi ai K blocchi (generando di fatto stringhe di N elementi usando un alfabeto di K cifre).
 - **Il problema (Perché Er è migliore):** Le disposizioni generano configurazioni non valide (es. blocchi rimasti vuoti) e configurazioni simmetricamente identiche (es. rinominare il blocco 0 in blocco 1 genera la stessa partizione logica ma una stringa diversa). Richiede per forza una complessa funzione di "filtraggio" a posteriori per scartare le partizioni duplicate dividendo per $K!$ (le permutazioni dei blocchi).
 - **Complessità:** Tempo $O(K^n)$, un'esplosione combinatoria enormemente superiore al numero di Bell, rendendo questo approccio puramente didattico e mai applicato nella pratica. Spazio $O(n)$ per generare la singola disposizione nell'albero di ricorsione.
-

Data : 17,25->20 (1,75 lab) 11/03/2025

Programmazione (Cabodi) :

Cabodi : Il successore data una chiave in un bst

Programmazione (Cabodi)

- **Logica dell'algoritmo (Successore data una chiave in un BST):**
 - **Obiettivo:** Trovare il nodo con la chiave strettamente maggiore e più vicina alla chiave k fornita.
 - **Meccanismo (Ricerca Top-Down):** Invece di usare i puntatori al padre o fare una visita completa, si sfrutta la proprietà del BST scendendo dalla radice. Usiamo un puntatore succ (inizialmente NULL) per memorizzare il "miglior candidato" trovato finora.

- **Regola di aggiornamento:** Se la chiave del nodo corrente è strettamente maggiore di k , questo nodo è un potenziale successore. Lo salviamo in `succ` e ci spostiamo nel sottoalbero sinistro per cercare un candidato ancora più piccolo (ma sempre maggiore di k). Se invece la chiave corrente è minore o uguale a k , il successore deve per forza trovarsi nel sottoalbero destro, quindi ci spostiamo a destra senza aggiornare `succ`.
- **Complessità Temporale:** $O(h)$, dove h è l'altezza dell'albero. Nel caso ottimo/bilanciato è $O(\log n)$, nel caso pessimo (albero degenerare a lista) è $O(n)$. Si compie un solo cammino dalla radice verso una foglia.
- **Complessità Spaziale:** $O(1)$, l'algoritmo è puramente iterativo e alloca solo due puntatori di supporto, azzerando l'uso dello stack di sistema.

```
#include <stdlib.h>

// Definizione standard del nodo del BST
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

// Funzione che ritorna il puntatore al nodo successore data una chiave k
Node* bst_successor(Node* root, int k) {
    // Puntatore per tracciare il nodo corrente durante la discesa
    Node* current = root;
    // Puntatore per memorizzare l'ultimo potenziale successore trovato
    Node* succ = NULL;

    // Scorro l'albero finché non arrivo a una foglia (o trovo il posto vuoto)
    while (current != NULL) {

        // Se il nodo corrente ha una chiave strettamente maggiore di k
        if (current->key > k) {
            // È un candidato valido: lo salvo come potenziale successore
            succ = current;
            // Scendo a sinistra per cercare un candidato migliore (più piccolo)
            current = current->left;
        }
        // Se la chiave del nodo corrente è minore o uguale a k
        else {
            // Il successore non può essere qui, deve essere per forza a destra
            current = current->right;
        }
    }

    // Ritorno il miglior candidato trovato (sarà NULL se k è il massimo assoluto)
    return succ;
}
```

Teoria (Camurati) :

Camurati : Teoremi e corollari mst Codici di huffman

Teoria (Camurati)

- **Teoremi e Corollari MST (Minimum Spanning Tree):**

- **Contesto:** Costituiscono la base matematica che garantisce la correttezza degli algoritmi Greedy (Prim, Kruskal) per la costruzione dell'MST.
- **Definizioni preliminari:** Un "taglio" $(S, V-S)$ è una partizione dei vertici in due insiemi disgiunti. Se possediamo un insieme di archi A' (sottoinsieme di un MST), un taglio "rispetta A' " se nessun arco di A' attraversa il taglio. Un "arco leggero" è l'arco con il peso minimo in assoluto tra tutti quelli che attraversano il taglio.
- **Teorema degli Archi Sicuri:** Sia A' un sottoinsieme di archi incluso in un MST. Sia $(S, V-S)$ un taglio qualsiasi che rispetta A' . Sia (u, v) un arco leggero che attraversa questo taglio. Allora, l'arco (u, v) è "sicuro" per A' (ovvero, l'unione di A' con l'arco (u, v) è ancora un sottoinsieme valido di un MST ottimo).
- **Corollario degli Archi Sicuri:** Immagina l'insieme A' come una foresta (un insieme di alberi disgiunti). Se scegliamo una singola componente connessa (un albero) di questa foresta, l'arco più leggero in assoluto che collega questo albero a un vertice esterno (al resto del grafo) è sempre un arco sicuro.
- **Complessità associata:** Grazie a questo corollario, si evitano approcci a forza bruta (Tempo $O(V^2)$) per usare algoritmi polinomiali efficienti. Prim e Kruskal risolvono l'MST in Tempo $O(E \log V)$ e Spazio $O(V)$.

- **Codici di Huffman:**

- **Scopo:** È un algoritmo Greedy utilizzato per la compressione dei dati senza perdita di informazioni. Genera il codice binario ottimo per un determinato alfabeto basandosi sulla frequenza di comparsa dei caratteri.
- **Codice libero da prefissi (Prefix-free code):** Huffman genera codici in cui nessuna sequenza di bit usata per un carattere è il prefisso iniziale di un altro carattere. Questo elimina l'ambiguità e permette una decodifica istantanea leggendo il flusso di bit uno alla volta, senza necessità di separatori.
- **Meccanismo (Costruzione dell'albero):** Si crea un nodo foglia per ogni carattere, contenente la sua frequenza. Si inseriscono tutti i nodi in una Coda a Priorità (Min-Heap) ordinata per frequenza crescente. A ogni iterazione, si estraggono i due nodi con la frequenza minima. Si crea un nuovo nodo padre interno la cui frequenza è la somma dei due figli estratti. Si reinserisce il padre

nella Coda a Priorità. Il processo termina quando resta un solo nodo: la radice dell'Albero di Huffman.

- **Assegnazione dei codici:** Percorrendo l'albero dalla radice alle foglie, si assegna il bit '0' a ogni ramo sinistro e '1' a ogni ramo destro. Il cammino definisce il codice del carattere. I caratteri più frequenti si troveranno vicini alla radice (codici corti), quelli rari in profondità (codici lunghi).
 - **Complessità Temporale:** $O(n \log n)$, dove n è il numero di caratteri distinti nell'alfabeto. L'inserimento iniziale nell'Heap costa $O(n \log n)$ o $O(n)$ con heapify bottom-up; ci sono $n-1$ iterazioni, e in ognuna si fanno estrazioni e inserimenti nell'Heap che costano $O(\log n)$.
 - **Complessità Spaziale:** $O(n)$ per memorizzare i nodi dell'albero e la struttura della Coda a Priorità.
-

Data : 17.25 -> 21 (1.5 lab) 11/03/2025

Programmazione (Cabodi) :

Cabodi: ● Dato un grafo scrivere una funzione che verifica se un vertice è un punto di articolazione.

Programmazione (Cabodi)

- **Logica dell'algoritmo (Verifica se un vertice è Punto di Articolazione):**
 - **Definizione:** Un vertice è un punto di articolazione se la sua rimozione disconnette il grafo (assumendo che in partenza sia connesso).
 - **Approccio intuitivo (Perfetto per l'orale):** Invece di implementare il complesso algoritmo di Tarjan con i tempi di scoperta, per testare un *singolo* vertice v basta "spegnerlo" logicamente.
 - **Meccanismo:** Lancio una singola visita DFS partendo da un nodo qualsiasi diverso da v . Durante la DFS, se incontro v , lo ignoro (come se non esistesse). Se alla fine della DFS il numero di nodi visitati è esattamente $V - 1$ (tutti tranne v), il grafo è rimasto connesso e v NON è un punto di articolazione. Se i nodi visitati sono di meno, il grafo si è spezzato, quindi v è un punto di articolazione.
 - **Complessità Temporale:** $O(V + E)$, poiché eseguiamo una normale visita DFS ignorando un nodo.

- **Complessità Spaziale:** $O(V)$ per l'array dei visitati e lo stack di sistema della ricorsione.

```
#include <stdlib.h>

// Definizione del nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione ricorsiva DFS che conta i nodi raggiungibili ignorando un target
int dfs_count(Graph* g, int curr, int target, int* visited) {
    int count = 1; // Conto il nodo corrente
    visited[curr] = 1; // Lo marco come visitato

    EdgeNode* edge = g->adj[curr];

    // Scorro tutti i vicini del nodo corrente
    while (edge != NULL) {
        int neighbor = edge->v;

        // Se il vicino non è stato visitato E NON è il nodo da ignorare, proseguo
        // la visita
        if (!visited[neighbor] && neighbor != target) {
            count += dfs_count(g, neighbor, target, visited);
        }

        edge = edge->next; // Passo al prossimo arco
    }

    return count; // Ritorno il totale dei nodi scoperti in questo ramo
}

// Funzione principale: verifica se 'target' è un punto di articolazione
// Assunzione: il grafo di partenza è interamente connesso
int is_articulation_point(Graph* g, int target) {
    if (g == NULL || target < 0 || target >= g->V || g->V <= 1) return 0;

    // Alloco e inizializzo a zero l'array dei visitati
    int* visited = (int*)calloc(g->V, sizeof(int));
    if (visited == NULL) return 0;

    // Scelgo un nodo di partenza per la DFS che sia diverso dal target
```



```

int start_node = (target == 0) ? 1 : 0;

// Lancio la DFS e conto quanti nodi riesco a raggiungere senza passare per
'target'
int reached_nodes = dfs_count(g, start_node, target, visited);

free(visited);

// Se i nodi raggiunti sono strettamente minori del totale dei nodi rimanenti
(V - 1),
// significa che la rimozione del target ha disconnesso il grafo.
if (reached_nodes < (g->V - 1)) {
    return 1; // VERO: è un punto di articolazione
} else {
    return 0; // FALSO: la connettività è intatta
}
}

```

Teoria (Camurati) :

Camurati: ● DAG ● Cammino di Hamilton. ● Per i cammini massimi si può applicare la programmazione dinamica?

Teoria (Camurati)

- **DAG (Directed Acyclic Graph):**
 - **Cos'è:** È un grafo orientato (con archi che hanno una direzione) in cui è matematicamente impossibile formare un ciclo chiuso. Non esiste alcun percorso che parta da un vertice e ritorni allo stesso vertice seguendo il verso delle frecce.
 - **Proprietà fondamentale:** Ogni DAG possiede almeno un Ordinamento Topologico (una disposizione lineare dei vertici tale che ogni arco punti rigorosamente in avanti).
 - **Complessità:** La verifica dell'aciclicità (ricerca di Back-edge, ovvero archi che puntano a un antenato nella DFS) e l'ordinamento topologico costano entrambi Tempo $O(V + E)$ e Spazio $O(V)$.
- **Cammino di Hamilton:**
 - **Cos'è:** È un cammino semplice all'interno di un grafo (orientato o non orientato) che visita **tutti** i vertici del grafo esattamente una e una sola volta. Se il cammino si chiude tornando al vertice di partenza, prende il nome di Ciclo Hamiltoniano.

Per i cammini massimi si può applicare la Programmazione Dinamica?

- **Nei grafi generici (NO):** In un grafo con cicli, il problema del "cammino semplice di peso massimo" NON gode della Proprietà della Sottostruttura Ottima (requisito vitale per la programmazione dinamica). Scegliere un sottocammino lunghissimo locale potrebbe costringerti a visitare vertici che ti impediranno di raggiungere la destinazione finale senza creare cicli (che sono vietati in un cammino semplice). È un problema NP-Arduo.
 - **Nei DAG (Sì):** L'unica eccezione in cui la Programmazione Dinamica funziona perfettamente è sui DAG. Essendo privi di cicli per definizione, il rischio di "incastrarsi" scompare e la sottostruttura ottima è garantita.
 - **Meccanismo sui DAG:** Si calcola l'Ordinamento Topologico dei vertici. Si scorrono i vertici in questo ordine e, per ogni vertice, si "rilassano" i suoi archi uscenti aggiornando le distanze massime (es. $\text{dist}[\text{destinazione}] = \max(\text{dist}[\text{destinazione}], \text{dist}[\text{sorgente}] + \text{peso})$).
 - **Complessità (Sui DAG):** Tempo $O(V + E)$ per l'ordinamento topologico e il rilassamento lineare. Spazio $O(V)$ per memorizzare le distanze massime parziali.
-

Data : 17.50 -> 21 (0 lab) 11/03/2025

Programmazione (Cabodi) :

Cabodi: ● Scrivi algoritmi BFS

Programmazione (Cabodi)

- **Logica dell'algoritmo (BFS - Visita in Ampiezza):**
 - **Obiettivo:** Esplorare un grafo procedendo per "livelli" concentrici a partire da un nodo sorgente: prima tutti i vicini a distanza 1, poi quelli a distanza 2, ecc.
 - **Meccanismo:** Si utilizza una Coda (struttura dati FIFO) per gestire l'ordine di esplorazione e un array di marcatura per tenere traccia dei nodi già scoperti. Il nodo viene marcato come visitato *nel momento stesso in cui viene inserito in coda*, per evitare che altri nodi vicini lo accodino una seconda volta creando duplicati e cicli infiniti.
 - **Trucco per l'orale:** Invece di scrivere una complessa struttura dati Coda con i nodi concatenati, all'esame simula la Coda allocando un semplice array grande

V e usando due indici interi (head per estrarre, tail per inserire). È elegantissimo, compatto e non richiede dipendenze esterne.

- **Complessità Temporale:** $O(V + E)$ con liste di adiacenza, poiché ogni vertice viene accodato/estratto una sola volta e ogni arco viene ispezionato esattamente una volta (due nei grafi non orientati).
- **Complessità Spaziale:** $O(V)$ per allocare sia l'array dei visitati sia la finta coda di supporto.

```
#include <stdlib.h>

// Nodo della lista di adiacenza
typedef struct EdgeNode {
    int v;
    struct EdgeNode* next;
} EdgeNode;

// Struttura del grafo
typedef struct {
    int V;
    EdgeNode** adj;
} Graph;

// Funzione principale della visita in ampiezza (BFS)
void BFS(Graph* g, int start) {
    // Controllo di sicurezza su grafo inesistente o nodo di partenza fuori dai
    limiti
    if (g == NULL || start < 0 || start >= g->V) return;

    // Alloco e azzerò l'array di marcatura per i nodi scoperti
    int* visited = (int*)calloc(g->V, sizeof(int));

    // Simulo una Coda (FIFO) allocando un array statico grande al massimo V nodi
    int* queue = (int*)malloc(g->V * sizeof(int));

    // Indici per la gestione della finta coda
    int head = 0; // Puntatore di estrazione
    int tail = 0; // Puntatore di inserimento

    // Inizializzazione: scopro il nodo sorgente e lo accodo
    visited[start] = 1;
    queue[tail++] = start;

    // Il ciclo prosegue finché ci sono elementi da elaborare nella coda
    while (head < tail) {

        // Estraggo il nodo corrente dalla testa della coda e avanzo l'indice
        int u = queue[head++];
```

```

// Posiziono un puntatore sulla lista dei vicini del nodo appena estratto
EdgeNode* edge = g->adj[u];

// Ispezione tutti gli archi uscenti dal nodo 'u'
while (edge != NULL) {
    int v = edge->v;

    // Se il vicino 'v' non è mai stato scoperto prima
    if (visited[v] == 0) {

        // Lo marco ISTANTANEAMENTE come visitato per evitare accodamenti
        visited[v] = 1;

        // Lo inserisco in coda al livello corrente
        queue[tail++] = v;
    }

    // Passo al prossimo vicino nella lista concatenata
    edge = edge->next;
}

// Libero la memoria di supporto al termine della visita
free(visited);
free(queue);
}

```

Teoria (Camurati) :

Camurati: ● Programmazione dinamica e memoization

Teoria (Camurati)

- **Programmazione Dinamica:**

- **Cos'è:** Paradigma di progettazione per problemi di ottimizzazione. Divide un problema complesso in sottoproblemi, li risolve una sola volta e ne memorizza il risultato per evitare ricalcoli inutili.
- **Requisiti fondamentali:** Applicabile solo se il problema gode della "Sottostruttura Ottima" (la soluzione ottima globale si costruisce assemblando le soluzioni ottime locali) e della "Sovrapposizione dei Sottoproblemi" (l'algoritmo ingenuo ricalcolerebbe gli stessi identici sottoproblemi innumerevoli volte, es. Fibonacci).
- **Complessità (Impatto):** Abbassa drasticamente la complessità Temporale (spesso da un tempo esponenziale $O(2^n)$ a uno polinomiale $O(n)$ o $O(n^2)$),

sacrificando memoria (complessità Spaziale che sale a $O(n)$ o $O(n^2)$ per le strutture dati di supporto).

- **Memoization (Approccio Top-Down):**

- **Meccanismo:** È una delle due tecniche della programmazione dinamica (l'altra è la tabulazione Bottom-Up). Mantiene l'intuitiva struttura ricorsiva del Divide et Impera, ma introduce una memoria o "cache" (tipicamente un array o una hash table) inizializzata con valori sentinella (es. -1).
- **Come funziona:** Prima di lanciare la costosa chiamata ricorsiva, la funzione consulta la cache. Se all'indice corrispondente al sottoproblema c'è un valore valido, lo restituisce all'istante (Tempo $O(1)$). Se non c'è, esegue il calcolo, lo salva nella cache, e poi lo restituisce al chiamante.
- **Vantaggi:** Rispetto alla tabulazione, calcola esclusivamente i sottoproblemi strettamente necessari per raggiungere la soluzione, ignorando quelli irraggiungibili o inutili nello spazio degli stati.

Data : 15->18 (1,25 lab) 11/03/2025

Programmazione (Cabodi) :

Cabodi: -heapInsert

Logica dell'algoritmo (Inserimento in un Max-Heap):

- **Fase 1 (Inserimento in coda):** Per mantenere intatta la proprietà strutturale dell'Heap (un albero "quasi completo" riempito da sinistra verso destra), il nuovo elemento viene sempre inserito nell'ultima posizione disponibile dell'array (in fondo). Si incrementa quindi la dimensione logica dell'Heap.
- **Fase 2 (Sift-Up / Rialita):** Ora bisogna ripristinare la proprietà di ordinamento (il padre deve essere maggiore o uguale al figlio). Si calcola matematicamente l'indice del padre $(i - 1) / 2$. Se il nuovo nodo è maggiore del padre, violando la regola, lo si fa "galleggiare" verso l'alto scambiandolo col padre. Il processo si ripete finché il nodo non diventa minore del suo nuovo padre o finché non raggiunge la radice.
- **Trucco di ottimizzazione (No Swap):** Invece di fare continui e costosi scambi (che richiedono 3 operazioni ciascuno), si salva il nuovo valore da parte. Si fanno "scivolare" i padri verso il basso liberando il posto, e alla fine si piazza il nuovo valore direttamente nella sua posizione definitiva.

- **Complessità Temporale:** $O(\log n)$, poiché nel caso peggiore il nuovo elemento deve risalire per tutta l'altezza dell'albero fino alla radice.
- **Complessità Spaziale:** $O(1)$, poiché il processo di risalita viene implementato iterativamente (con un while), senza usare ricorsione o strutture d'appoggio.

```
#include <stdlib.h>

// Funzione per inserire un elemento in un Max-Heap. Ritorna 1 se successo, 0 se pieno.
int heapInsert(int* heap, int* size, int capacity, int value) {
    // Controllo di sicurezza: se la dimensione logica ha raggiunto la capacità fisica, fermo tutto
    if (*size >= capacity) {
        return 0; // Overflow
    }

    // Il nuovo elemento partirà dall'ultima foglia disponibile (indice uguale all'attuale size)
    int i = *size;

    // Aggiorno immediatamente la dimensione totale dell'Heap
    (*size)++;

    // SIFT-UP (Risalita ottimizzata senza swap continui)
    // Il ciclo continua finché non sono arrivato alla radice (i != 0)
    // E finché il valore del padre è strettamente minore del nuovo valore che voglio inserire
    while (i != 0 && heap[(i - 1) / 2] < value) {

        // Il padre è troppo piccolo: lo faccio scivolare giù nella posizione corrente del figlio
        heap[i] = heap[(i - 1) / 2];

        // Aggiorno l'indice 'i' spostandomi matematicamente al livello del padre appena analizzato
        i = (i - 1) / 2;
    }

    // Uscito dal ciclo, ho trovato il "buco" corretto. Inserisco fisicamente il valore.
    heap[i] = value;

    return 1; // Inserimento completato con successo
}
```

Teoria (Camurati) :

Camurati: -programmazione dinamica -stackframe

❓ Programmazione Dinamica:

- **Definizione:** È un paradigma di ottimizzazione che scompone un problema complesso in sottoproblemi più piccoli, li risolve rigorosamente una sola volta e ne salva il risultato in memoria per riutilizzarlo istantaneamente in futuro.
- **Requisiti per l'applicazione:** Il problema deve presentare la "Sottostruttura Ottima" (la soluzione ottima globale si costruisce assemblando le soluzioni ottime locali dei sottoproblemi) e la "Sovrapposizione dei Sottoproblemi" (l'algoritmo ricorsivo di base finirebbe per ricalcolare lo stesso nodo dell'albero delle decisioni innumerevoli volte).
- **Tecniche implementative:** Si applica tramite "Memoization" (approccio Top-Down: si mantiene la ricorsione ma si controlla prima una cache) o tramite "Tabulazione" (approccio Bottom-Up: si elimina la ricorsione e si riempie iterativamente una tabella partendo dai casi base).
- **Complessità:** Scambia spazio con tempo. Fa crollare la complessità Temporale da tempi intrattabili (es. esponenziale $O(2^n)$) a tempi polinomiali (es. lineare $O(n)$ o quadratico $O(n^2)$). Di contro, fa salire la complessità Spaziale a $O(n)$ o $O(n^2)$ per allocare la matrice o l'array di cache.

❓ Stackframe (Record di Attivazione):

- **Definizione:** È un blocco di memoria allocato automaticamente sullo Stack di sistema (Call Stack) ogni singola volta che viene invocata una funzione.
 - **Cosa contiene:** Memorizza le variabili locali della funzione, i parametri che le sono stati passati, lo stato dei registri del chiamante e, cosa più importante, l'Indirizzo di Ritorno (Return Address), ovvero l'esatta istruzione di memoria a cui il programma deve tornare una volta che la funzione esegue il return.
 - **Meccanismo di vita (LIFO):** Viene creato (Pushed) in cima allo Stack all'inizio della chiamata e viene distrutto e rimosso (Popped) non appena la funzione termina.
 - **Impatto sulla ricorsione:** È il motivo fisico per cui la ricorsione consuma memoria. Ogni chiamata ricorsiva impila un nuovo Stackframe. L'altezza massima raggiunta dall'albero di ricorsione determina la complessità Spaziale. Se manca il caso base o la ricorsione è troppo profonda, lo Stack si riempie fino a esaurire la memoria, causando il fatale errore di Stack Overflow.
 - **TCO (Tail Call Optimization):** Se la funzione è "Tail Recursive" (la chiamata a se stessa è l'assoluta ultima istruzione eseguita, senza calcoli in sospenso), i compilatori moderni non creano nuovi Stackframe, ma sovrascrivono e riutilizzano sempre lo stesso, abbattendo la complessità Spaziale da $O(n)$ a $O(1)$.
-

Data :

Programmazione (Cabodi) :

Teoria (Camurati) :
